

**BDBT – tworzenie aplikacji do
zarządzania i obsługi baz danych**

Tomasz Mrozek
Instytut Telekomunikacji
Wydział Elektroniki i Technik Informacyjnych



Wstęp

- Przedstawienie procesów tworzenia prostej aplikacji webowej wg. wzorca MVC w Spring Boot w Eclipse IDE. W analogiczny sposób przedstawione etapy tworzenia aplikacji można zaimplementować w IntelliJ IDE lub innym środowisku programistycznym.

UWAGA! Przedstawiona aplikacja ma prostą formę, jednak prezentuje wszystkie niezbędne mechanizmy w omawianej technologii. **W ramach projektu w części 2 stworzona aplikacja powinna być dopasowana do stworzonej bazy danych w 1 części projektu, oraz bardziej rozbudowana i dopracowana pod kątem designu.**

Zakres prezentacji

- Zastosowane technologie
- Wymagane oprogramowanie
- Tworzenia aplikacji webowej
- Testowanie działania aplikacji webowej poza środowiskiem Eclipse IDE

Zastosowane technologie

- Spring Boot z wbudowanym serwerem Tomcat, technologia lekka, prosta, szybka i łatwa w użyciu, idealna dla małych projektów
- Spring MVC (wzorzec) do zarządzania warstwami aplikacji webowej (Controller layer) jako standard w aplikacji webowej Spring
- Spring JDBC (JdbcTemplate) do pracy z bazami Oracle, pozwala na komunikację z relacyjną bazą danych (Oracle) za pośrednictwem JdbcTemplate. Można użyć Spring JDBC do prostych operacji CRUD na pojedynczej tabeli.
- Thymeleaf dla warstwy widoku (View layer) zamiast klasycznego JSP. Thymeleaf jest bardziej nowoczesny, prostszy i łatwiejszy w użyciu niż JSP i JSTL.
- Sterownik Oracle JDBC, do pracy z bazą danych (np. po jej zbudowaniu)
- JUnit – do tworzenia testów jednostkowych

Spring Boot

Zanim powstał projekt Spring Boot, programiści używając frameworka Spring musieli sami dbać o zachowanie prawidłowych zależności pomiędzy dodawanymi bibliotekami. Częstym problemem podczas konfiguracji API była niekompatybilność dołączanych bibliotek.

Dodatkowym wyzwaniem była konieczność posiadania zewnętrznego serwera (np.: Tomcat), który pozwalał na wgranie aplikacji i weryfikację poprawności jej działania. Konfiguracja, wgrywanie, uruchamianie oraz testowanie aplikacji było bardzo pracochłonne.

Z tego względu grupa developerów wspierających rozwój *Spring Framework* postanowiła skrócić tworzenie nowej aplikacji.

Spring Boot

W celu rozwiązania wcześniej zdefiniowanych problemów, programiści stworzyli tzw. startery. W uproszczeniu – startery to zbiór wszystkich wymaganych bibliotek oraz konfiguracji. Posiada również wbudowany serwer aplikacyjny Tomcat. Aby zacząć używać Spring Boot wystarczy:

- dodać odpowiednie zależności do pliku konfiguracyjnego pom.xml (w przypadku używania Maven),
- dodać „rodzica”, po którym aplikacja będzie dziedziczyć zależności.

Spring Boot

Spring Boot jest to rozwiązanie convention-over-configuration bazujące na Springu. Główne korzyści jakie dostarcza ten framework to:

- Łatwość uruchomienia – paczka projektu posiada wbudowany serwer i inne niezbędne komponenty, które są potrzebne do uruchomienia aplikacji.
- Automatyczna konfiguracja – do uruchomienia zasadniczej aplikacji nie jest wymagana żadna dodatkowa konfiguracja. Dodanie własnej jest opcjonalne dla osiągnięcia innych/nowych korzyści.
- Szybkość – tworzenie aplikacji z wykorzystaniem Spring Boot jest uproszczone co przekłada się szybszy i łatwiejszy proces developmentu.

Spring MVC

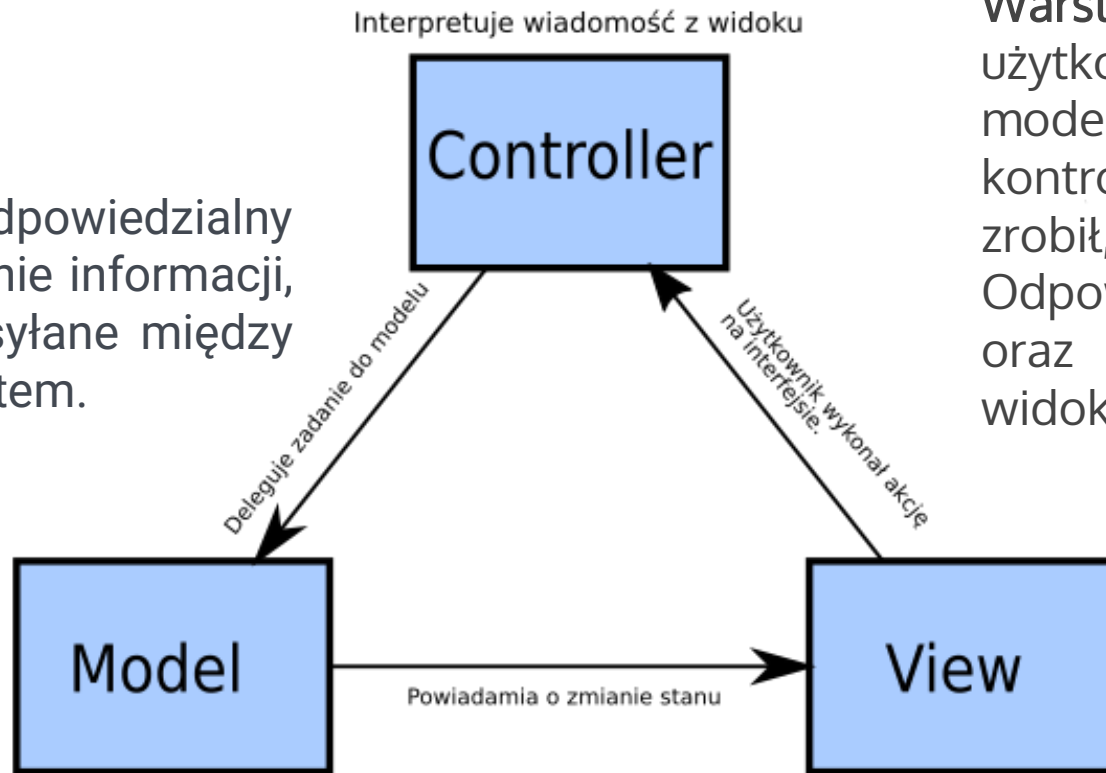
Spring Web MVC, potocznie zwany jako Spring MVC to potężny framework webowy stanowiący jedną ze składowych całego Spring Framework-a. Kluczowe funkcjonalności Spring MVC:

- Dyspozytor serwletu (DispatcherServlet),
- Mapowanie żądań (Request Mapping),
- Metody obsługi żądań (Handler methods),
- Obsługa wyjątków (Exceptions handling).

Spring MVC

Model-View-Controller to wzorec złożony, którego przeznaczeniem jest podzielić aplikację na trzy odseparowane od siebie warstwy:

Model – jest odpowiedzialny za przechowywanie informacji, które będą przesyłane między serwerem a klientem.



Warstwa kontrolera - odbierającą żądania użytkownika(z widoku) i przekazującą je do modelu, oraz aktualizującą widoki. Dzięki kontrolerowi model wie, że użytkownik coś zrobił, jednocześnie nie wiedząc nic o widoku. Odpowiedzialny jest za uzupełnienie modelu oraz zwrócenie klientowi odpowiedniego widoku.

Warstwa widoku - definiuje sposób reprezentacji danych, ostatecznie jest to plik HTML wyświetlany użytkownikowi, jednak zanim do tego dojdzie zostaje zazwyczaj wypełniany odpowiednimi danymi z modelu;

Spring JDBC

Spring JDBC (JdbcTemplate) służy do pracy z bazami Oracle i pozwala na komunikację z relacyjną bazą danych (Oracle). Jest interfejsem upraszczającym wykonywanie zapytań za pomocą interfejsu JDBC. Dodatkowo pozwala wyeliminować podstawowe błędy przy wykorzystaniu czystego JDBC. Dzięki użyciu JdbcTemplate kod jest bardziej zwięzły i przejrzysty.

Spring JDBC pozwala na wykorzystywanie prostych operacji CRUD (create, read, update, delete) na pojedynczej tabeli.

Spring Thymeleaf

Thymeleaf to silnik szablonów umożliwiający budowę elementów interfejsu użytkownika po stronie serwera (backendu). Thymeleaf jest bardziej nowoczesny, prostszy i łatwiejszy w użyciu niż JSP i JSTL.

Pozwala definiować własne atrybuty szablonów lub znaczników, jednak domyślnie zawiera standardowe dialekty (Standard i SpringStandard), które definiują bogaty zestaw funkcjonalności.

Standardowe dialekty są łatwe do rozpoznania, gdyż zawierają atrybuty zaczynające się od przedrostka th.

Sterownik Java DataBase Connectivity

Sterownik JDBC – zestaw klas, które implementują interfejsy pakietu `java.sql` dla konkretnego systemu baz danych.

Jest interfejsem programistycznym niskiego poziomu wywołującym bezpośrednio polecenia języka SQL, skonstruowanym w Javie i przeznaczonym dla języka Java.

JUnit – testy jednostkowe

JUnit to testy jednostkowe, który docelowo sprawdzają, czy dana funkcjonalność działa zgodnie z jej przeznaczeniem.

Dobłą praktyką jest dzielenie JUnitów, tak aby testowały one tylko wybraną metodę lub funkcjonalność, zaczynając od tych najprostszych, do coraz bardziej zaawansowanych. Testy powinny być od siebie niezależne.

Wymagane oprogramowanie

- Java Development Kit (JDK), środowisko niezbędne do programowania w języku Java,
 - Java Runtime Environment (JRE), środowisko uruchomieniowe dla programów napisanych w języku Java,
 - Eclipse IDE for Enterprise Java Developers lub IntelliJ IDEA lub inne,
 - Oracle Database (np. Oracle Database 18c Express Edition),
 - Oracle SQL Developer
-
- Sterownik JDBC8 (dla JDK8, 9, 11) lub JDBC 10 (dla JDK10, 11)

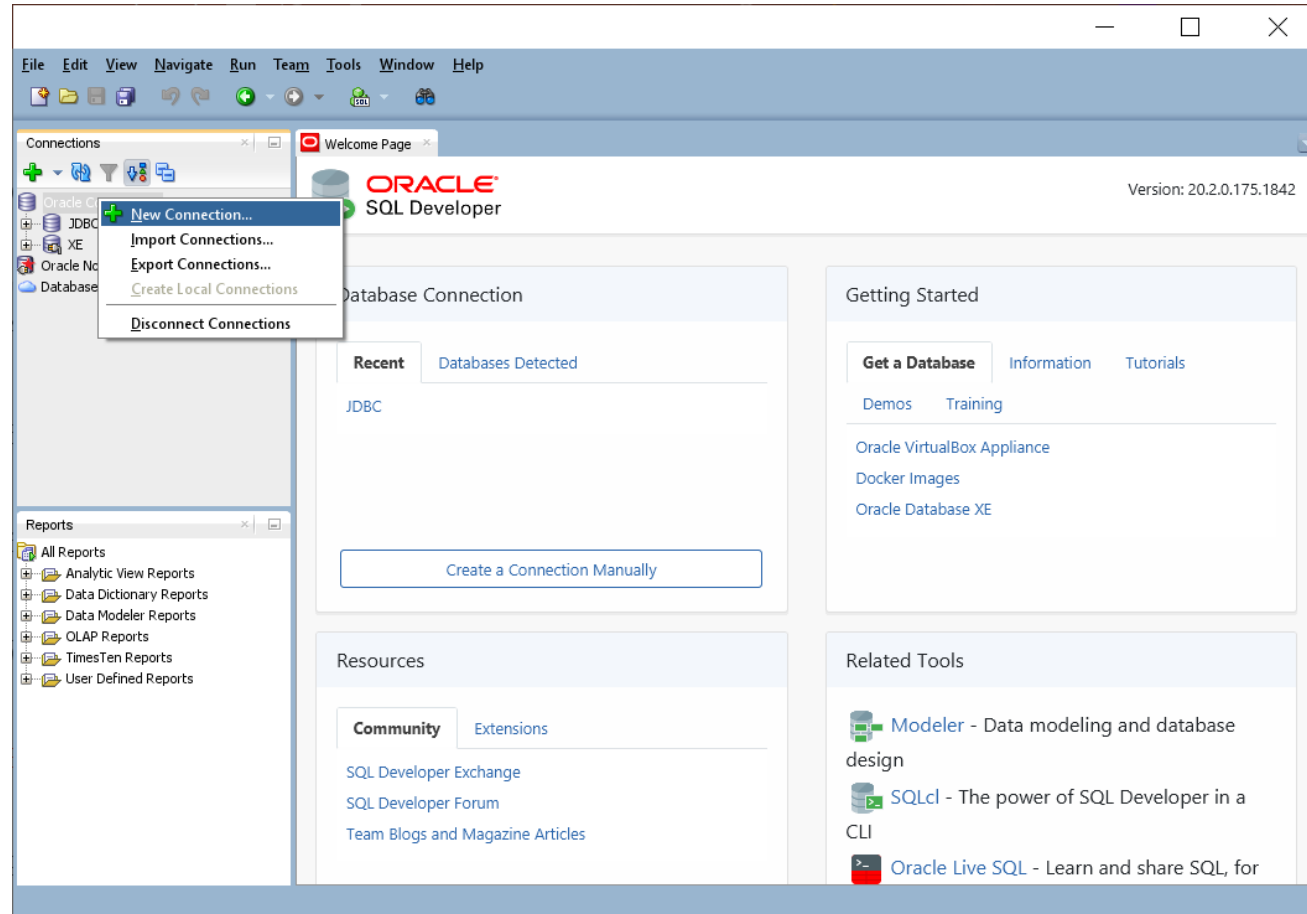
Tworzenie aplikacji

1. Tworzenie/wykorzystanie (stworzonej) bazy danych w Oracle,
2. Stworzenie projektu Spring Boot w Eclipse,
3. Określenie właściwości połączenia z bazą danych,
4. Stworzenie podstawowych relacji w kodzie,
5. Implementacja podstawowego designu w HTML,
6. Dodanie funkcjonalności: dodawania, edytowania, usuwania rekordów z bazy danych,
7. Uruchomienie stworzonej aplikacji webowej do zarządzania bazą danych poza projektem w środowisku Java.

1

TWORZENIE BAZY DANYCH W ORACLE

1. Tworzenie bazy danych w Oracle



1. Tworzenie bazy danych w Oracle

New / Select Database Connection

Connection Name: ...
JDBC ...
XE ...

Name: XE Color

Database Type: Oracle

User Info Proxy User

Authentication Type: Default

Username: system Role: default

Password: ☒ Save Password

Connection Type: Basic

Details Advanced

Hostname: 192.168.0.14

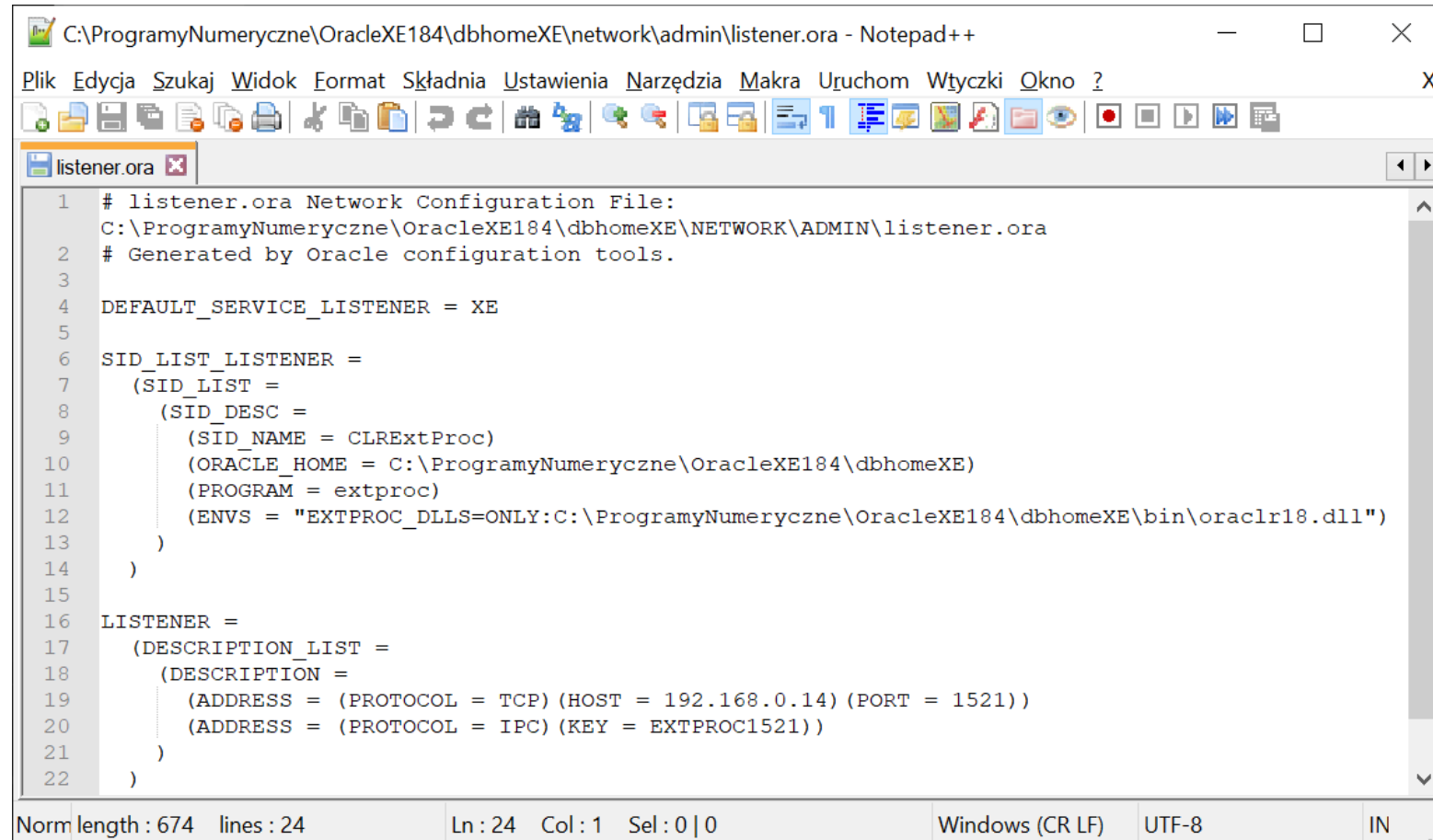
Port: 1521

☒ SID: xe

☐ Service name:

Status : Pomoc Save Clear Test Connect Anuluj

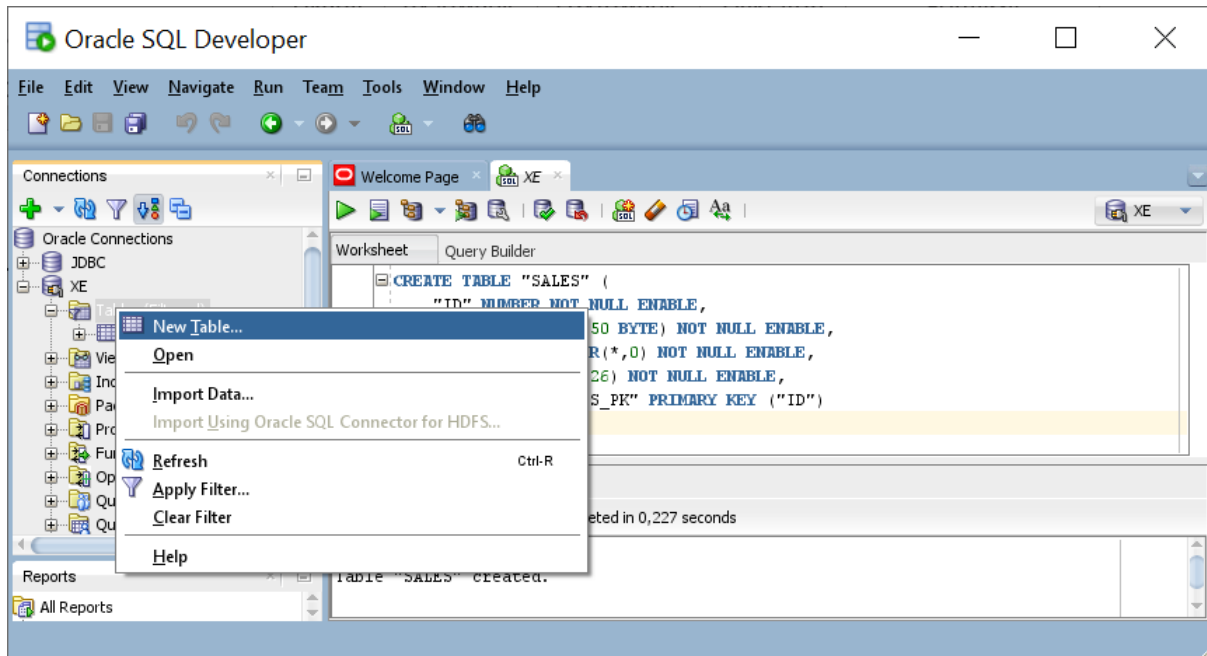
1. Tworzenie bazy danych w Oracle



```
1 # listener.ora Network Configuration File:
2 C:\ProgramyNumeryczne\OracleXE184\dbhomeXE\NETWORK\ADMIN\listener.ora
3 # Generated by Oracle configuration tools.
4
5 DEFAULT_SERVICE_LISTENER = XE
6
7 SID_LIST_LISTENER =
8     (SID_LIST =
9         (SID_DESC =
10             (SID_NAME = CLRExtProc)
11             (ORACLE_HOME = C:\ProgramyNumeryczne\OracleXE184\dbhomeXE)
12             (PROGRAM = extproc)
13             (ENVS = "EXTPROC_DLLS=ONLY:C:\ProgramyNumeryczne\OracleXE184\dbhomeXE\bin\oraclr18.dll")
14         )
15     )
16
17 LISTENER =
18     (DESCRIPTION_LIST =
19         (DESCRIPTION =
20             (ADDRESS = (PROTOCOL = TCP) (HOST = 192.168.0.14) (PORT = 1521))
21             (ADDRESS = (PROTOCOL = IPC) (KEY = EXTPROC1521))
22         )
23     )
```

Norm length : 674 lines : 24 Ln : 24 Col : 1 Sel : 0 | 0 Windows (CR LF) UTF-8 IN

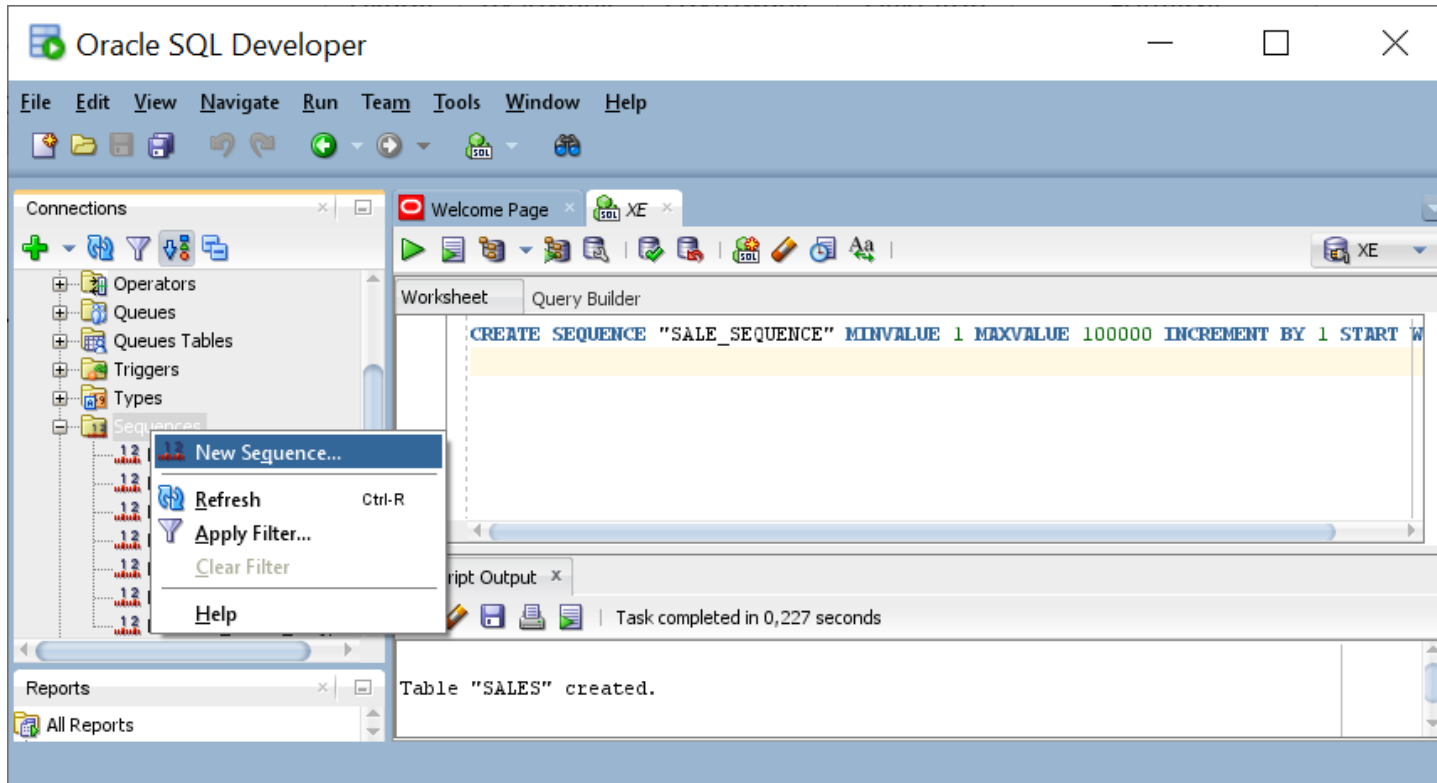
1. Tworzenie bazy danych w Oracle (tabela)



Tworzenie bazy danych przez obszar roboczy:

```
CREATE TABLE "SALES" (  
  "ID" NUMBER NOT NULL ENABLE,  
  "ITEM" VARCHAR2(50 BYTE) NOT NULL ENABLE,  
  "QUANTITY" NUMBER(*,0) NOT NULL ENABLE,  
  "AMOUNT" FLOAT(126) NOT NULL ENABLE,  
  CONSTRAINT "SALES_PK" PRIMARY KEY ("ID")  
)
```

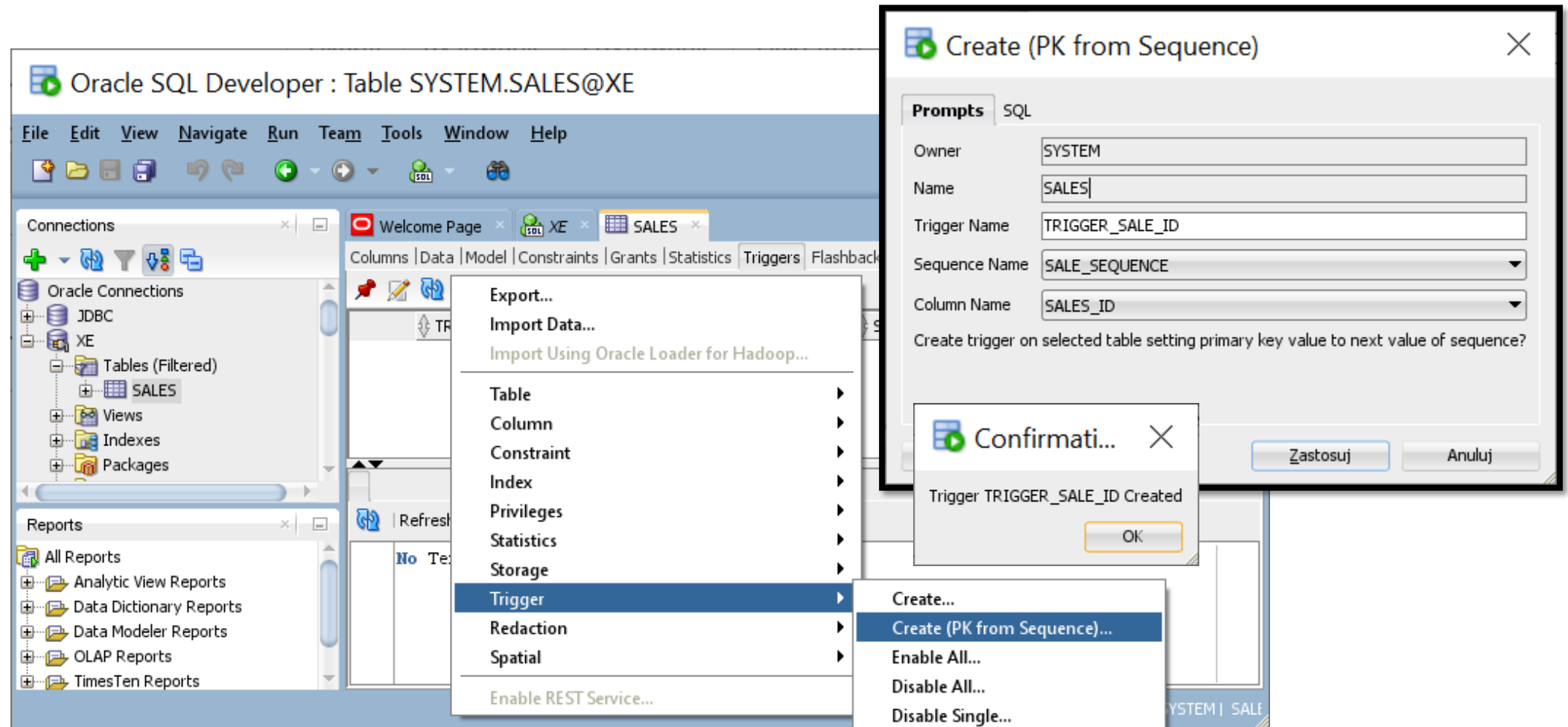

1. Tworzenie bazy danych w Oracle



Sekwencja auto-iteracji:

```
CREATE SEQUENCE "SALE_SEQUENCE"  
MINVALUE 1 MAXVALUE 100000 INCREMENT  
BY 1 START WITH 1;
```

1. Tworzenie bazy danych w Oracle (wyzwalacz)



2

TWORZENIE PROJEKTU SPRING BOOT

2. Tworzenie projektu Spring Boot (Maven)

Maven to najpopularniejsze narzędzia automatyzujące proces tworzenia aplikacji napisanych w Java. Innym takim narzędziem jest Gradle.

Maven zajmuje się dwoma aspektami tworzenia oprogramowania – sposobem, w jaki oprogramowanie jest tworzone oraz jego zależnościami.

2. Tworzenie projektu Spring Boot (Maven)

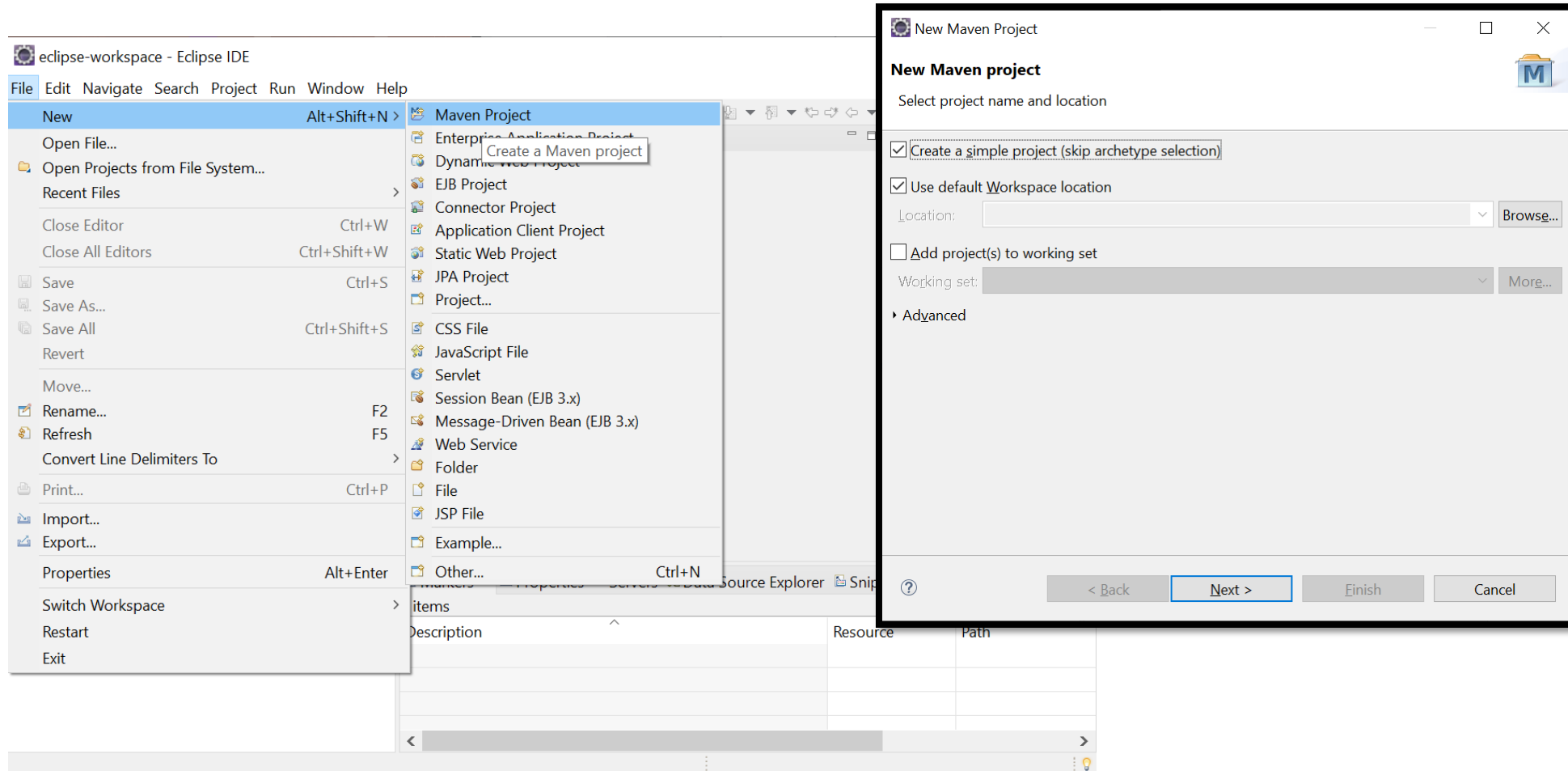
Plik XML opisuje budowany projekt aplikacji, zawiera on:

- zależności do innych zewnętrznych modułów i komponentów,
- kolejność budowy,
- katalogi i wymagane wtyczki,
- zdefiniowane cele do wykonania zdefiniowanych zadań, takich jak kompilacja kodu i jego pakowanie (packaging).

Maven dynamicznie pobiera biblioteki Java i wtyczki z jednego lub więcej repozytoriów (np. Maven Central Repository) i przechowuje je w lokalnej pamięci podręcznej (tzw. katalog m2).

Przede wszystkim zawiera on informacje o projekcie, wersji Java, zależności do zewnętrznych narzędzi w sekcji *dependencies* oraz informacji na temat budowy projektu w ramach znacznika *build*.

2. Tworzenie projektu Spring Boot



2. Tworzenie projektu Spring Boot

New Maven Project
Configure project

Artifact

Group Id: bada_bdbt_proj

Artifact Id: SalesManager

Version: 0.0.1-SNAPSHOT

Packaging: jar

Name:

Description:

Parent Project

Group Id: org.springframework.boot

Artifact Id: spring-boot-parent

Version: 2.4.0 Browse... Clear

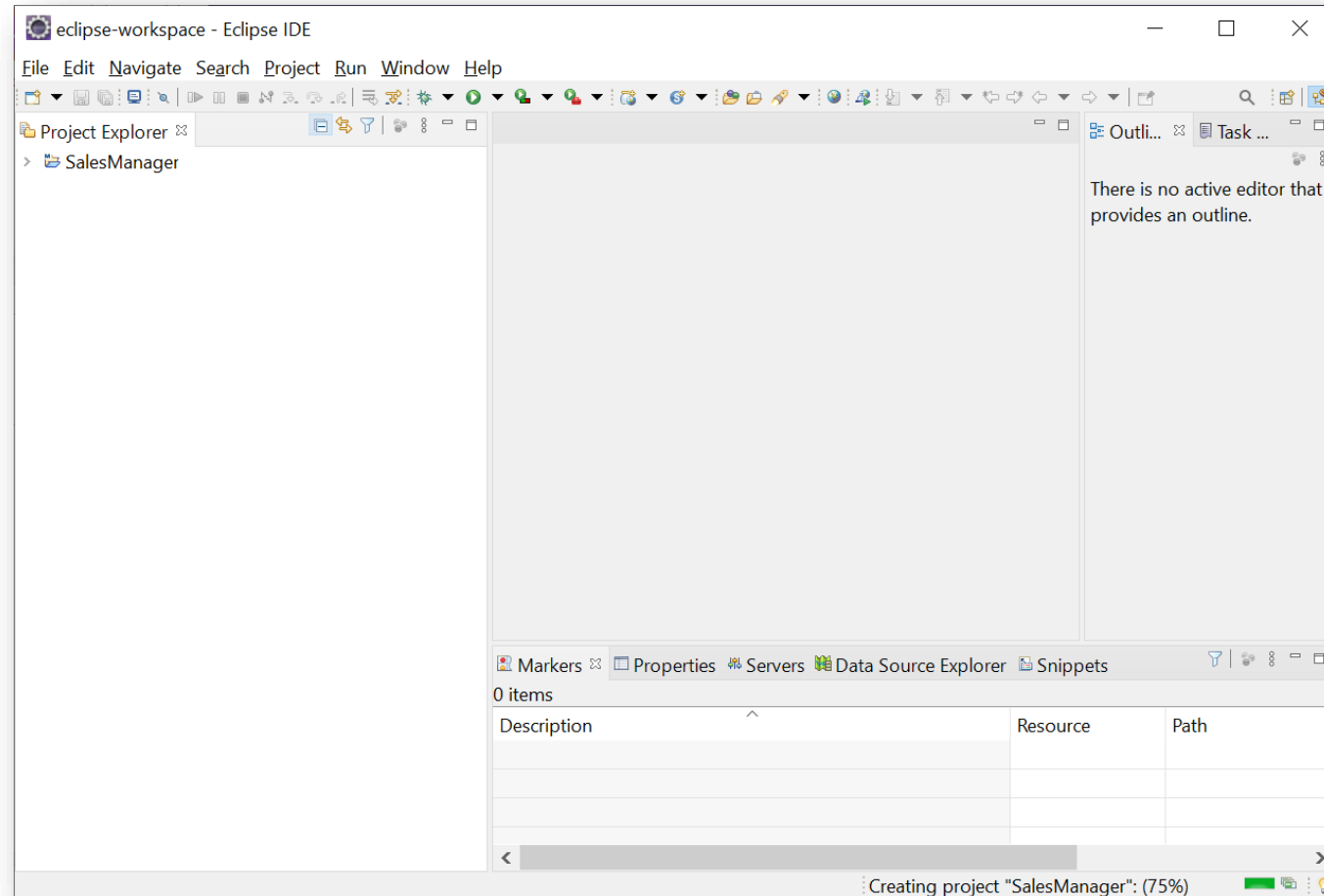
▸ Advanced

? < Back Next > Finish Cancel

Tworząc nowy projekt w Mavenie możemy wybrać jego 'packaging', czyli sposób, w jaki wszystkie elementy zostaną ostatecznie połączone razem.

Packaging JAR powoduje, że w momencie uruchomienia budowania Maven wszystkie nasze klasy zostaną połączone w jeden plik JAR (który można następnie uruchamiać samodzielnie).

2. Tworzenie projektu Spring Boot



2. Tworzenie projektu Spring Boot (struktura projektu)

Elementy struktury projektu:

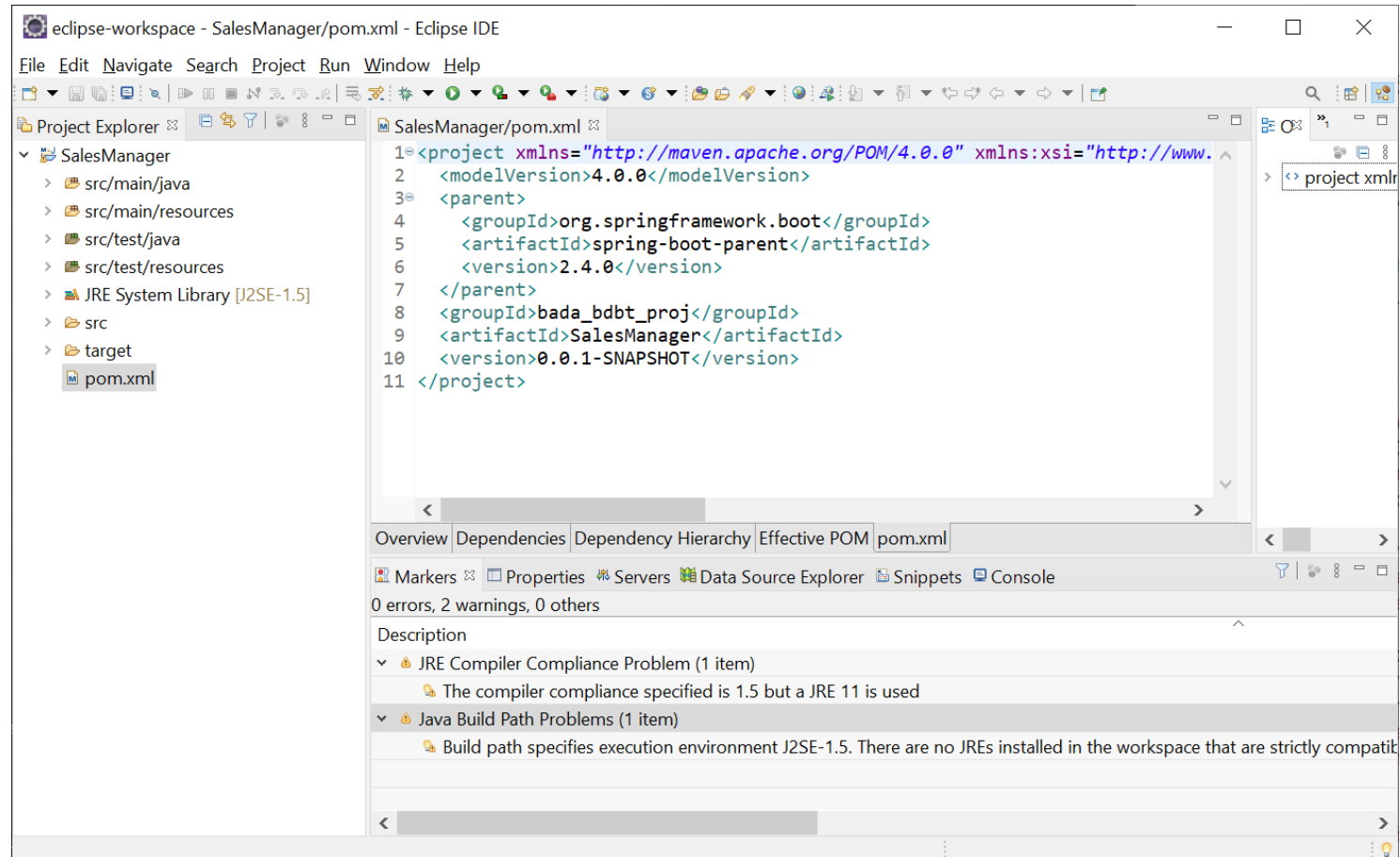
- pom.xml — główny plik konfiguracji Maven,
- /src/main — katalog, w którym znajdują się pliki tworzonego programu, są tam dwa podkatalogi:
 - java — tutaj trafiają wszystkie klasy (cały kod naszego modułu),
 - resources — tutaj będą wszystkie pliki, które nie są kodem, np. grafiki, pliki XML, konfiguracje itp.
- w przypadku projektów webowych będziemy mieli także katalog webapp, który jest używany do umieszczania wszystkich treści webowych

2. Tworzenie projektu Spring Boot (struktura projektu)

- /src/test — ma podobną strukturę jak katalog /main z tą różnicą, że jest on wykorzystywany tylko w trakcie automatycznych testów,
- /target — tutaj trafia skompilowany projekt (czyli np. w postaci wykonywalnego pliku JAR lub aplikacji webowej WAR). Zawartość katalogu target jest pakowana do archiwów paczek, które są gotowe do wdrożenia na serwer.

Poza wymienionymi w projekcie elementami struktury mogą się znaleźć dodatkowe pliki i katalogi, nie są one jednak najczęściej ściśle związane z samym Mavenem .

2. Tworzenie projektu Spring Boot (co to jest plik pom.xml)



2. Pom.xml

Plik pom.xml jest tym, co łączy projekt w Javie oraz Maven'a. Zawiera całą konfigurację związaną z projektem, opis jak ma być budowany, konfiguracja pluginów itp.

Plik pom.xml musi zawierać koordynaty artefaktu, czyli groupId oraz artifactId. Identyfikator grupy to najczęściej nazwa projektu, natomiast identyfikator artefaktu to unikalna nazwa np. modułu.

Idea jest taka, że poszczególne moduły tego samego projektu mają wspólny groupId, ale unikalny artifactId. Para wartości groupId i artifactId powinna być unikalna.

2. Pom.xml – informacje o projekcie

Informacje o projekcie, w tym między innymi nazwa artefaktu oraz numer jego wersji (szczegóły w tabelce pod listingiem):

Nazwa taga	Znaczenie
modelVersion	numer modelu zgodny z numerem w przestrzeni nazw
groupId	grupa projektowa - najczęściej odwrócona nazwa domeny firmy, dla której pracujemy, a jeśli to nasz prywatny projekt to możemy użyć innego formatu, np. <nazwisko>.<imie>
artifactId	nazwa identyfikująca artefakt - najczęściej nazwa naszego projektu
version	numer wersji naszego artefaktu - w najprostszej wersji jest to zbiór trzech cyfr rozdzielonych kropkami, natomiast w prawdziwym projekcie biznesowym do tego numeru doklejane są jeszcze postfixy oznaczające etap w jakim znajduje się kod projektu (np. SNAPSHOT, M1, RC1, RELEASE, Final)

2. Pom.xml - dependencies

Plik pom.xml pozwala nam także określić, z jakich bibliotek i innych modułów korzysta nasz moduł.

Służy do tego tag **<dependencies>**, który może wystąpić tylko raz. Wewnątrz tego tagu możemy dodać wiele tagów **<dependency>**. Tag dependency może wskazywać na dowolny artefakt z repozytorium centralnego, lokalnego lub dowolnego innego skonfigurowanego w ustawieniach Maven. Tag musi zawierać trzy informacje:

- groupId, artifactId (razem to tzw. coordinates — czyli precyzyjne określenie, o który artefakt nam chodzi),
- version — wersję.

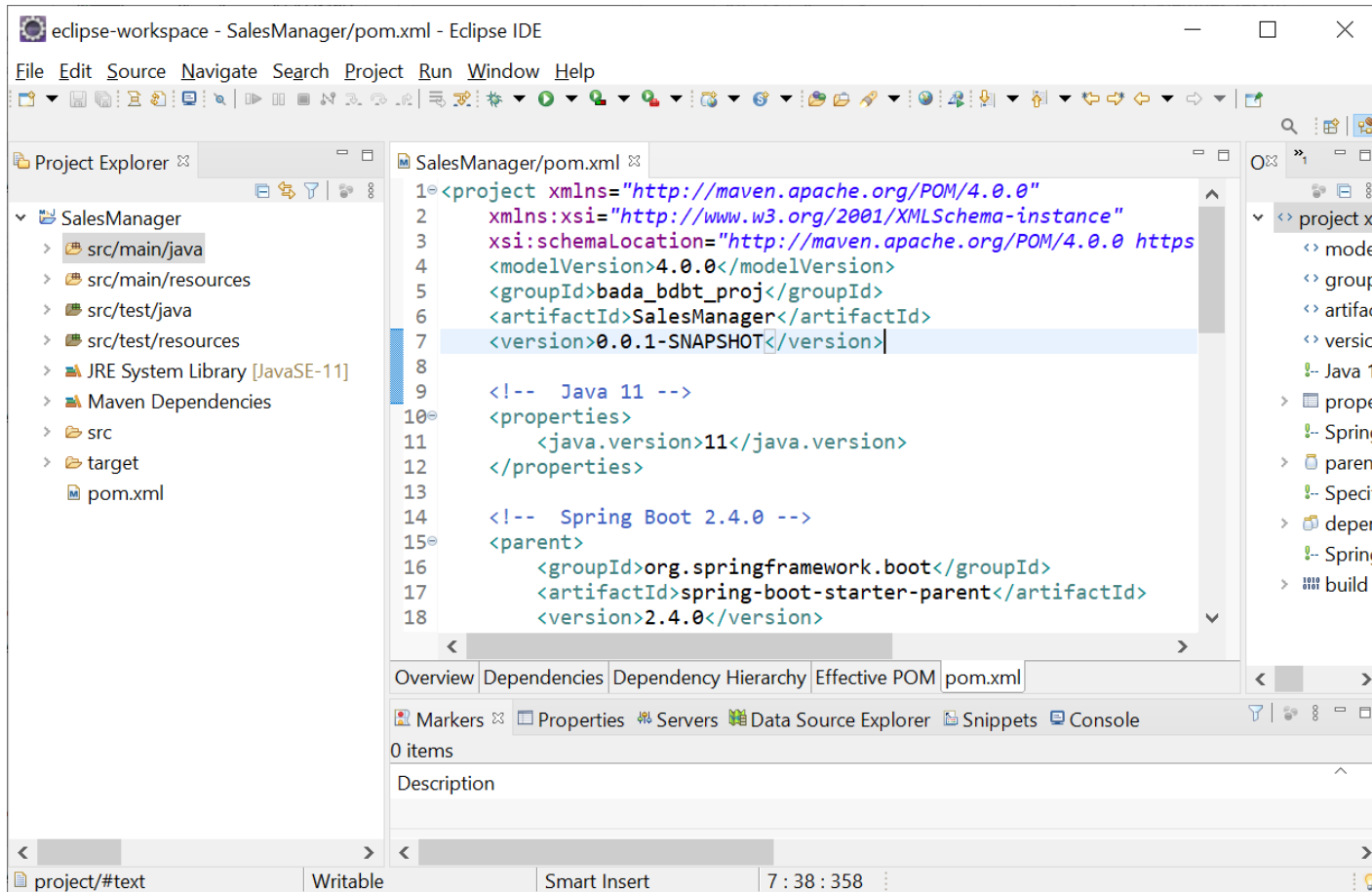
2. Pom.xml - build

Sekcja dodatkowych opcji uruchamianych podczas procesu budowy projektu (możemy tutaj na przykład dodać i skonfigurować wtyczki)

2. Pom.xml - repositories

Sekcja, w której możemy podać repozytoria zewnętrzne, na których Maven będzie szukał zewnętrznych artefaktów, z których korzysta projekt. Jest ona dość często pomijana, ze względu na to, że repozytoria częściej definiujemy globalnie dla wielu projektów poprzez plik z ustawieniami zewnętrznymi settings.xml.

2. Tworzenie projektu Spring Boot – wykorzystane startery



Startery – definiują domyślną konfigurację, oraz wszystkie potrzebne do działania wybranego startera zależności w wymaganych wersjach.

2. Tworzenie projektu Spring Boot – wykorzystane startery

- Spring Boot w wersji 2.4.0 – inicjalizacja i podanie wersji Spring Boot
- Spring JDBC (spring-boot-starter-jdbc) (starter do JDBC z HikariCP – CP to Connection Pool, jest to bardzo szybkie i lekkie narzędzie do łączenia z Javą, do operowania na bazach SQL)
- Spring MVC (spring-boot-starter-web) (służy do budowania aplikacji opartych na REST API z wykorzystaniem Spring MVC z wbudowanym serwerem Tomcat)
- Thymeleaf (spring-boot-starter-thymeleaf)

Aby używać sterownika Oracle JDBC trzeba ściągnąć go ręcznie:

<https://www.oracle.com/database/technologies/appdev/jdbc-ucp-19-8-c-downloads.html>

Jego lokalizację należy wskazać w pliku pom.xml

2. Tworzenie projektu Spring Boot - Info

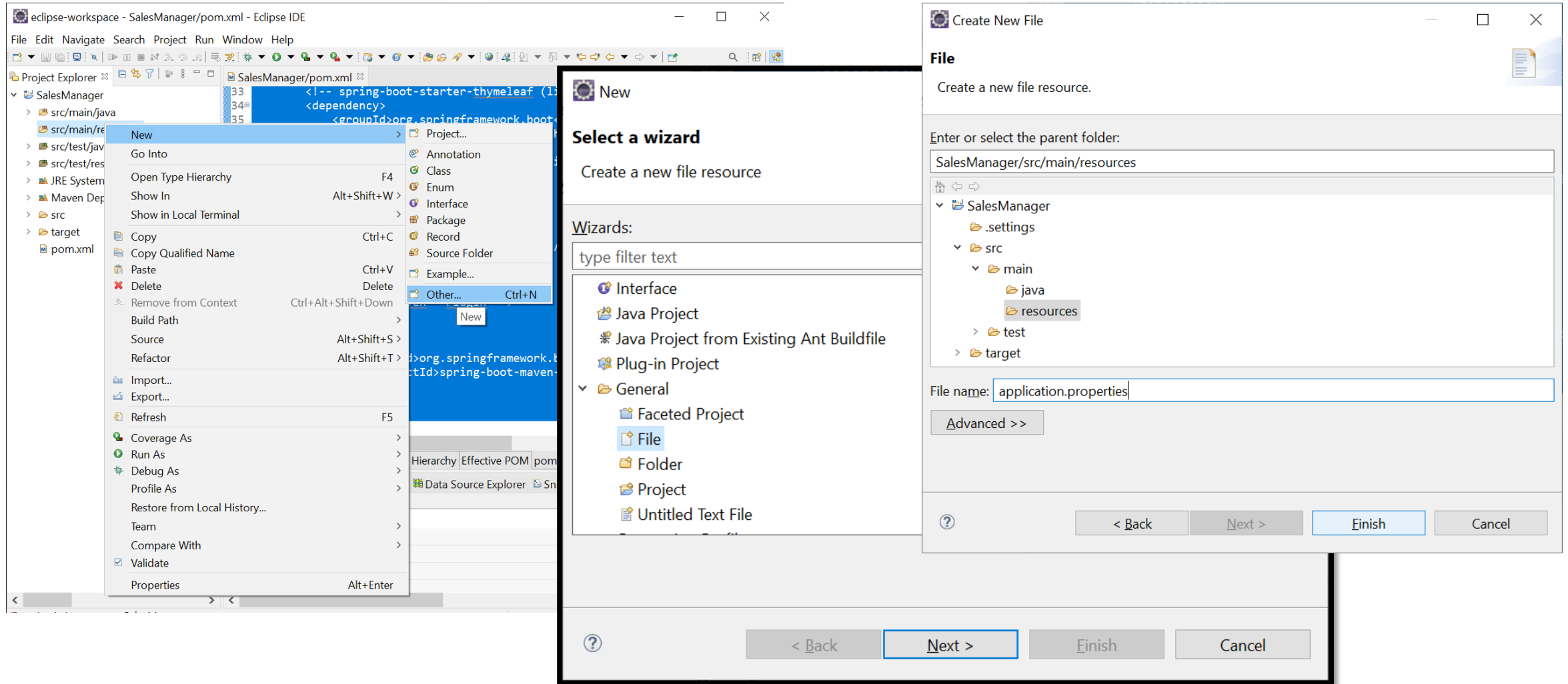
Plugin Spring Boot do Mavena (użycie pluginu):

- spring-boot-maven-plugin

3

OKREŚLENIE WŁAŚCIWOŚCI POŁĄCZENIA Z BAZĄ ORACLE

3. Określenie właściwości połączenia z bazą Oracle



3. Określenie właściwości połączenia z bazą Oracle

Zawartość pliku **properties**:

spring.datasource.url=jdbc:oracle:thin:@192.168.0.14:1521:xe

spring.datasource.username=system

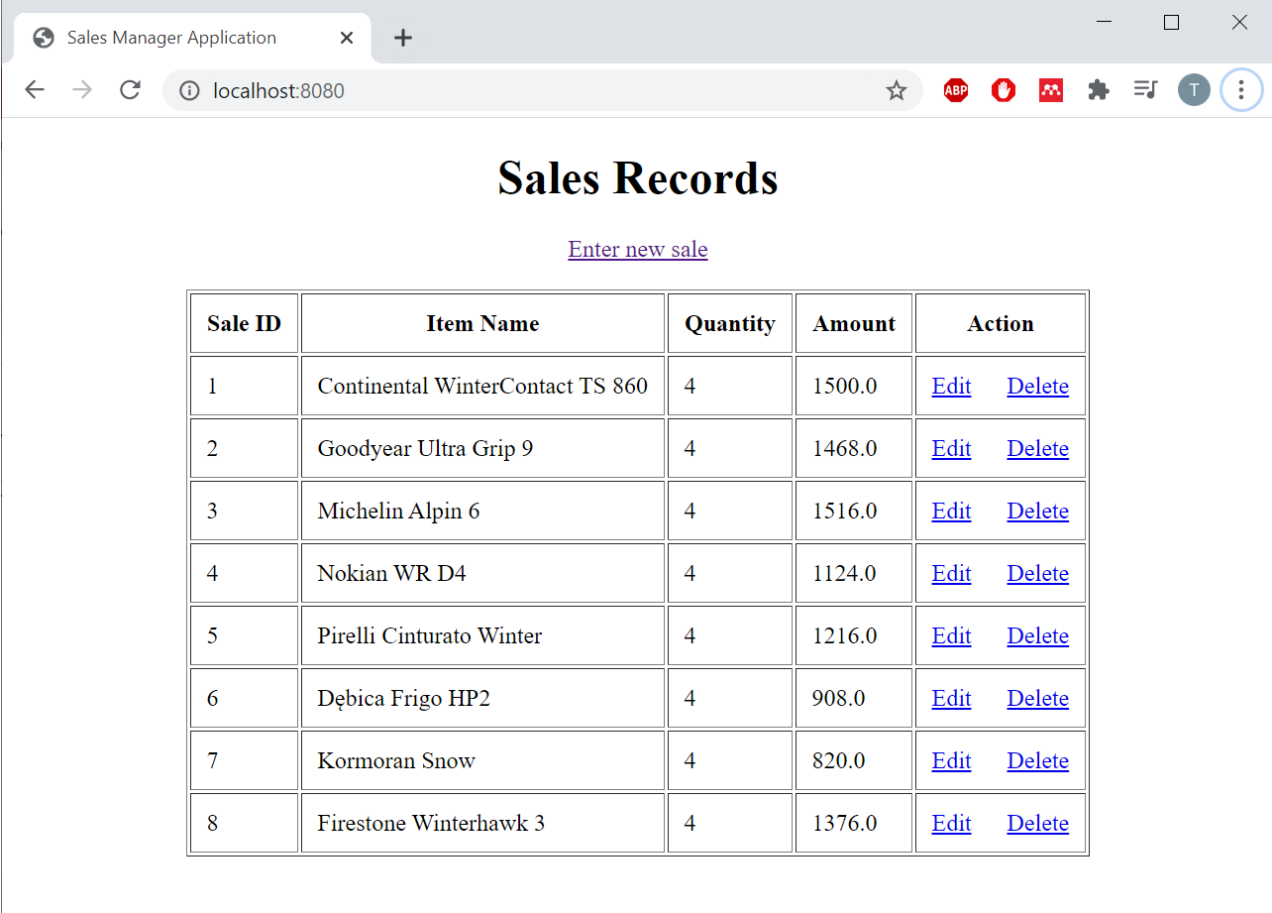
spring.datasource.password=tmrozek

logging.level.root=INFO

The screenshot shows the 'New / Select Database Connection' dialog box. On the left, a list of connection names includes 'JDBC' and 'xe', with 'xe' selected. The main configuration area on the right is divided into 'User Info' and 'Details' tabs. The 'User Info' tab is active, showing 'Name' as 'xe', 'Database Type' as 'Oracle', 'Authentication Type' as 'Default', 'Username' as 'system', 'Role' as 'default', and a masked 'Password' field. The 'Connection type' is set to 'Basic'. The 'Details' tab is also visible, showing 'Host name' as '192.168.0.14', 'Port' as '1521', and 'SID' as 'xe'. Red arrows from the text above point to these fields: one from '192.168.0.14' to the 'Host name' field, one from '1521' to the 'Port' field, and one from 'xe' to the 'SID' field. At the bottom, there are buttons for 'Pomoc', 'Save', 'Clear', 'Test', 'Connect', and 'Anuluj'.

WIZUALIZACJA TWORZONEJ PROSTEJ APLIKACJI WEBOWEJ

Wizualizacja tworzonej prostej aplikacji webowej



Sales Manager Application

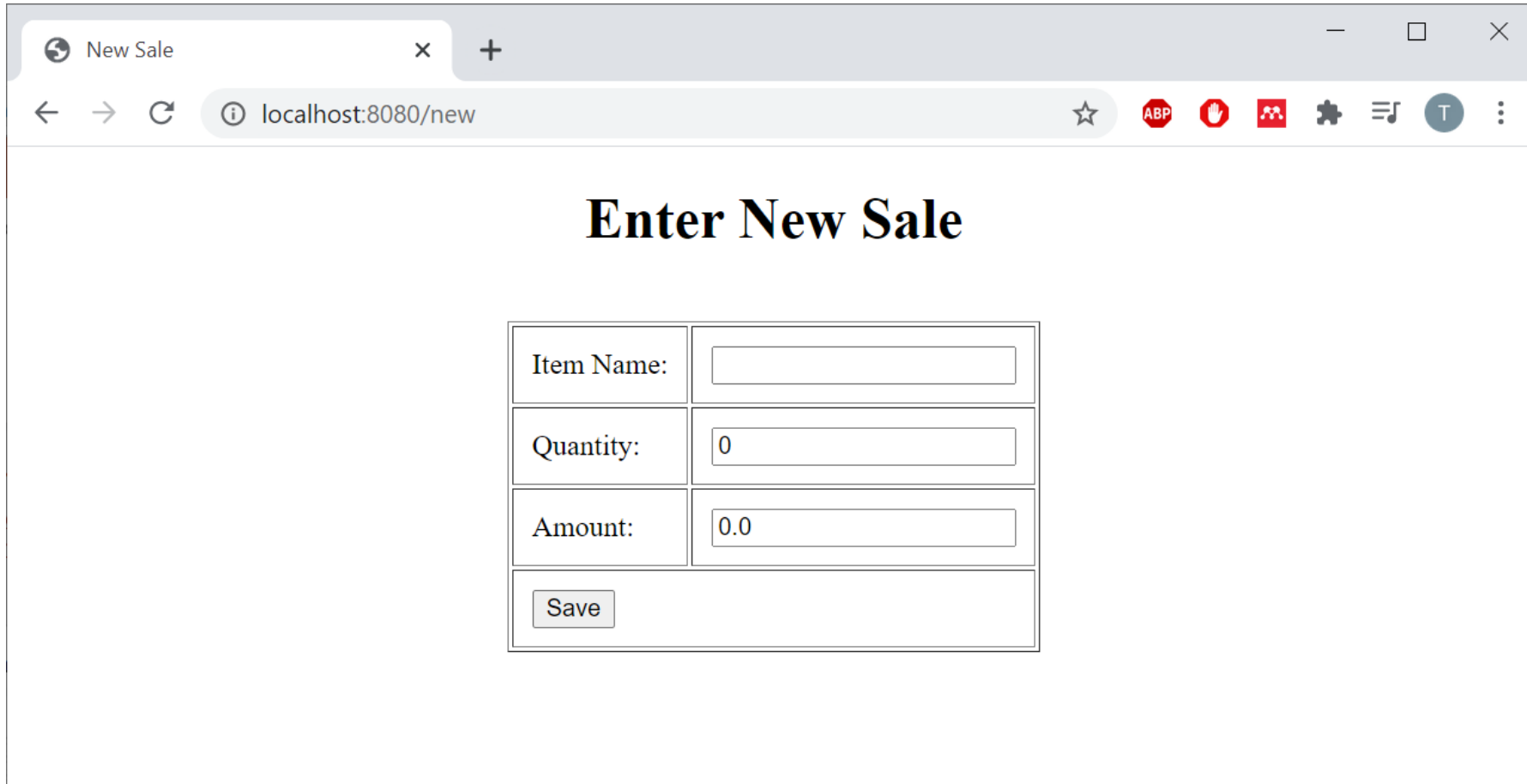
localhost:8080

Sales Records

[Enter new sale](#)

Sale ID	Item Name	Quantity	Amount	Action
1	Continental WinterContact TS 860	4	1500.0	Edit Delete
2	Goodyear Ultra Grip 9	4	1468.0	Edit Delete
3	Michelin Alpin 6	4	1516.0	Edit Delete
4	Nokian WR D4	4	1124.0	Edit Delete
5	Pirelli Cinturato Winter	4	1216.0	Edit Delete
6	Dębica Frigo HP2	4	908.0	Edit Delete
7	Kormoran Snow	4	820.0	Edit Delete
8	Firestone Winterhawk 3	4	1376.0	Edit Delete

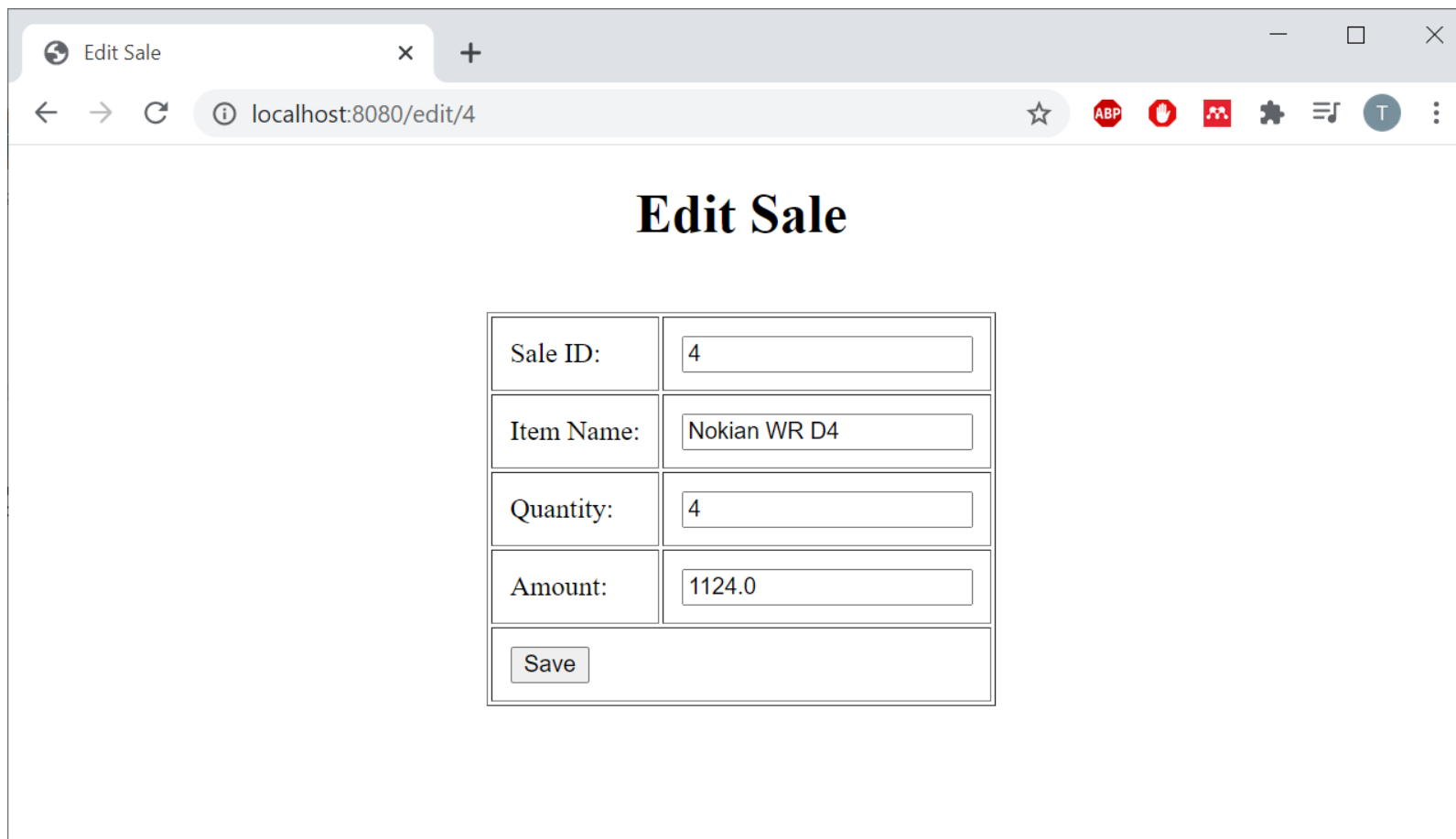
Wizualizacja tworzonej prostej aplikacji webowej



The screenshot shows a web browser window with a single tab titled 'New Sale'. The address bar displays 'localhost:8080/new'. The page content features a large heading 'Enter New Sale' in a bold, serif font. Below the heading is a form with three input fields: 'Item Name:', 'Quantity:', and 'Amount:'. The 'Quantity' field contains the value '0' and the 'Amount' field contains '0.0'. At the bottom of the form is a 'Save' button.

Item Name:	
Quantity:	0
Amount:	0.0
Save	

Wizualizacja tworzonej prostej aplikacji webowej



The screenshot shows a web browser window with a single tab titled 'Edit Sale'. The address bar displays 'localhost:8080/edit/4'. The page content features a large heading 'Edit Sale' in a bold, serif font. Below the heading is a form with four input fields, each with a label to its left: 'Sale ID:' with the value '4', 'Item Name:' with the value 'Nokian WR D4', 'Quantity:' with the value '4', and 'Amount:' with the value '1124.0'. At the bottom of the form is a 'Save' button. The browser's developer tools are visible in the bottom right corner.

Sale ID:	4
Item Name:	Nokian WR D4
Quantity:	4
Amount:	1124.0
Save	

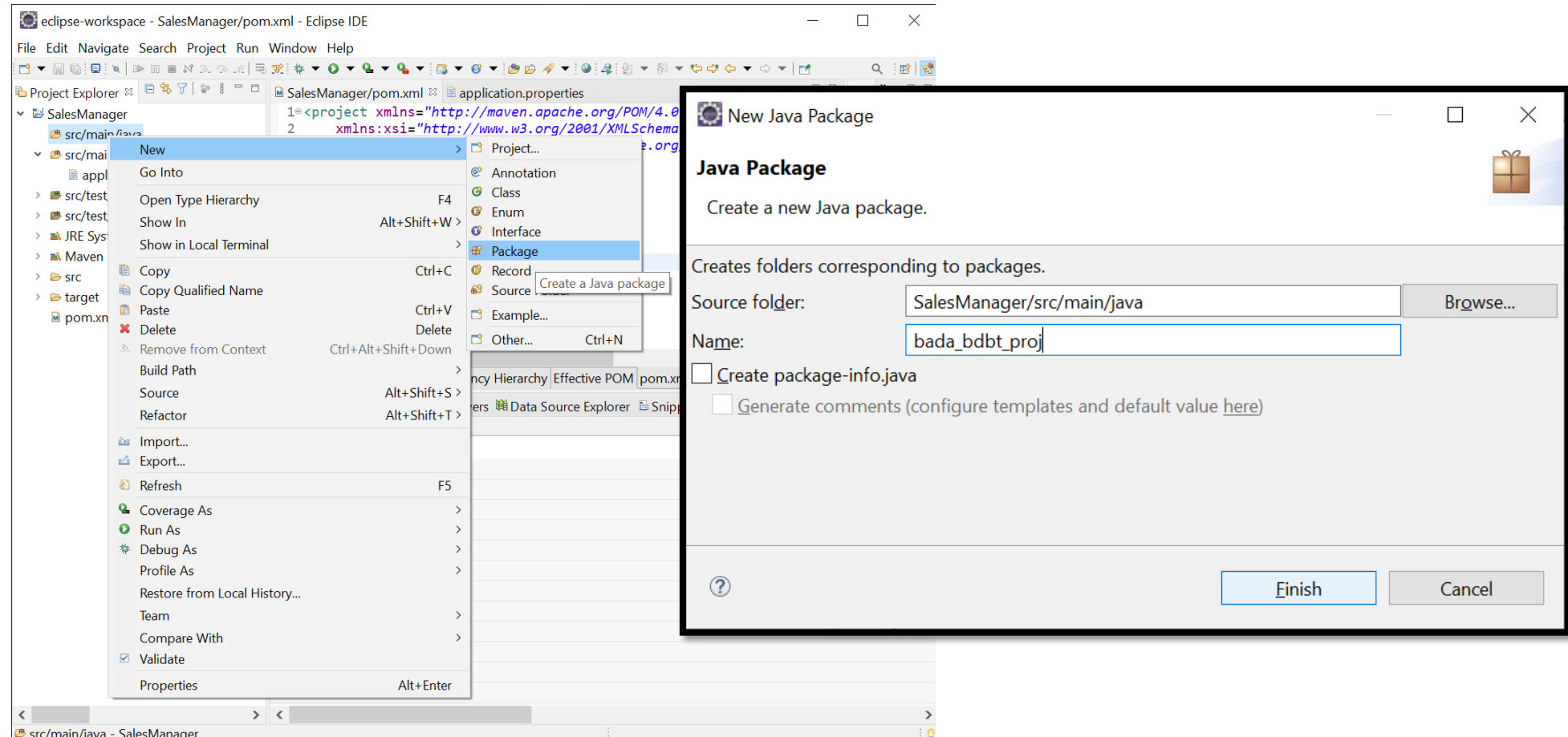
4

KLASA MAPUJĄCA KOLUMNY Z BAZY DANYCH

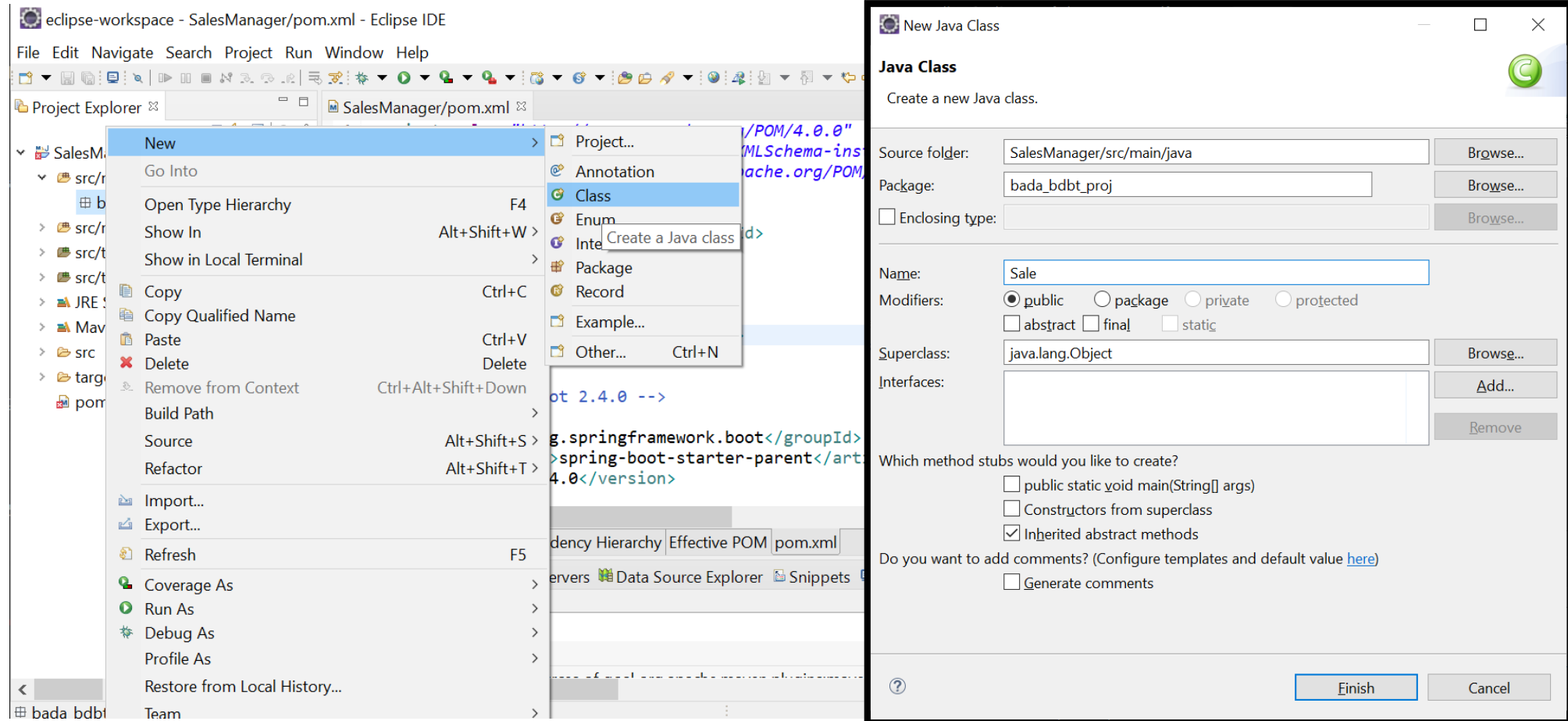
4. Klasa mapująca kolumny z bazy danych

Tworzenie klasy „Sale” z polami, które mapują kolumny tabeli „Sales” w bazie danych.

4. Klasa mapująca kolumny z bazy danych



4. Klasa mapująca kolumny z bazy danych



generate constructor using fields

4. Klasa mapująca kolumny z bazy

tworzenie pól odpowiadających kolumnom z bazy

tworzymy konstruktor przypisujący wartości swoich parametrów odpowiednim polom klasy.

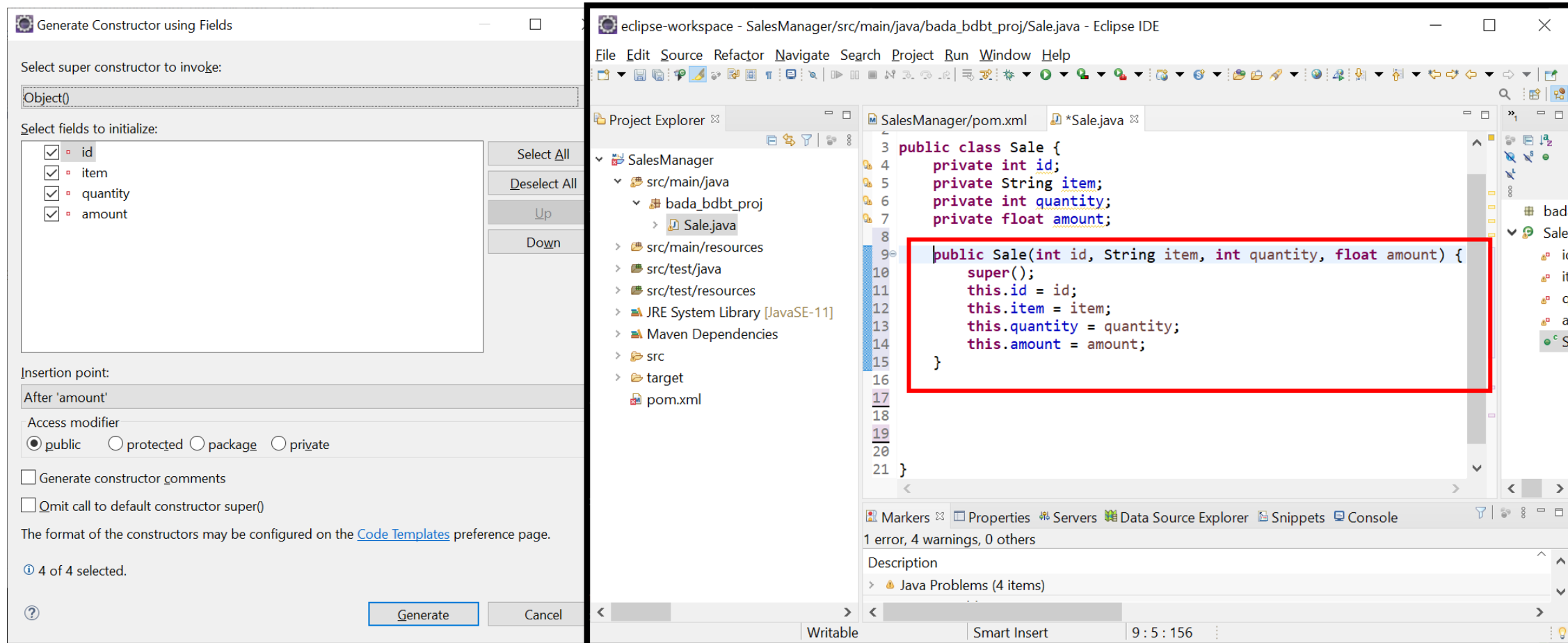
The screenshot displays the Eclipse IDE interface with three overlapping windows. The leftmost window shows the 'Project Explorer' for the 'SalesManager' project, with the 'bada_bdbt_proj' package selected. The middle window shows the 'SalesManager/pom.xml' file. The rightmost window shows the 'Sale.java' file, which contains the following code:

```
1 package bada_bdbt_proj;
2
3 public class Sale {
4     private int id;
5     private String item;
6     private int quantity;
7     private float amount;
8 }
9
10
11
12
13
14
```

A red rectangle highlights the class definition. A context menu is open over the code, showing various actions. The 'Generate Constructor Using Fields' option is highlighted in blue. The menu also includes options like 'Undo Typing', 'Save', 'Open Declaration', 'Quick Outline', 'Cut', 'Copy', 'Paste', 'Quick Fix', 'Source', 'Refactor', 'Local History', 'References', 'Declarations', 'Add to Snippets...', 'Coverage As', 'Run As', 'Debug As', 'Profile As', 'Team', 'Compare With', 'Replace With', 'Validate', 'Preferences...', 'Remove from Context', 'Toggle Comment', 'Remove Block Comment', 'Generate Element Comment', 'Correct Indentation', 'Format', 'Format Element', 'Add Import', 'Organize Imports', 'Sort Members...', 'Clean Up...', 'Override/Implement Methods...', 'Generate Getters and Setters...', 'Generate Delegate Methods...', 'Generate hashCode() and equals()...', 'Generate toString()...', and 'Generate Constructor using Fields...'.

4. Klasa mapująca kolumny z bazy

wybieramy pola, które chcemy zainicjować i otrzymujemy:



generate constructor from superclass

4. Klasa mapująca kolumny z bazy *tworzymy konstruktor superklasy z dziedziczeniem*

The screenshot shows the Eclipse IDE interface. The main editor displays the `Sale.java` file with the following code:

```
public class Sale {
    private int id;
    private String item;
    private int quantity;
    private float amount;

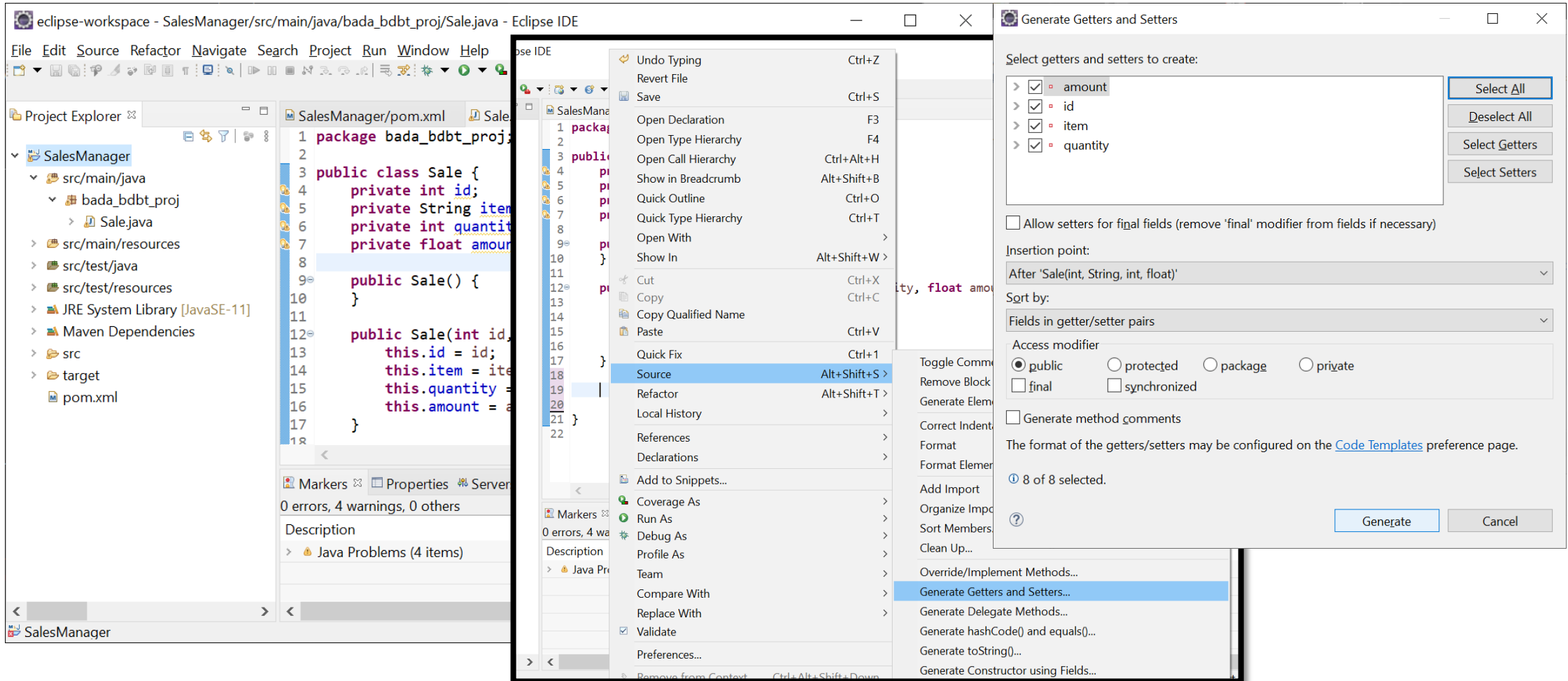
    public Sale(int id, String item, int quantity, float amount) {
        super();
        this.id = id;
        this.item = item;
        this.quantity = quantity;
        this.amount = amount;
    }
}
```

The `Generate Constructors from Superclass` dialog is open, showing the following options:

- Select constructors to implement: ☒ `Object()`
- Insertion point: After 'amount'
- Access modifier: ☒ public ☐ protected ☐ package ☐ private
- ☐ Generate constructor comments
- ☐ Omit call to default constructor super()
- The format of the constructors may be configured on the [Code Templates](#) preference page.
- 1 of 1 selected.
- Buttons: Generate, Cancel

The context menu is also visible, showing the `Generate Constructors from Superclass...` option.

4. Klasa mapująca kolumny z bazy

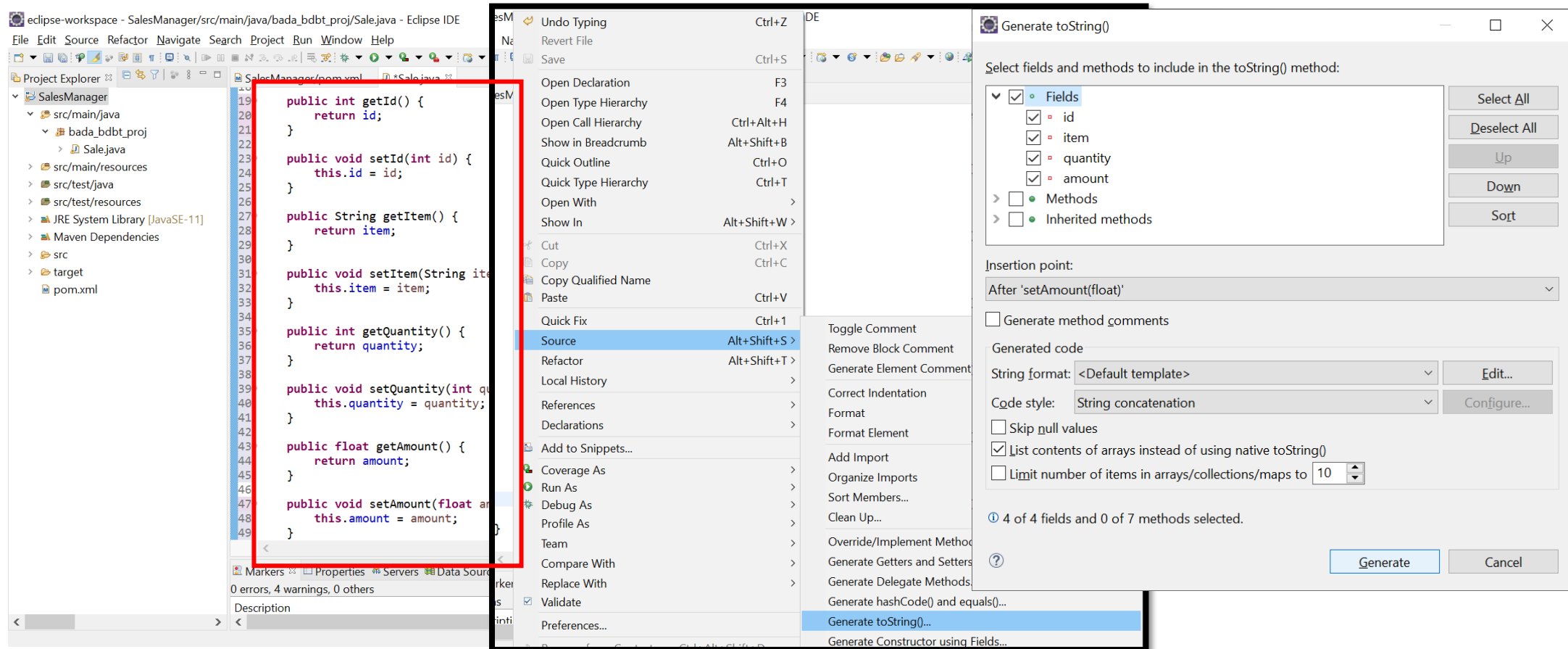


4. Metoda, Getters & Setters

Metod – podprogram składowy klasy, którego zadaniem jest działanie na rzecz określonych elementów danej klasy lub klas z nią spokrewnionych. Metody wiąże się z klasami po to, aby uniknąć użycia podprogramów globalnych, które i tak nie zostaną użyte w celu innym niż na rzecz konkretnej klasy.

Gettery i **settery** - metody, które pozwalają odpowiednio pobierać i zapisywać wartość pól klasy. Dobrą praktyką jest używanie tylko metod innych klas, a nie bezpośrednio ich pól. Tak zbudowane klasy to tzw. Beany — czyli klasy które swoje pola udostępniają tylko za pośrednictwem metod.

4. Klasa mapująca kolumny z bazy



4. Klasa mapująca kolumny z bazy danych



```
eclipse-workspace - SalesManager/src/main/java/bada_bdbt_proj/Sale.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help
SalesManager/pom.xml *Sale.java
Project Explorer
SalesManager
  src/main/java
    bada_bdbt_proj
      Sale.java
  src/main/resources
  src/test/java
  src/test/resources
  JRE System Library
  Maven Dependencies
  src
  target
  pom.xml
SalesManager/src/main/java/bada_bdbt_proj/Sale.java
37 }
38
39 public void setQuantity(int quantity) {
40     this.quantity = quantity;
41 }
42
43 public float getAmount() {
44     return amount;
45 }
46
47 public void setAmount(float amount) {
48     this.amount = amount;
49 }
50
51 @Override
52 public String toString() {
53     return "Sale [id=" + id + ", item=" + item + ", quantity=" + quantity + ", amount=" + amount + "];";
54 }
55
56
57 }
58
59
Markers Properties Servers Data Source Explorer Snippets Console
0 items
Description
bada_bdbt_proj.Sale.getAmount() : float - SalesManager/src/main/java
```

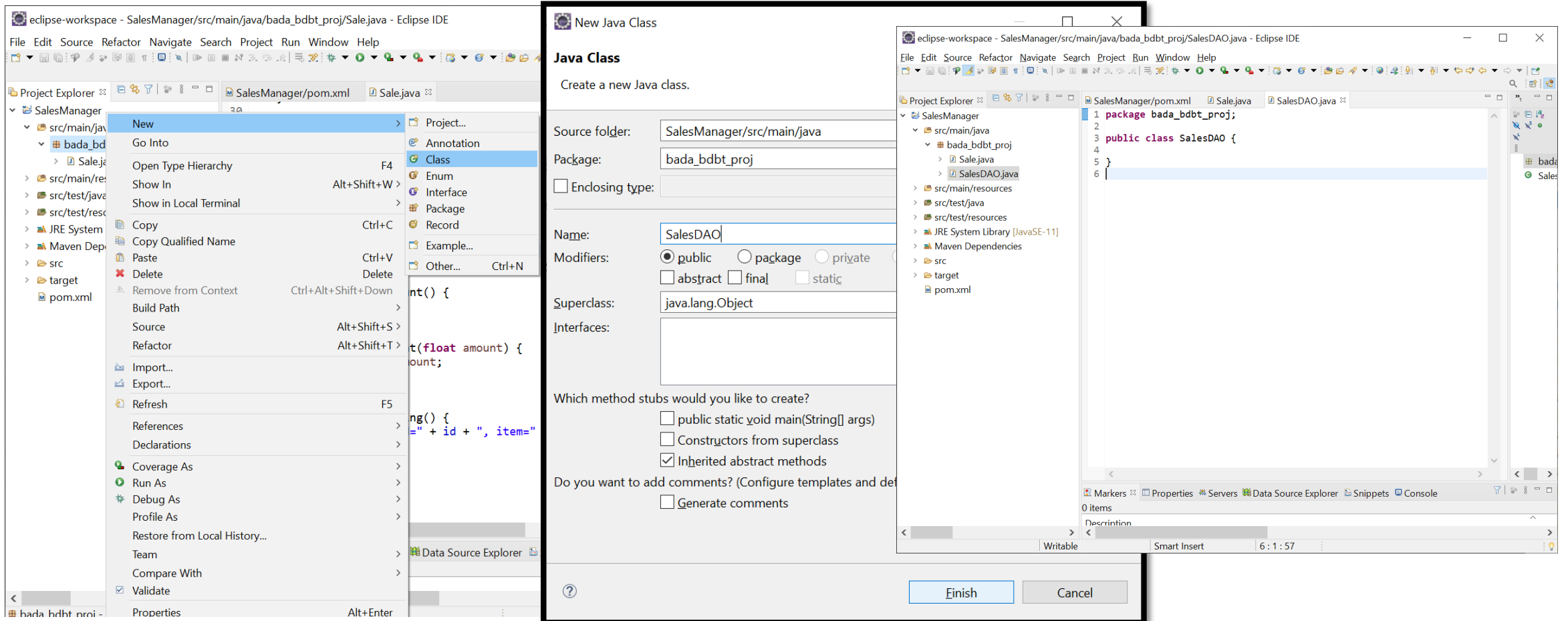
5

KLASA DO WYKONYWANIA OPERACJI TYPU CRUD

5. Code Data Access Class (DAO)

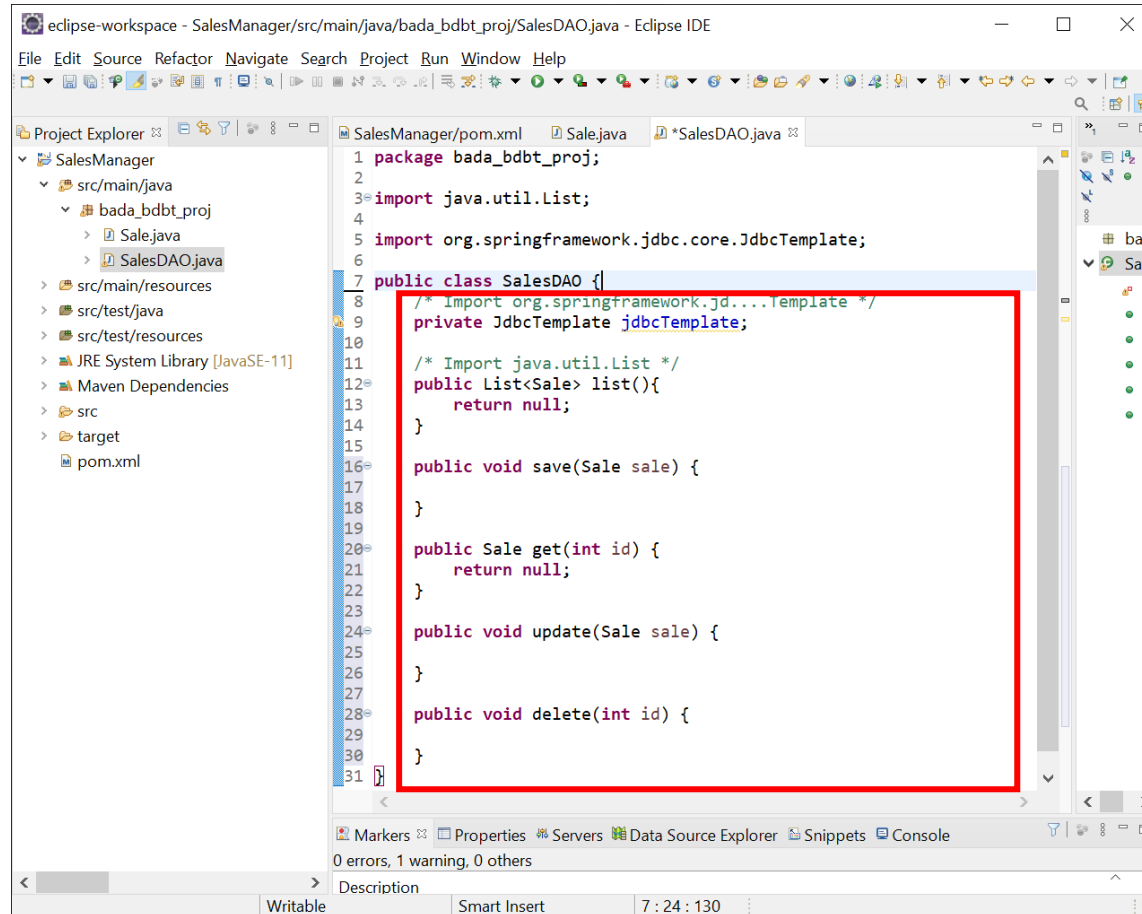
- Stworzenie klasy, która używa Spring JDBC do wykonywania operacji typu CRUD (C-create, R-read, U-update, D-delete) w tabeli Sales.

5. Code Data Access Class (DAO)



5. Code Data Access Class

Kluczową kwestią jest to, że określamy, że instancja klasy JdbcTemplate zostanie automatycznie utworzona i wstrzyknięta przez Spring. Następnie zostanie użyta w metodach klasy do wykonywania operacji CRUD



```
/* Import org.springframework.jdbc....Template */
private JdbcTemplate jdbcTemplate;

/* Import java.util.List */    (zawiera info z bazy danych)
public List<Sale> list(){
    return null;
}

/* Insert - wstawianie nowego wiersza do bazy */
public void save(Sale sale) {

}

/* Read - odczytywanie danych z bazy */
public Sale get(int id) {
    return null;
}

/* Update - aktualizacja danych */
public void update(Sale sale) {

}

/* Delete - wybrany rekord z danym id */
public void delete(int id) {

}
}
```

5. Code Data Access Class (DAO)

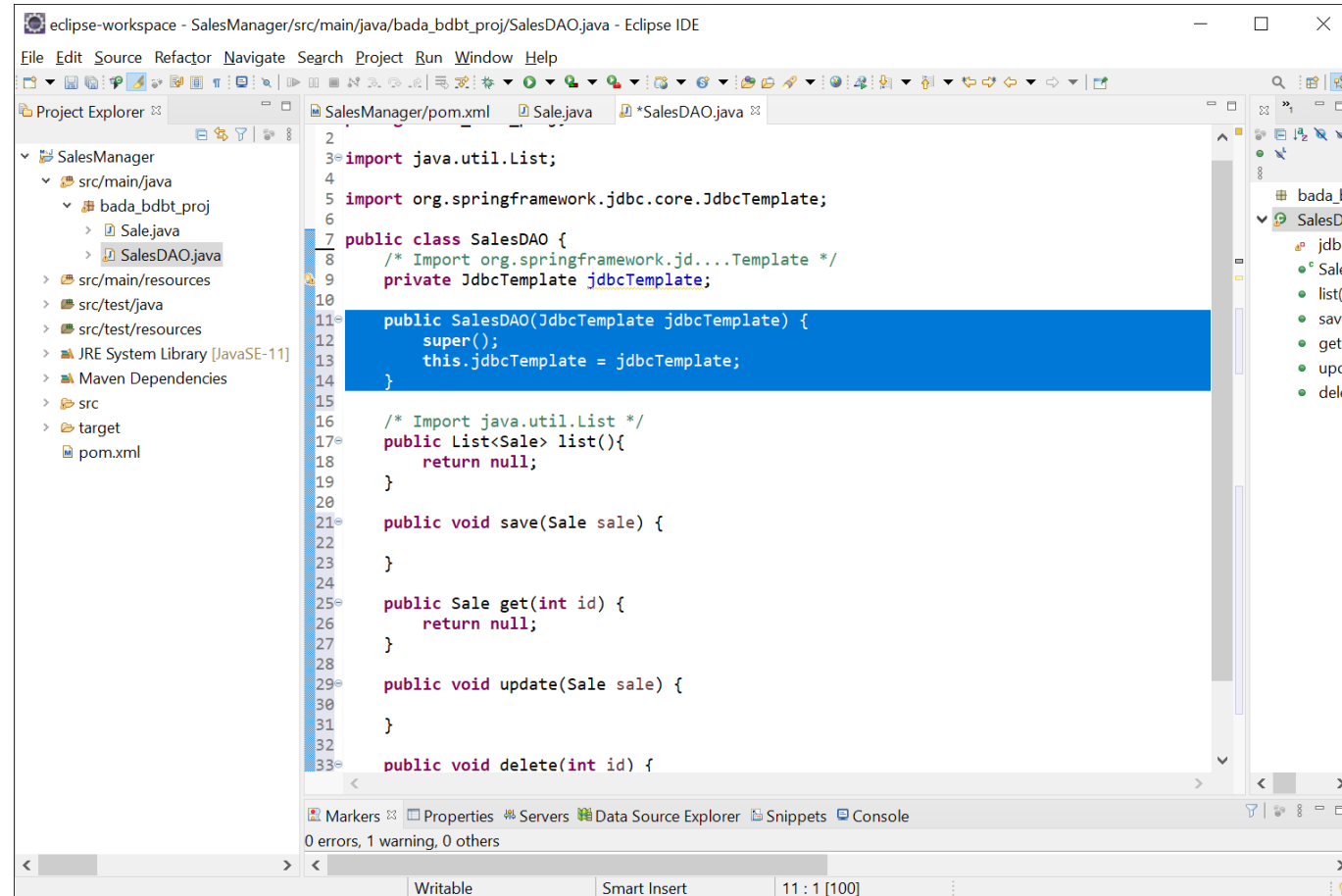
Tworzenie konstruktora dla jdbcTemplate

The screenshot shows the Eclipse IDE with the 'SalesManager' project open. The 'SalesDAO.java' file is selected in the Project Explorer. The 'Generate Constructor using Fields' dialog box is open, showing the following configuration:

- Select super constructor to invoke:** Object()
- Select fields to initialize:** jdbcTemplate (checked)
- Insertion point:** After 'jdbcTemplate'
- Access modifier:** public (selected), protected, package, private
- Generate constructor comments:** unchecked
- Omit call to default constructor super():** unchecked

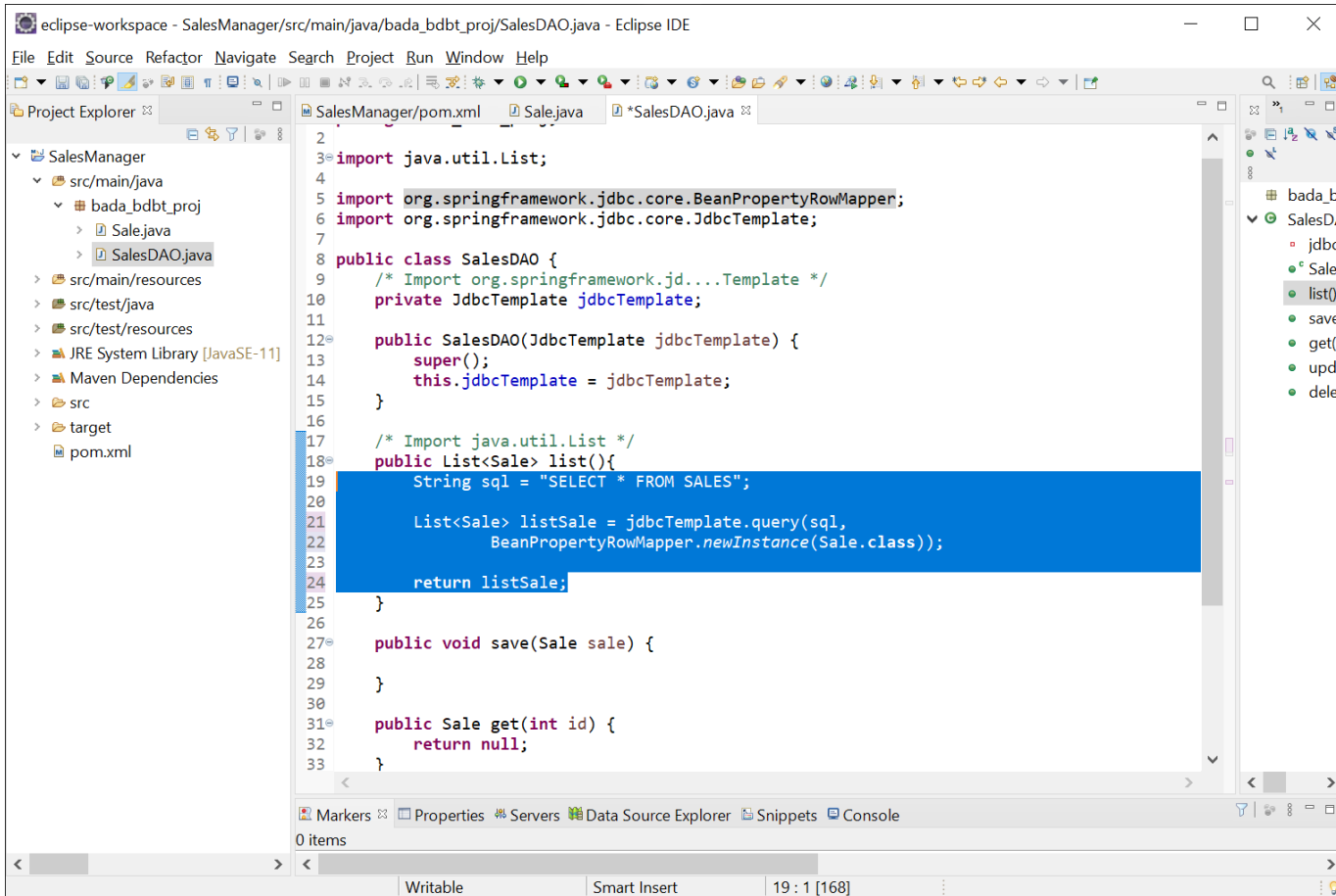
The dialog also includes a note: "The format of the constructors may be configured on the [Code Templates](#) preference page." and a status bar at the bottom indicating "1 of 1 selected." The 'Generate' button is highlighted.

5. Code Data Access Class (DAO)



```
1
2
3 import java.util.List;
4
5 import org.springframework.jdbc.core.JdbcTemplate;
6
7 public class SalesDAO {
8     /* Import org.springframework.jd...Template */
9     private JdbcTemplate jdbcTemplate;
10
11     public SalesDAO(JdbcTemplate jdbcTemplate) {
12         super();
13         this.jdbcTemplate = jdbcTemplate;
14     }
15
16     /* Import java.util.List */
17     public List<Sale> list(){
18         return null;
19     }
20
21     public void save(Sale sale) {
22     }
23
24     public Sale get(int id) {
25         return null;
26     }
27
28     public void update(Sale sale) {
29     }
30
31     public void delete(int id) {
32     }
33 }
```

5. Code Data Access Class (DAO)



```
1 2
3  import java.util.List;
4
5  import org.springframework.jdbc.core.BeanPropertyRowMapper;
6  import org.springframework.jdbc.core.JdbcTemplate;
7
8  public class SalesDAO {
9      /* Import org.springframework.jd...Template */
10     private JdbcTemplate jdbcTemplate;
11
12     public SalesDAO(JdbcTemplate jdbcTemplate) {
13         super();
14         this.jdbcTemplate = jdbcTemplate;
15     }
16
17     /* Import java.util.List */
18     public List<Sale> list(){
19         String sql = "SELECT * FROM SALES";
20
21         List<Sale> listSale = jdbcTemplate.query(sql,
22             BeanPropertyRowMapper.newInstance(Sale.class));
23
24         return listSale;
25     }
26
27     public void save(Sale sale) {
28
29     }
30
31     public Sale get(int id) {
32         return null;
33     }
34 }
```

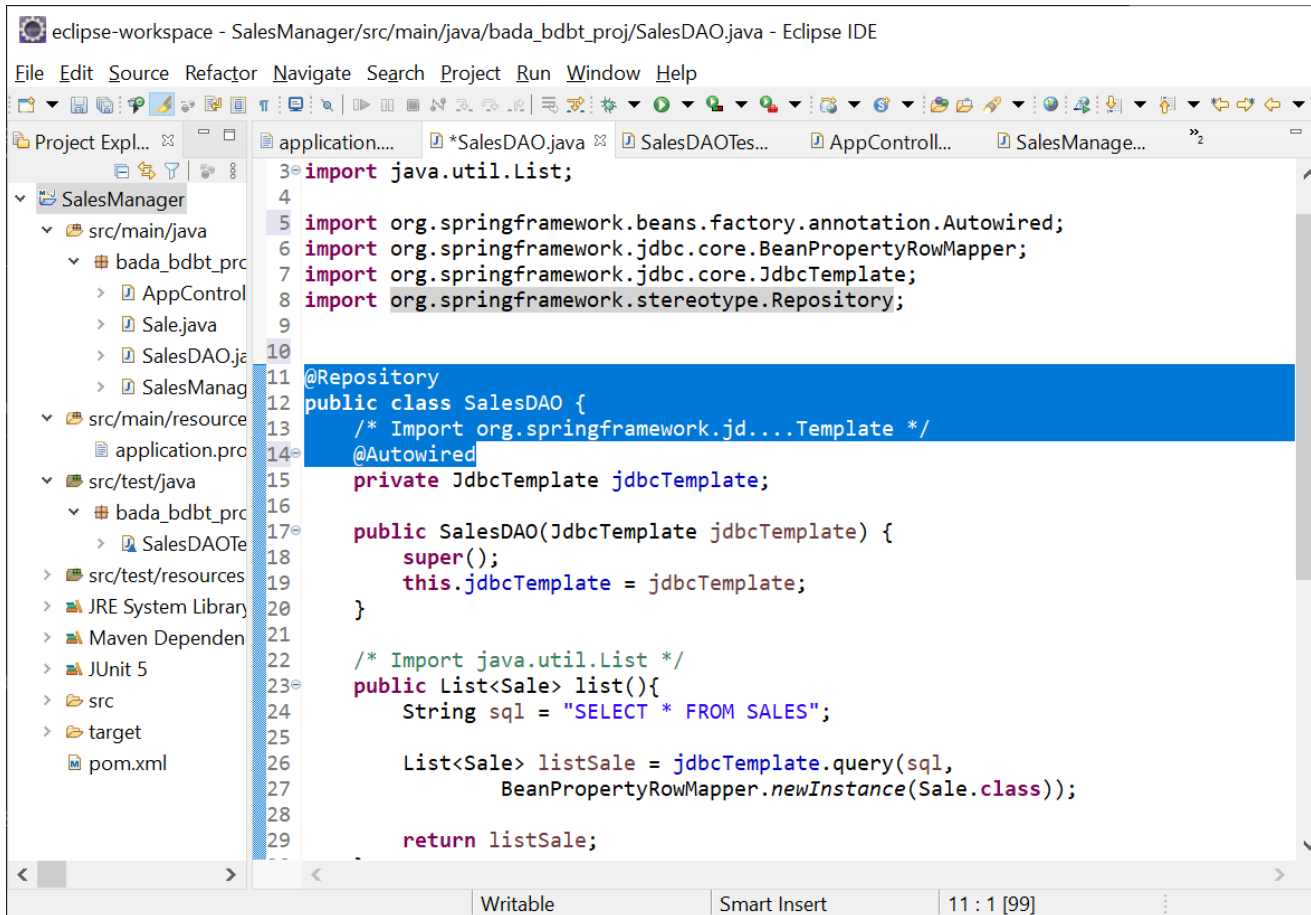
Tworzenie metody dla Listy, dla pobierania wszystkich danych z tabeli Sales i wstawiania ich do listy.

Metoda query – umożliwia wysyłanie wszystkich typów zapytań SQL do bazy danych, możliwe są zapytania które:

- modyfikują,
- wstawiają,
- zmieniają ustawienia połączenia z bazą danych.

tutaj zapytanie typu SELECT

5. Code Data Access Class (DAO)



```
eclipse-workspace - SalesManager/src/main/java/bada_bdbt_proj/SalesDAO.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help
Project Expl... application... *SalesDAO.java SalesDAOTes... AppControll... SalesManage...
SalesManager
  src/main/java
    bada_bdbt_prc
      AppControl
      Sale.java
      SalesDAO.java
      SalesManag
  src/main/resource
    application.pro
  src/test/java
    bada_bdbt_prc
      SalesDAOTe
  src/test/resources
  JRE System Library
  Maven Depend
  JUnit 5
  src
  target
  pom.xml

3 import java.util.List;
4
5 import org.springframework.beans.factory.annotation.Autowired;
6 import org.springframework.jdbc.core.BeanPropertyRowMapper;
7 import org.springframework.jdbc.core.JdbcTemplate;
8 import org.springframework.stereotype.Repository;
9
10
11 @Repository
12 public class SalesDAO {
13     /* Import org.springframework.jd...Template */
14     @Autowired
15     private JdbcTemplate jdbcTemplate;
16
17     public SalesDAO(JdbcTemplate jdbcTemplate) {
18         super();
19         this.jdbcTemplate = jdbcTemplate;
20     }
21
22     /* Import java.util.List */
23     public List<Sale> list(){
24         String sql = "SELECT * FROM SALES";
25
26         List<Sale> listSale = jdbcTemplate.query(sql,
27             BeanPropertyRowMapper.newInstance(Sale.class));
28
29         return listSale;
30     }
31 }
```

Adnotacje są konstrukcją, która pozwala na przekazywanie dodatkowych informacji na temat kodu. Informacje te mogą być wykorzystane później w kilku miejscach. Adnotacje służą do przekazywania metadanych. Innymi słowy przekazują one dane o danych – dane o kodzie źródłowym

Zastosowanie:

- dodatkowe informacje dla kompilatora,
- adnotacje przetwarzane w trakcie kompilacji,
- adnotacje przetwarzane w trakcie uruchomienia programu.

5. Pierwszy test jednostkowy - JUnit

The screenshot shows the Eclipse IDE interface. The 'New' menu is open, and the 'JUnit Test Case' wizard is displayed. The wizard is configured with the source folder 'SalesManager/src/test/java', package 'bada_bdbt_proj', name 'SalesDAOTest', superclass 'java.lang.Object', and method stubs 'setUp()' and 'tearDown()'. The 'Finish' button is highlighted.

New JUnit Test Case

Select the name of the new JUnit test case. You have the options to specify the class under test and on the next page, to select methods to be tested.

☐ New JUnit 3 test ☐ New JUnit 4 test ☒ New JUnit Jupiter test

Source folder: SalesManager/src/test/java

Package: bada_bdbt_proj

Name: SalesDAOTest

Superclass: java.lang.Object

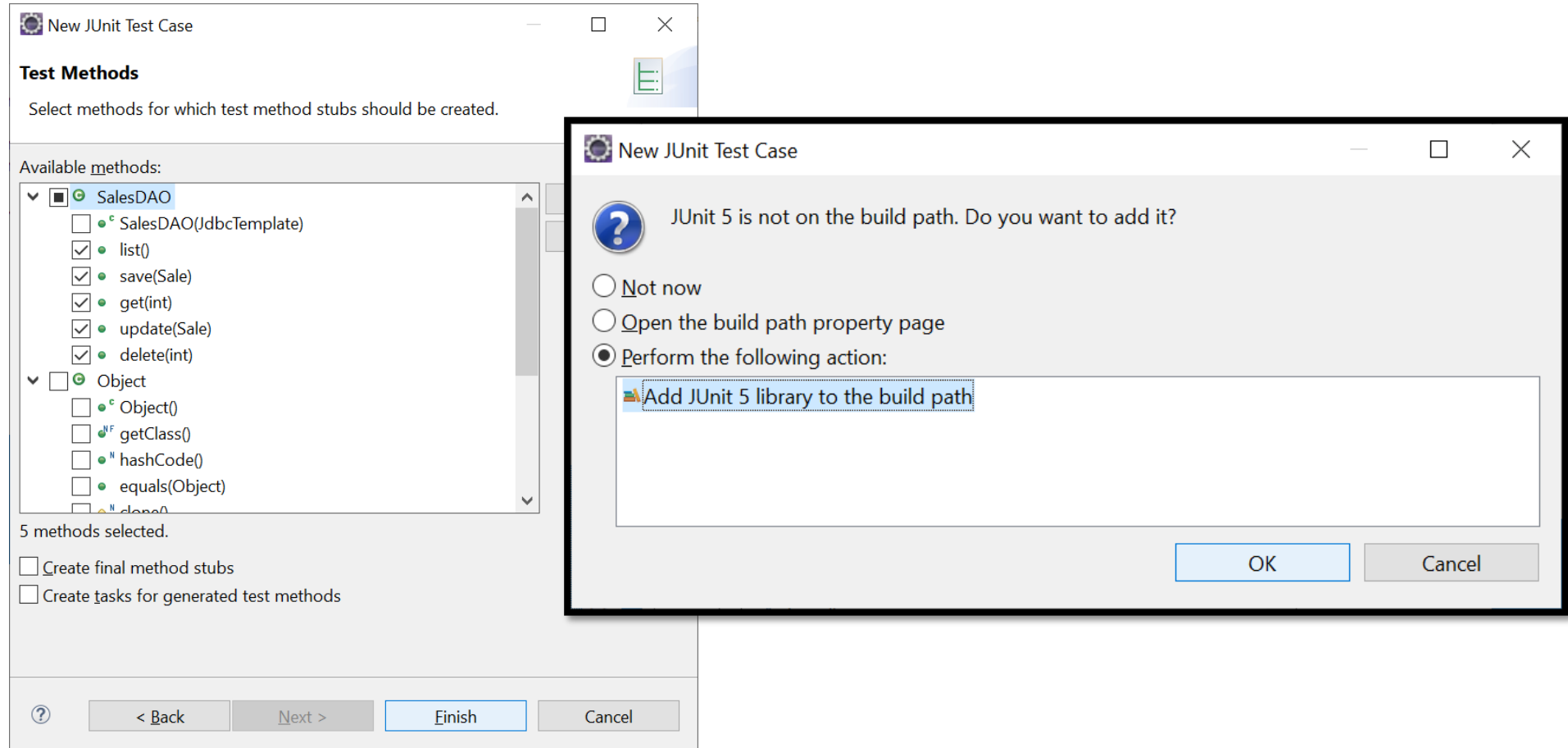
Which method stubs would you like to create?

☐ setUpBeforeClass() ☐ tearDownAfterClass()
☒ setUp() ☐ tearDown()
☐ constructor

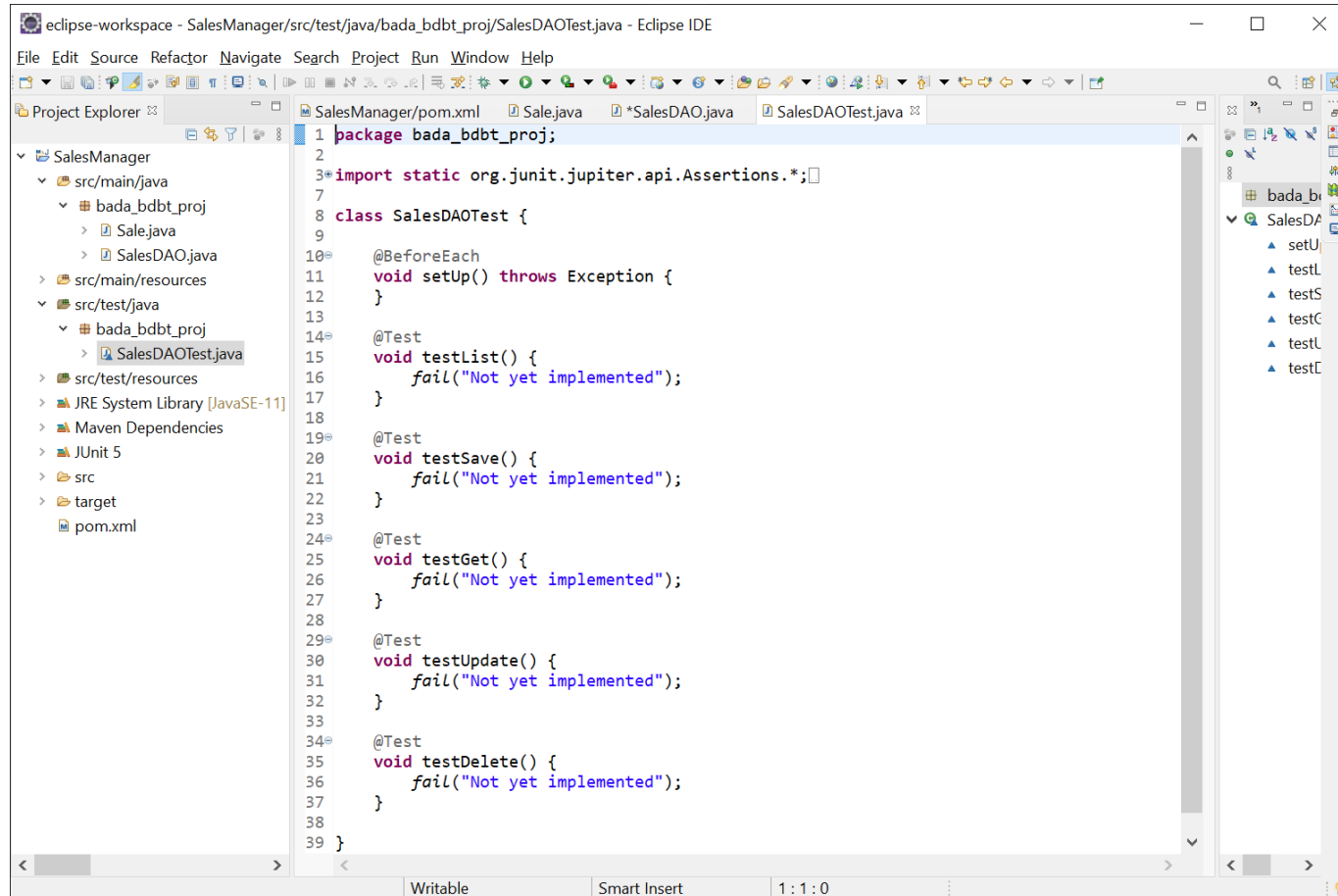
Do you want to add comments? (Configure templates and default value [here](#))
☐ Generate comments

Class under test: bada_bdbt_proj.SalesDAO

5. Pierwszy test jednostkowy - JUnit



5. Pierwszy test jednostkowy - JUnit



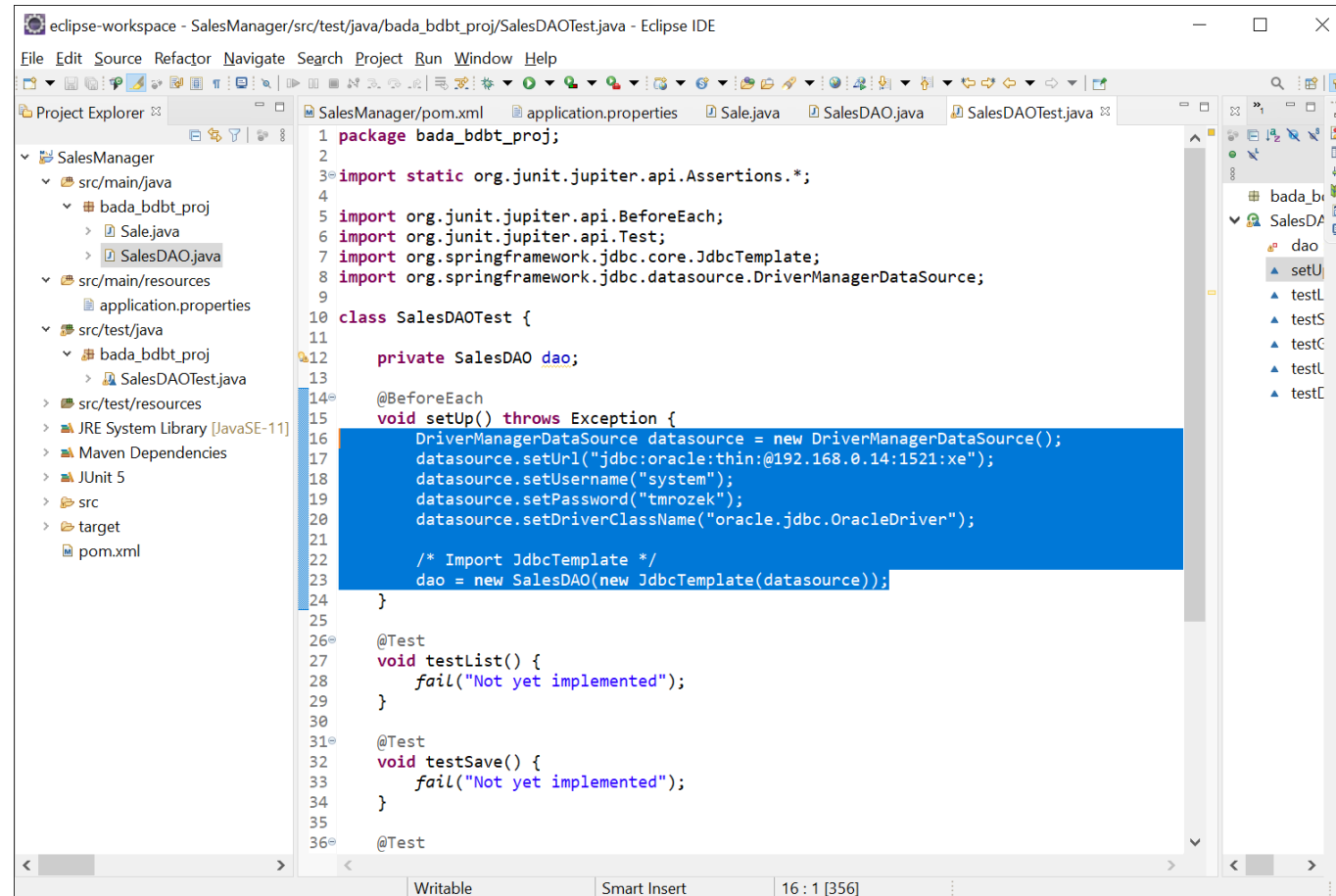
```
1 package bada_bdbt_proj;
2
3 import static org.junit.jupiter.api.Assertions.*;
4
5 class SalesDAOTest {
6
7     @BeforeEach
8     void setUp() throws Exception {
9     }
10
11     @Test
12     void testList() {
13         fail("Not yet implemented");
14     }
15
16     @Test
17     void testSave() {
18         fail("Not yet implemented");
19     }
20
21     @Test
22     void testGet() {
23         fail("Not yet implemented");
24     }
25
26     @Test
27     void testUpdate() {
28         fail("Not yet implemented");
29     }
30
31     @Test
32     void testDelete() {
33         fail("Not yet implemented");
34     }
35 }
```

Czasami zdarza się, że kilka testów jednostkowych wymaga pewnego „przygotowania”. Na przykład trzeba utworzyć instancję, którą będziemy później testowali. Twórcy biblioteki JUnit przyszli nam z pomocą. Istnieje adnotacja `@Before`, którą możemy dodać do metody w klasie z testami. Metoda ta zostanie uruchomiona przed każdym testem jednostkowym.

Metoda `setUp` zostanie wywołana przed uruchomieniem każdego z testów. Dzięki temu nie musimy tworzyć instancji wewnątrz testu. Odpowiednie użycie tej adnotacji pomaga pisać krótsze testy jednostkowe.

`SetUp` powoduje uruchamianie się metody, przed każdym uruchomieniem naszego testu, np. aby zainicjalizować dane.

5. Pierwszy test jednostkowy - JUnit – setUp()



The screenshot shows the Eclipse IDE interface. The Project Explorer on the left displays the project structure: SalesManager > src/main/java > bada_bdbt_proj > SalesDAO.java. The main editor shows the SalesDAOTest.java file with the following code:

```
1 package bada_bdbt_proj;
2
3 import static org.junit.jupiter.api.Assertions.*;
4
5 import org.junit.jupiter.api.BeforeEach;
6 import org.junit.jupiter.api.Test;
7 import org.springframework.jdbc.core.JdbcTemplate;
8 import org.springframework.jdbc.datasource.DriverManagerDataSource;
9
10 class SalesDAOTest {
11
12     private SalesDAO dao;
13
14     @BeforeEach
15     void setUp() throws Exception {
16         DriverManagerDataSource datasource = new DriverManagerDataSource();
17         datasource.setUrl("jdbc:oracle:thin:@192.168.0.14:1521:xe");
18         datasource.setUsername("system");
19         datasource.setPassword("tmrozek");
20         datasource.setDriverClassName("oracle.jdbc.OracleDriver");
21
22         /* Import JdbcTemplate */
23         dao = new SalesDAO(new JdbcTemplate(datasource));
24     }
25
26     @Test
27     void testList() {
28         fail("Not yet implemented");
29     }
30
31     @Test
32     void testSave() {
33         fail("Not yet implemented");
34     }
35
36     @Test
```

The right-hand side of the IDE shows the Package Explorer with the following structure: bada_bdbt_proj > SalesDAO > dao > setUp, testList, testSave, testCreate, testUpdate, testDelete.

5. Pierwszy test jednostkowy - JUnit – setUp()

The screenshot displays the Eclipse IDE interface. The left pane shows the Project Explorer with the project structure: SalesManager, src/main/java, bada_bdbt_proj, Sale.java, SalesDAO.java, and SalesDAOTest.java. The main editor shows the code for SalesDAOTest.java, which includes a setUp() method and a testList() method. The testList() method is highlighted in blue. The right pane shows the Outline view with the SalesDAOTest class and its methods. A context menu is open over the testList() method, showing various actions. The 'Run As' option is selected, and a submenu is visible with 'JUnit Test' and 'Run Configurations...' options.

```
10 import org.springframework.jdbc.datasource.DriverManagerDataSource;
11
12 class SalesDAOTest {
13
14     private SalesDAO dao;
15
16     @BeforeEach
17     void setUp() throws Exception {
18         DriverManagerDataSource datasource = new DriverManagerDataSource();
19         datasource.setUrl("jdbc:oracle:thin:@192.168.0.14:1521:xe");
20         datasource.setUsername("system");
21         datasource.setPassword("tmrozek");
22         datasource.setDriverClassName("oracle.jdbc.OracleDriver");
23
24         /* Import JdbcTemplate */
25         dao = new SalesDAO(new JdbcTemplate(datasource));
26     }
27
28     @Test
29     void testList() {
30         /* Import java.util */
31         List<Sale> listSale = dao.list();
32
33         assertTrue(listSale.isEmpty());
34     }
35
36     @Test
37     void testSave() {
38         fail("Not yet implemented");
39     }
40
41     @Test
42     void testGet() {
43         fail("Not yet implemented");
44     }
45 }
```

Context Menu Options:

- Open Type Hierarchy (F4)
- Open Call Hierarchy (Ctrl+Alt+H)
- Show In (Alt+Shift+W >)
- Cut (Ctrl+X)
- Copy (Ctrl+C)
- Copy Qualified Name
- Paste (Ctrl+V)
- Delete (Delete)
- Source (Alt+Shift+S >)
- Refactor (Alt+Shift+T >)
- References (>)
- Declarations (>)
- Toggle Method Breakpoint (>)
- Coverage As (>)
- Run As (>)
 - JUnit Test (Alt+Shift+X, T)
 - Run Configurations...
- Profile As (>)
- Restore from Local History... (>)
- Compare With (>)
- Replace With (>)
- Properties (Alt+Enter)

5. Pierwszy test jednostkowy - JUnit – setUp()

The screenshot shows an IDE window with the following components:

- Console:** Displays debug logs from Spring Framework. The logs indicate that a JDBC driver (oracle.jdbc.OracleDriver) was loaded and a connection was established to a database at 192.168.0.14:1521:xe.
- Project Explorer:** Shows the project structure, including the `SalesDAOTest.java` file.
- Editor:** Displays the code for `SalesDAOTest.java`. The `testList()` method is highlighted, showing a call to `dao.list()` and an assertion that the list is empty.
- Test Explorer:** Lists the test methods: `setUp() : void`, `testList() : void`, `testSave() : void`, `testGet() : void`, `testUpdate() : void`, and `testDelete() : void`.
- Status Bar:** Shows the test results: "Finished after 1,24 seconds", "Runs: 1/1", "Errors: 0", and "Failures: 0".

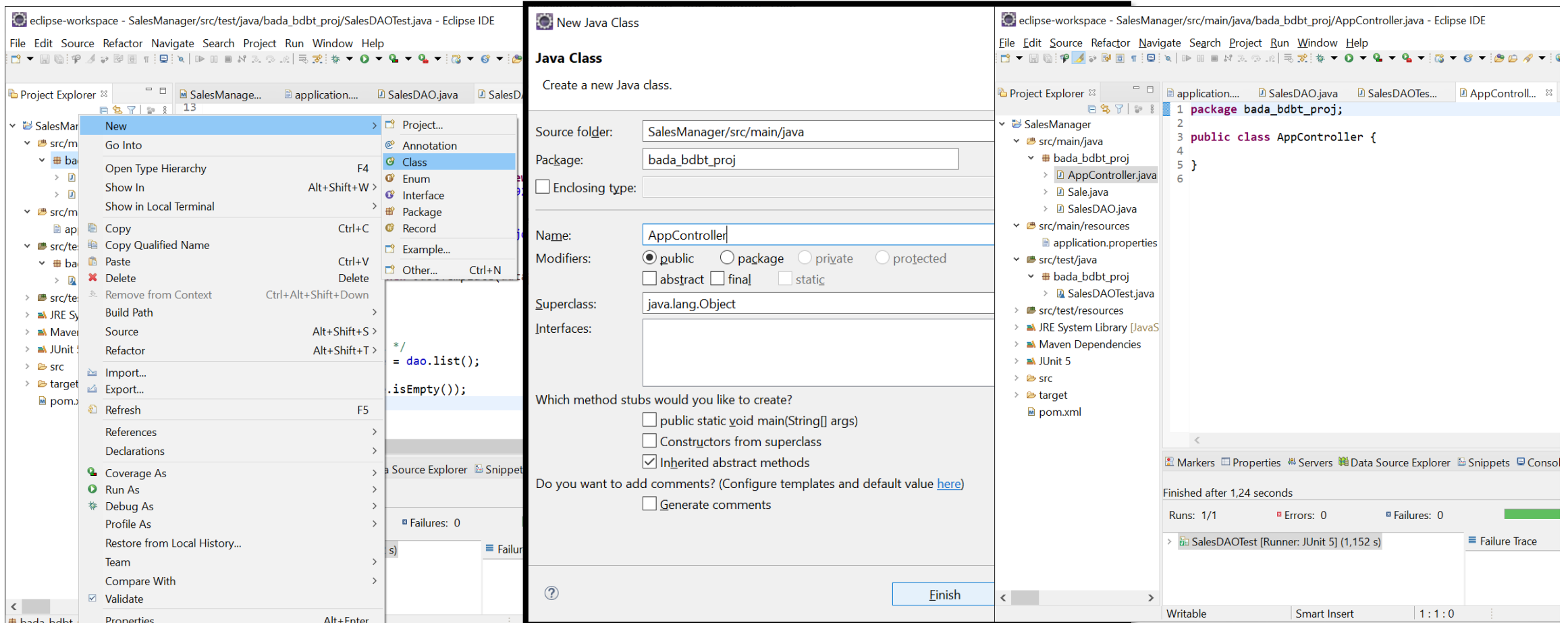
```
24      /* Import JdbcTemplate */
25      dao = new SalesDAO(new JdbcTemplate(datasource));
26  }
27
28  @Test
29  void testList() {
30      /* Import java.util */
31      List<Sale> listSale = dao.list();
32
33      assertTrue(listSale.isEmpty());
34  }
35
36  @Test
```

6

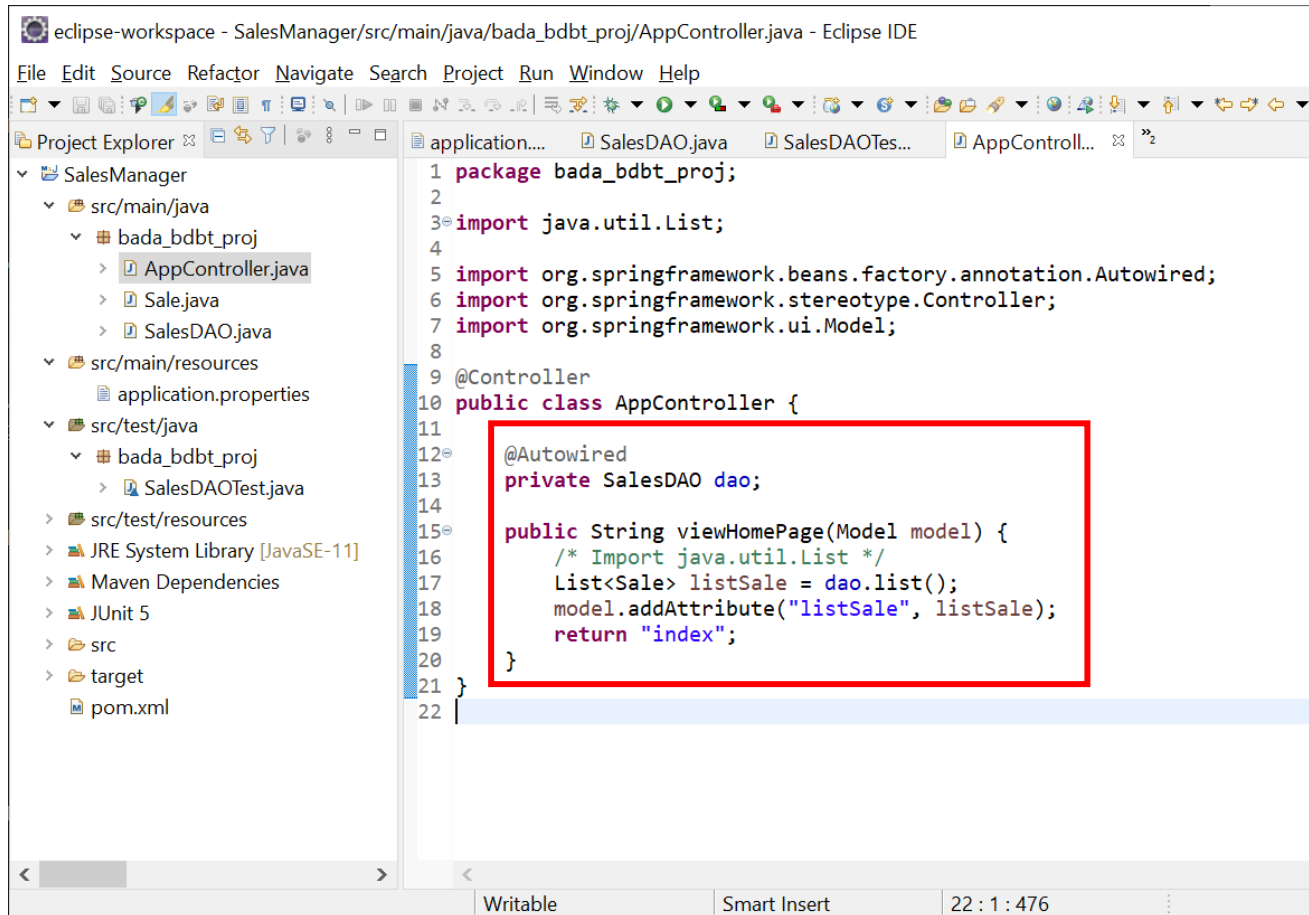
KONTROLER APLIKACJI MVC

6. Kontroler aplikacji MVC

Tworzymy aplikację internetową Spring Boot, dlatego potrzebujemy klasę kontrolera do obsługi żądań klientów.



6. Kontroler aplikacji MVC



```
1 package bada_bdbt_proj;
2
3 import java.util.List;
4
5 import org.springframework.beans.factory.annotation.Autowired;
6 import org.springframework.stereotype.Controller;
7 import org.springframework.ui.Model;
8
9 @Controller
10 public class AppController {
11
12     @Autowired
13     private SalesDAO dao;
14
15     public String viewHomePage(Model model) {
16         /* Import java.util.List */
17         List<Sale> listSale = dao.list();
18         model.addAttribute("listSale", listSale);
19         return "index";
20     }
21 }
22
```

Kluczową kwestią jest tutaj to, że instancja klasy SalesDAO zostanie utworzona i wstrzyknięta automatycznie przez Spring.

Kontroler wyświetli również wszystkie rekordy ze strony głównej. Zwracany zostanie indeks nazwy widoku tzn. index.html. Index to nazwa template'u.

Następnie w metodach obsługi wywoływane będą jej metody CRUD.

6. Kontroler aplikacji MVC

The image displays two side-by-side screenshots of the Eclipse IDE, showing the development of an MVC application. Both screenshots show the same project structure in the Project Explorer: SalesManager, src/main/java (bada_bdbt_proj), src/main/resources (application.properties), and src/test/java (bada_bdbt_proj, SalesDAOTest.java). The main editor shows the AppController.java file.

Left Screenshot: The code in AppController.java is as follows:

```
1 package bada_bdbt_proj;
2
3 import java.util.List;
4
5 import org.springframework.beans.factory.annotation.Autowired;
6 import org.springframework.stereotype.Controller;
7 import org.springframework.ui.Model;
8
9 @Controller
10 public class AppController {
11
12     @Autowired
13     private SalesDAO dao;
14
15     public String viewHomePage(Model model) {
16         /* Import java.util.List */
17         List<Sale> listSale = dao.list();
18         model.addAttribute("listSale", listSale);
19         return "index";
20     }
21 }
```

Right Screenshot: The code in AppController.java is updated with an additional import for RequestMapping, which is highlighted with a red box:

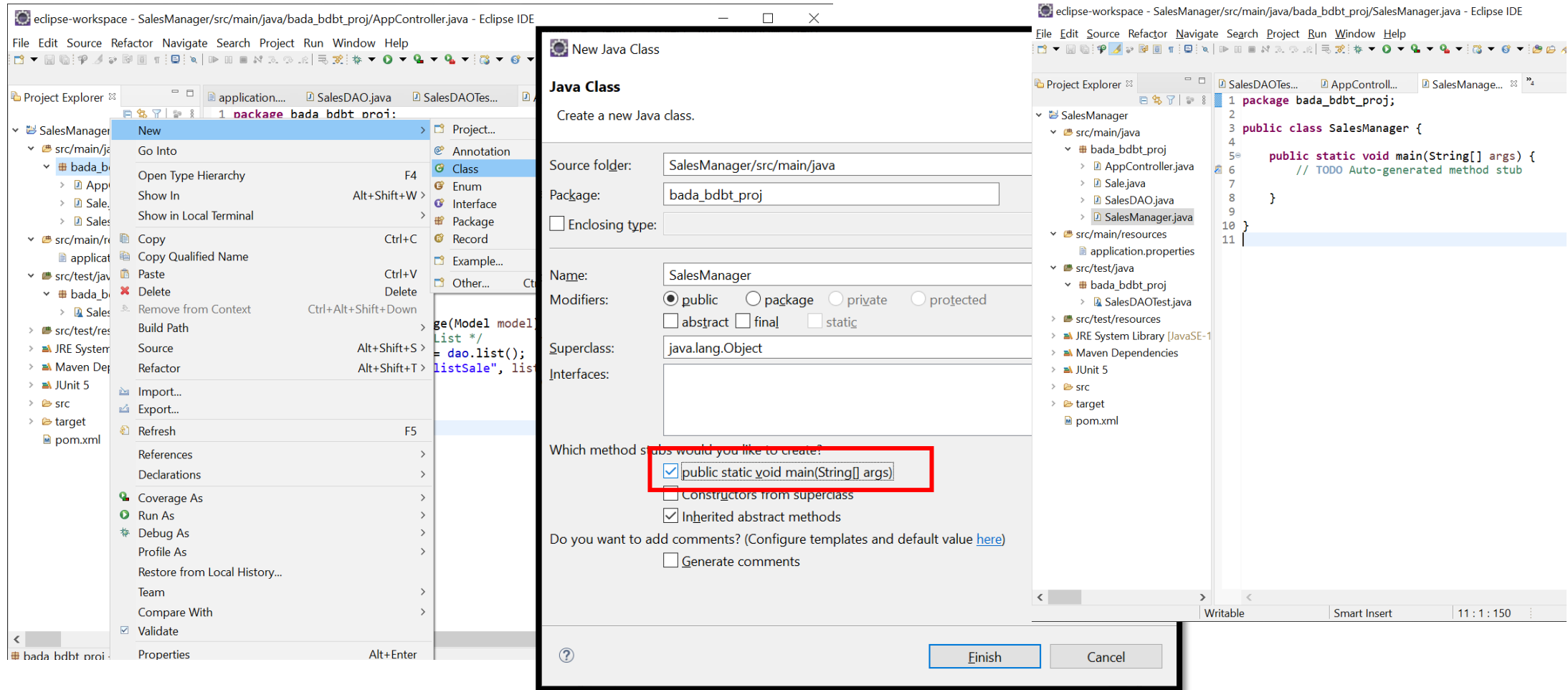
```
1 package bada_bdbt_proj;
2
3 import java.util.List;
4
5 import org.springframework.beans.factory.annotation.Autowired;
6 import org.springframework.stereotype.Controller;
7 import org.springframework.ui.Model;
8 import org.springframework.web.bind.annotation.RequestMapping;
9
10 @Controller
11 public class AppController {
12
13     @Autowired
14     private SalesDAO dao;
15
16     @RequestMapping("/")
17     public String viewHomePage(Model model) {
18         /* Import java.util.List */
19         List<Sale> listSale = dao.list();
20         model.addAttribute("listSale", listSale);
21         return "index";
22     }
23 }
```

7

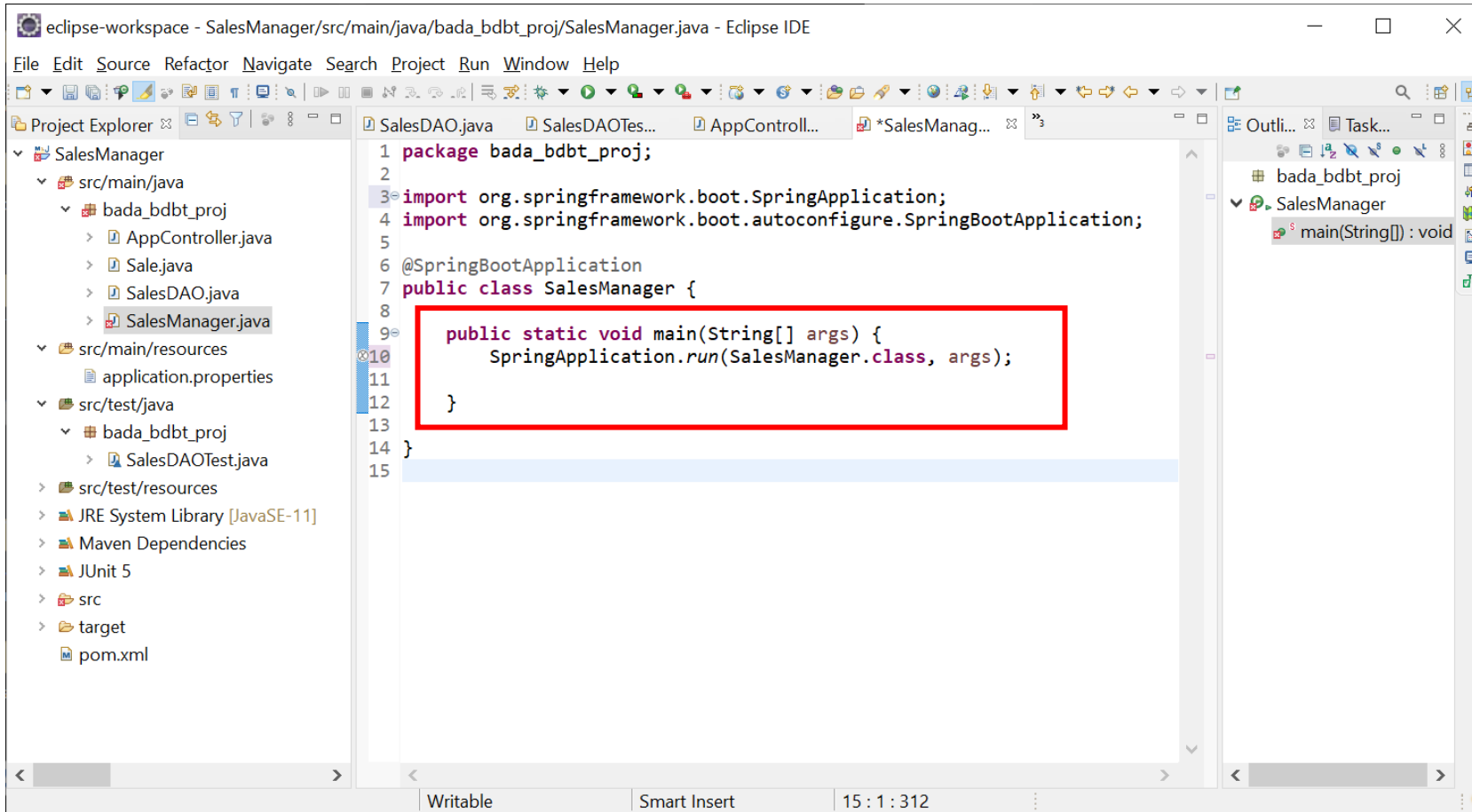
GŁÓWNA KLASA APLIKACJI WEBOWEJ SPRING BOOT „SALES MANAGER”

7. Główna klasa (Sales Manager)

Główna klasa, do uruchomienia aplikacji webowej SpringBoot. Klasa SalesManager.



7. Główna klasa (Sales Manager)

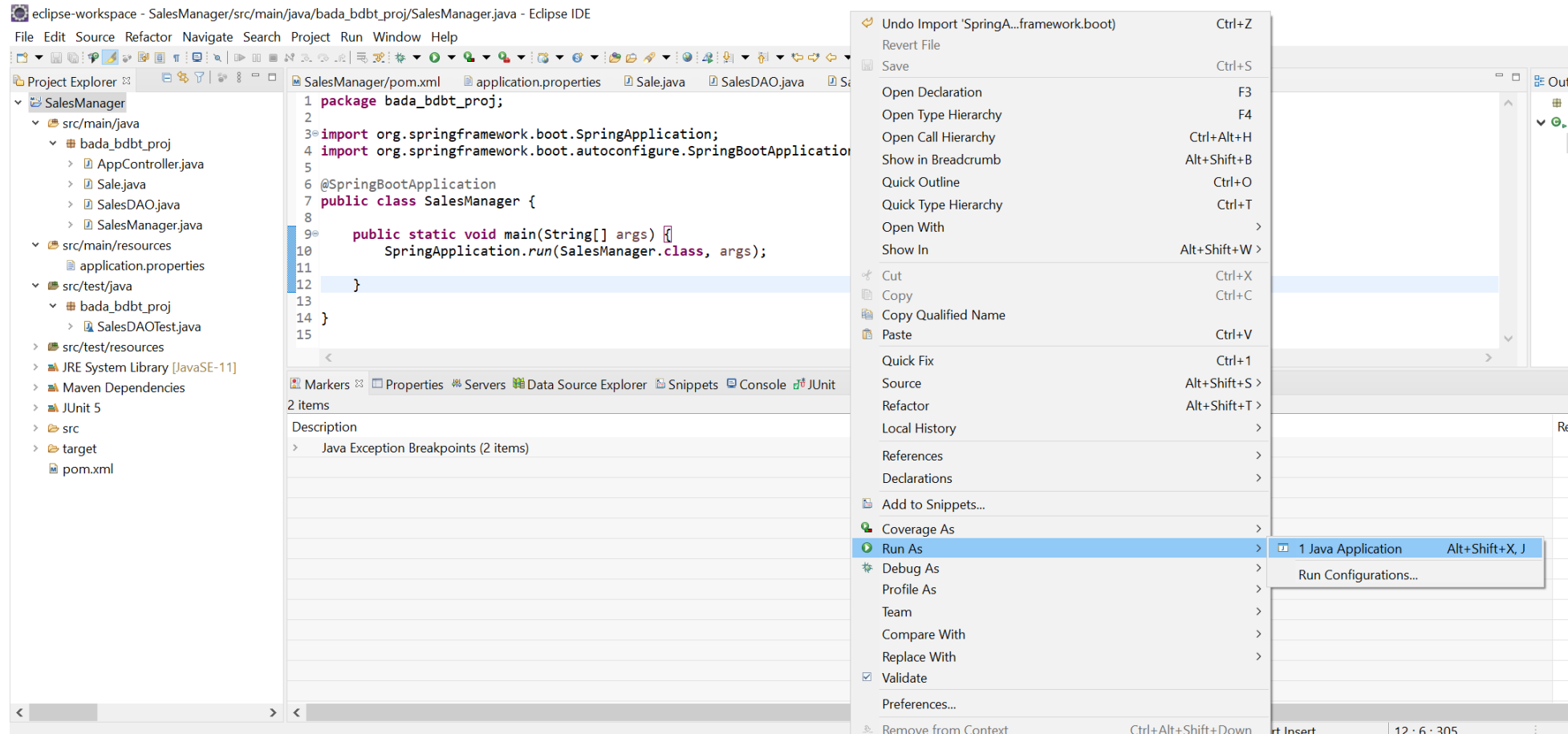


```
1 package bada_bdbt_proj;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6 @SpringBootApplication
7 public class SalesManager {
8
9     public static void main(String[] args) {
10         SpringApplication.run(SalesManager.class, args);
11     }
12 }
13
14
15
```

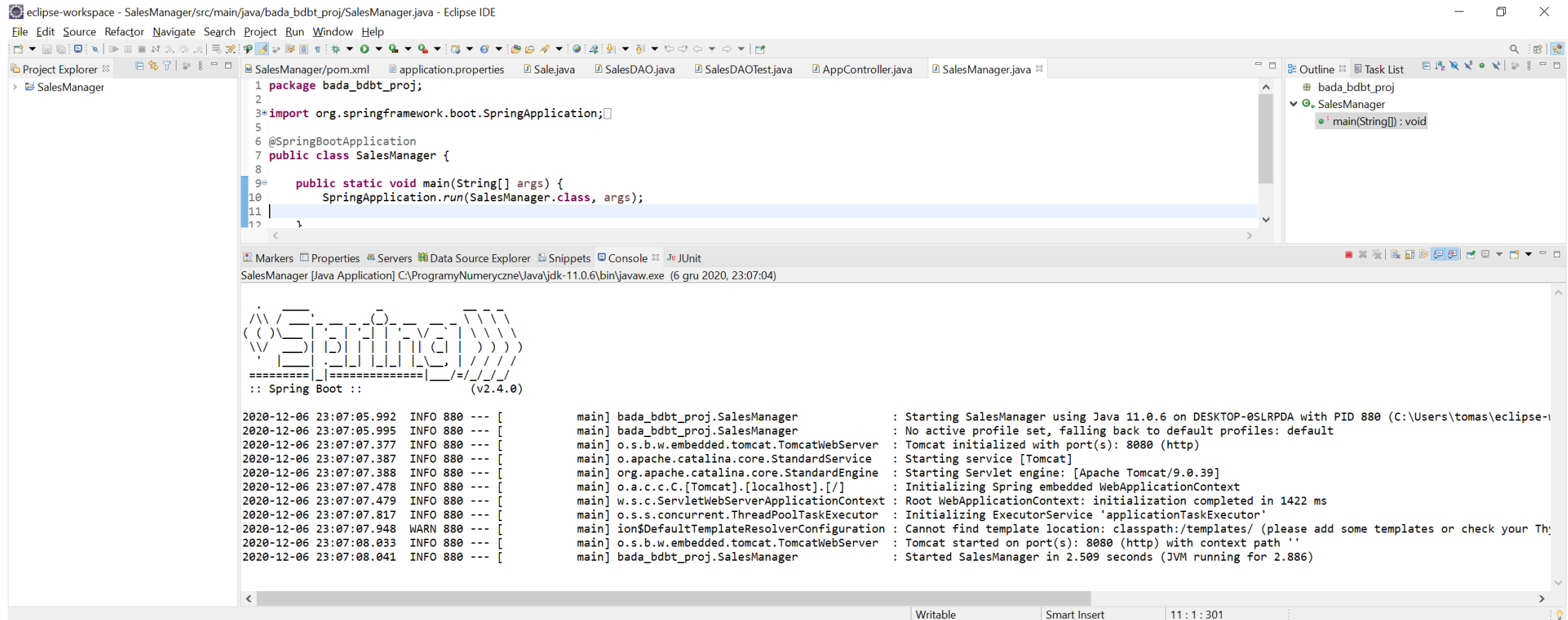
The screenshot shows the Eclipse IDE with the SalesManager.java file open. The code is for a Spring Boot application. A red box highlights the main method. The Project Explorer on the left shows the project structure, including the src/main/java directory where the SalesManager.java file is located. The Outline view on the right shows the class SalesManager with the main method.

Ten program uruchomi wbudowany serwer Tomcat, na którym jest nasza aplikacja webowa SpringBoot.

7. Główna klasa (Sales Manager) (uruchamianie)



7. Główna klasa (Sales Manager) (uruchamianie)



The screenshot shows the Eclipse IDE interface. The main editor displays the `SalesManager.java` file with the following code:

```
1 package bada_bdbt_proj;
2
3 import org.springframework.boot.SpringApplication;
4
5
6 @SpringBootApplication
7 public class SalesManager {
8
9     public static void main(String[] args) {
10         SpringApplication.run(SalesManager.class, args);
11     }
12 }
```

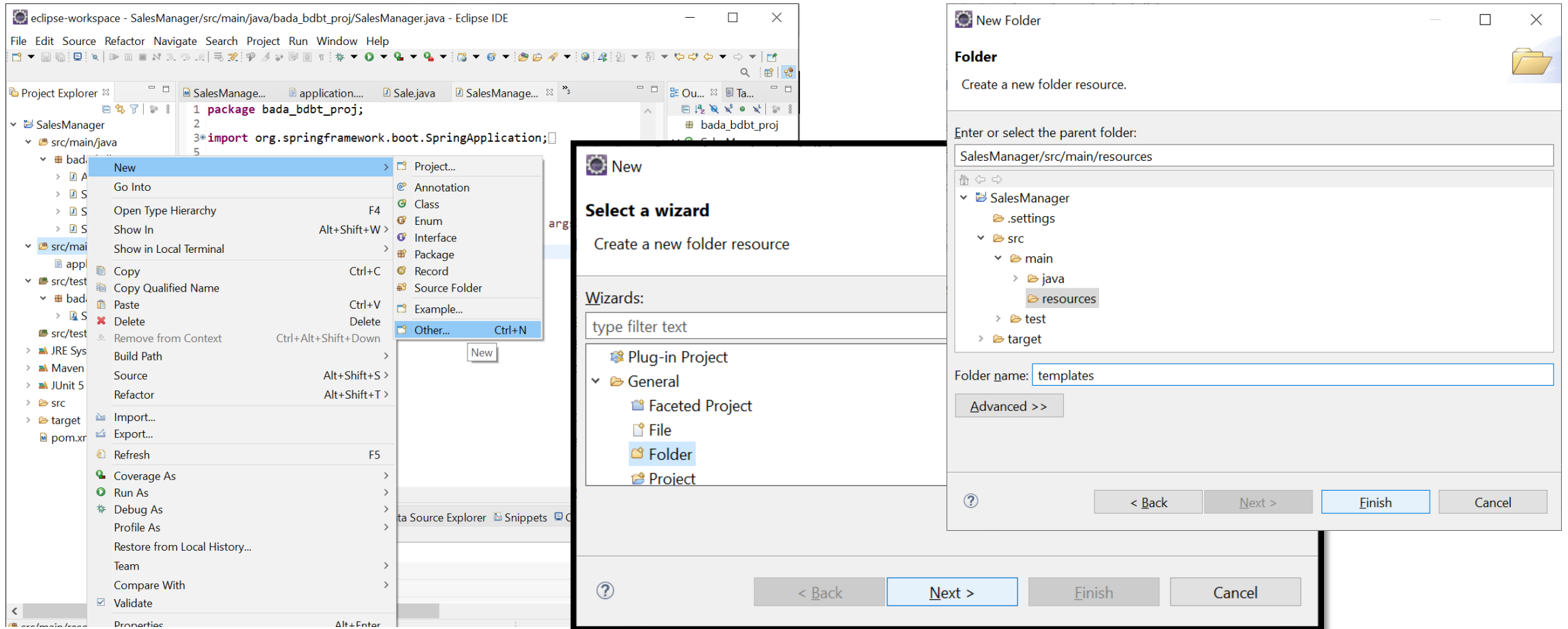
The Project Explorer on the left shows the project structure with `SalesManager` selected. The Outline view on the right shows the `main(String[]) : void` method. The Console view at the bottom shows the output of the application, including the Spring Boot logo and various log messages:

```
2020-12-06 23:07:05.992 INFO 880 --- [main] bada_bdbt_proj.SalesManager : Starting SalesManager using Java 11.0.6 on DESKTOP-0SLRPDA with PID 880 (C:\Users\tomas\eclipse-
2020-12-06 23:07:05.995 INFO 880 --- [main] bada_bdbt_proj.SalesManager : No active profile set, falling back to default profiles: default
2020-12-06 23:07:07.377 INFO 880 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port(s): 8080 (http)
2020-12-06 23:07:07.387 INFO 880 --- [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2020-12-06 23:07:07.388 INFO 880 --- [main] org.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/9.0.39]
2020-12-06 23:07:07.478 INFO 880 --- [main] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2020-12-06 23:07:07.479 INFO 880 --- [main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 1422 ms
2020-12-06 23:07:07.817 INFO 880 --- [main] o.s.s.concurrent.ThreadPoolTaskExecutor : Initializing ExecutorService 'applicationTaskExecutor'
2020-12-06 23:07:07.948 WARN 880 --- [main] ion$DefaultTemplateResolverConfiguration : Cannot find template location: classpath:/templates/ (please add some templates or check your Th
2020-12-06 23:07:08.033 INFO 880 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port(s): 8080 (http) with context path ''
2020-12-06 23:07:08.041 INFO 880 --- [main] bada_bdbt_proj.SalesManager : Started SalesManager in 2.509 seconds (JVM running for 2.886)
```

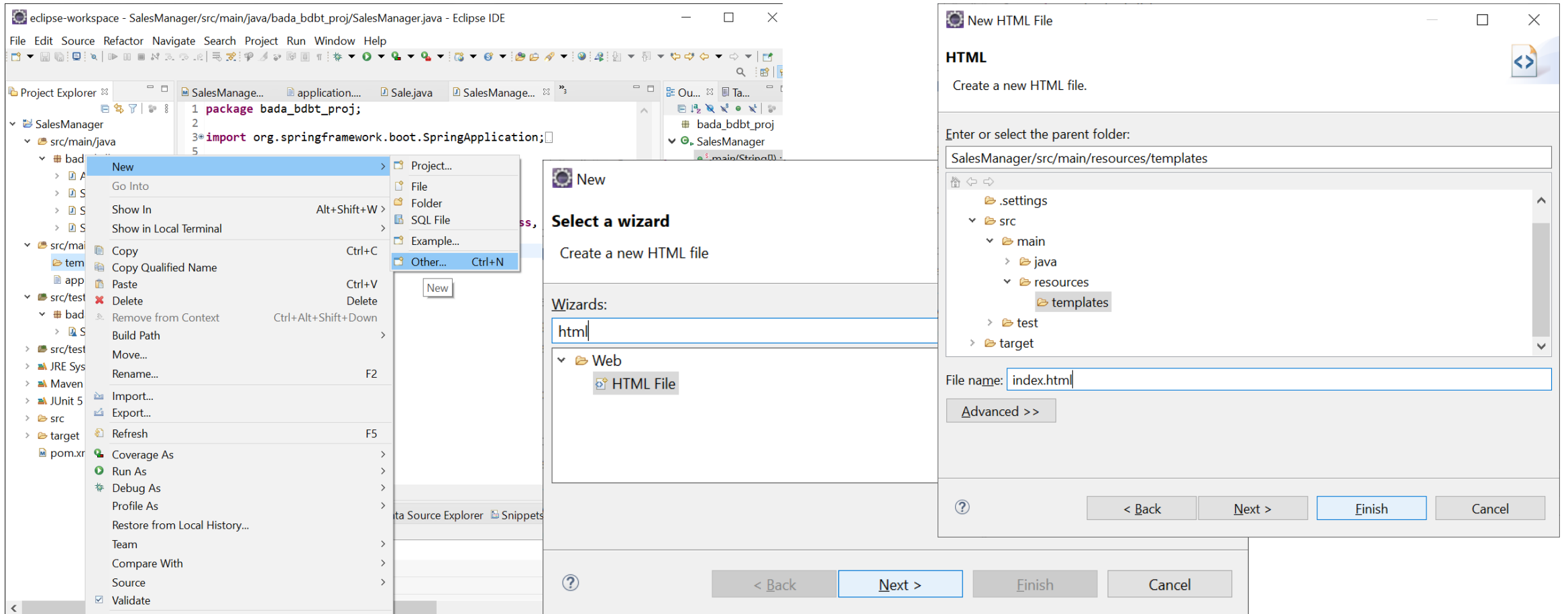
8

PLIK INDEX.HTML – PIERWSZA STRONA

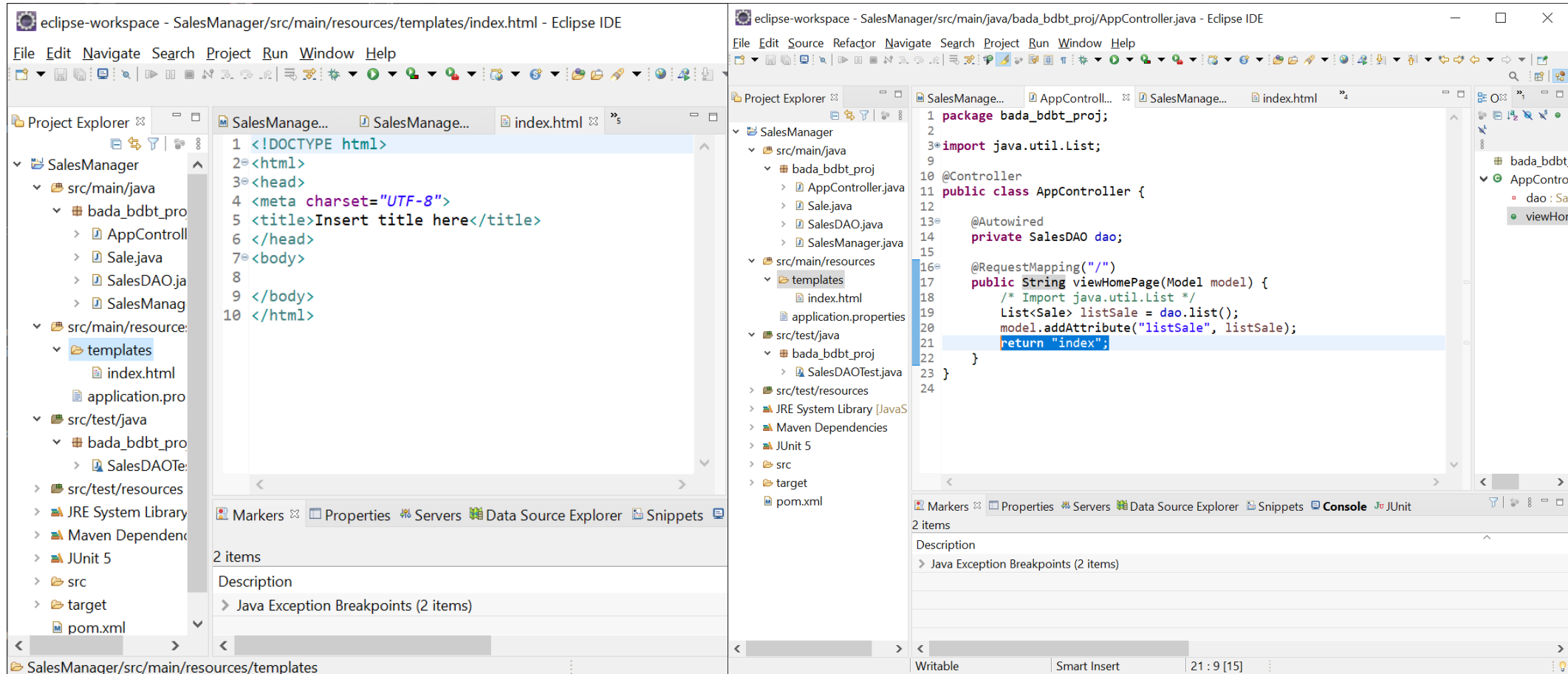
8. Plik index.html



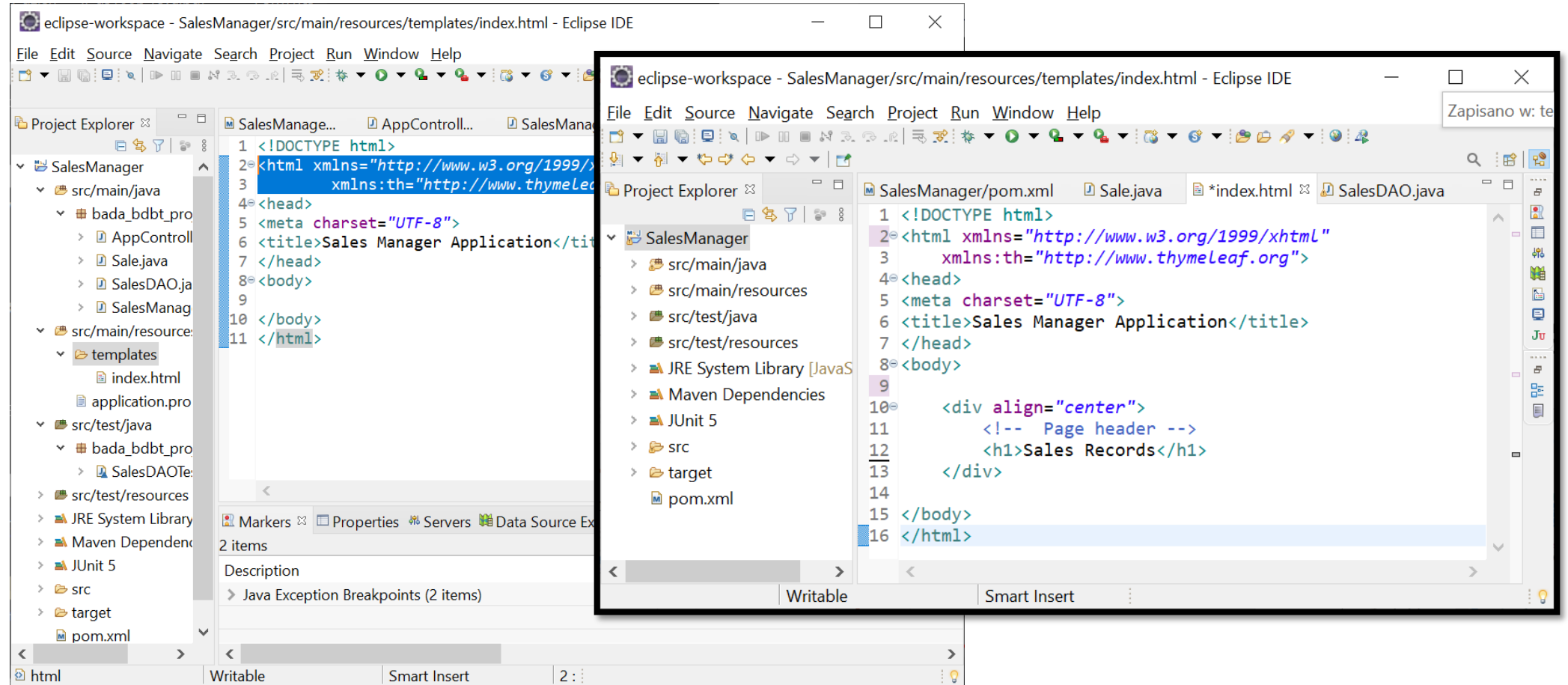
8. Plik index.html



8. Plik index.html



8. Plik index.html



8. Plik index.html (przestrzeń nazw , xmlns: nazwa)

Przestrzenie nazw są potrzebne w każdym dokumencie, który używa więcej niż jednego słownika znaczników (zbioru nazw znaczników).

Za jej wstawienie odpowiada atrybut „xmlns:X”, gdzie X jest skrótową nazwą-aliasem przestrzeni, a wartość tego atrybutu jest jej właściwym identyfikatorem.

Identyfikator taki musi mieć postać adresu URL, choć dokument na który wskazuje nie musi wcale istnieć i nawet jeśli istnieje, nie musi mieć żadnego związku z daną przestrzenią nazw.

Adres ten często wskazuje np. na schemat zawierający definicję języka, którego znaczników zamierzamy używać. Domyślna przestrzeń nazw zawsze posiada www.w3.org i nie trzeba jej podawać.

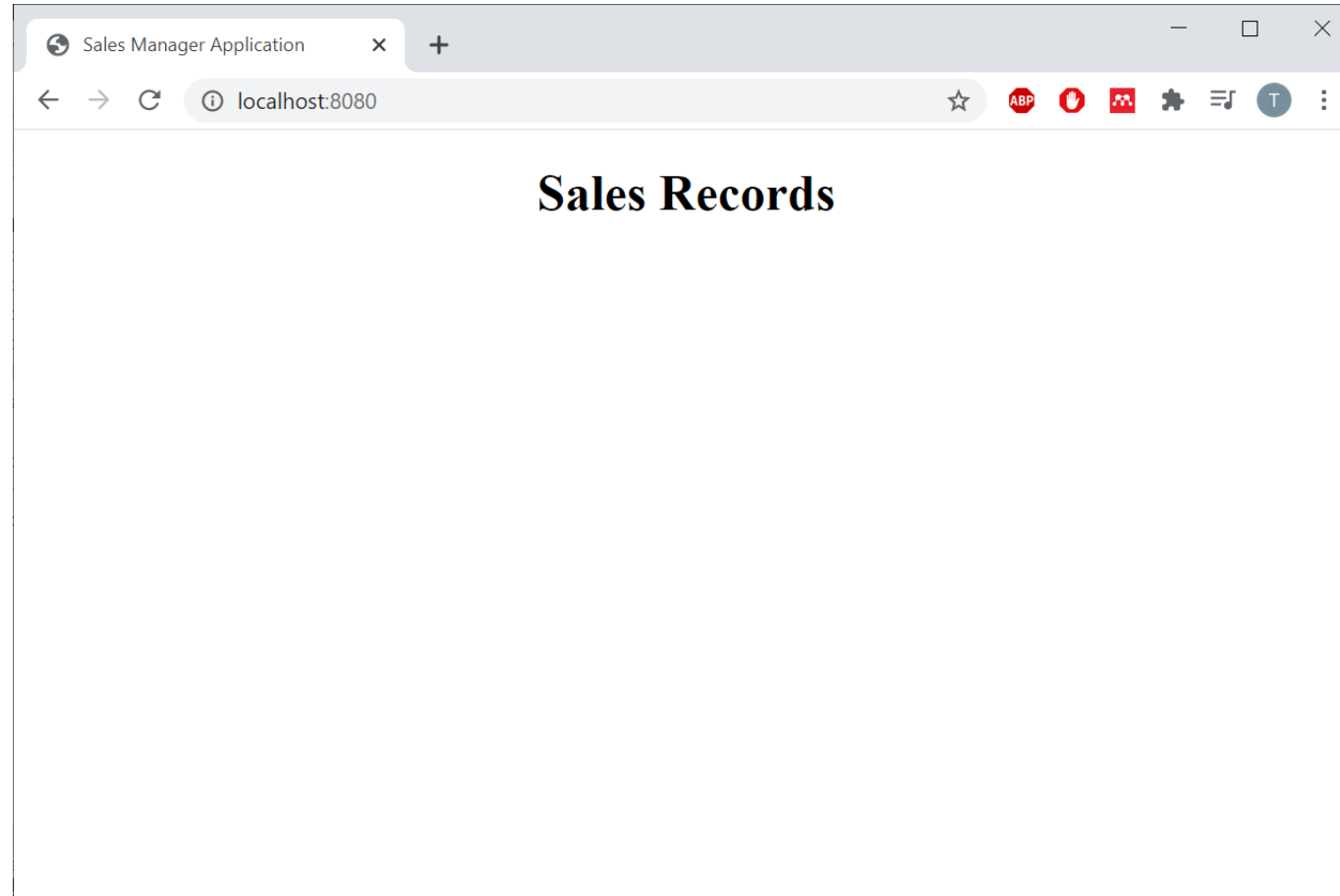
8. Plik index.html (przestrzeń nazw, thymeleaf)

Thymeleaf to silnik szablonów umożliwiający budowę elementów interfejsu użytkownika po stronie serwera (backendu).

Pozwala definiować własne atrybuty szablonów lub znaczników. Domyślnie zawiera standardowe dialekty (Standard i SpringStandard), które definiują bogaty zestaw funkcjonalności.

Standardowe dialekty są łatwe do rozpoznania, gdyż zawierają atrybuty zaczynające się od przedrostka **th**. Thymeleaf może być integrowany ze Springiem (dialekt SpringStandard).

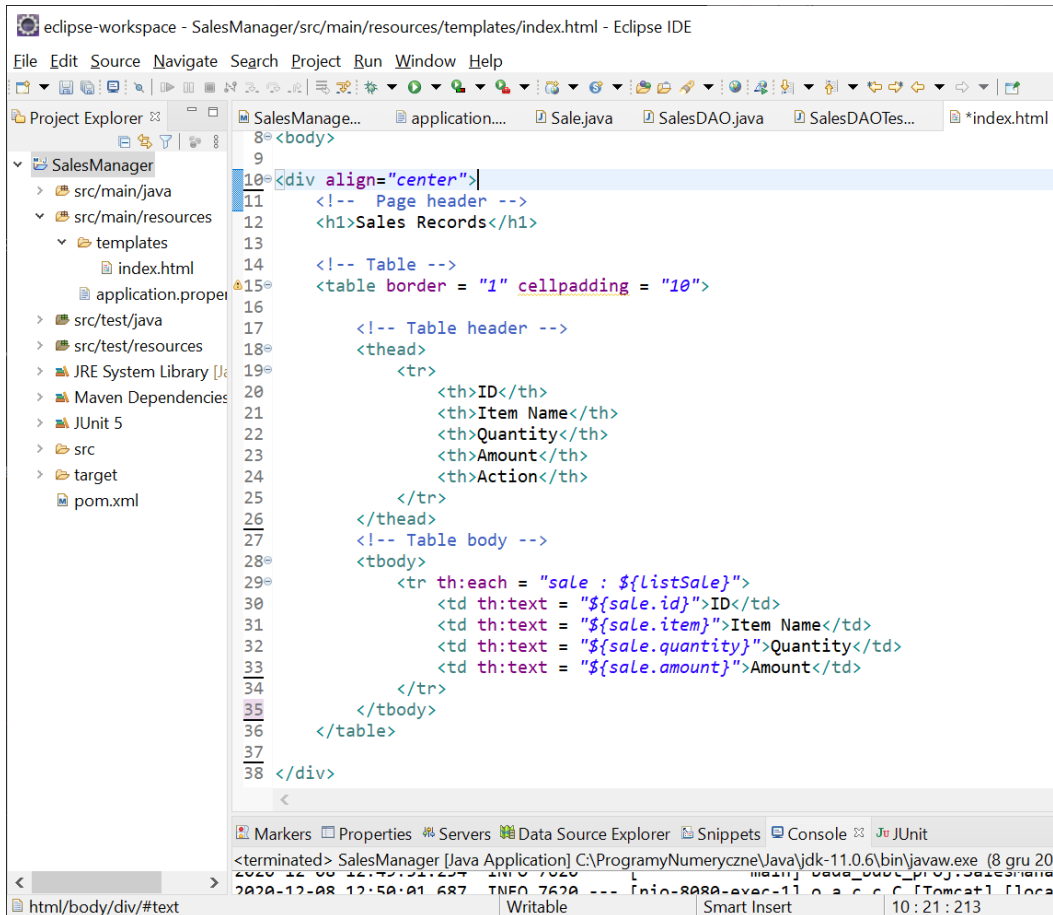
8. Plik index.html



9

PLIK INDEX.HTML – TABELA Z DANYMI Z BAZY

9. Plik index.html – tabela z danymi z bazy

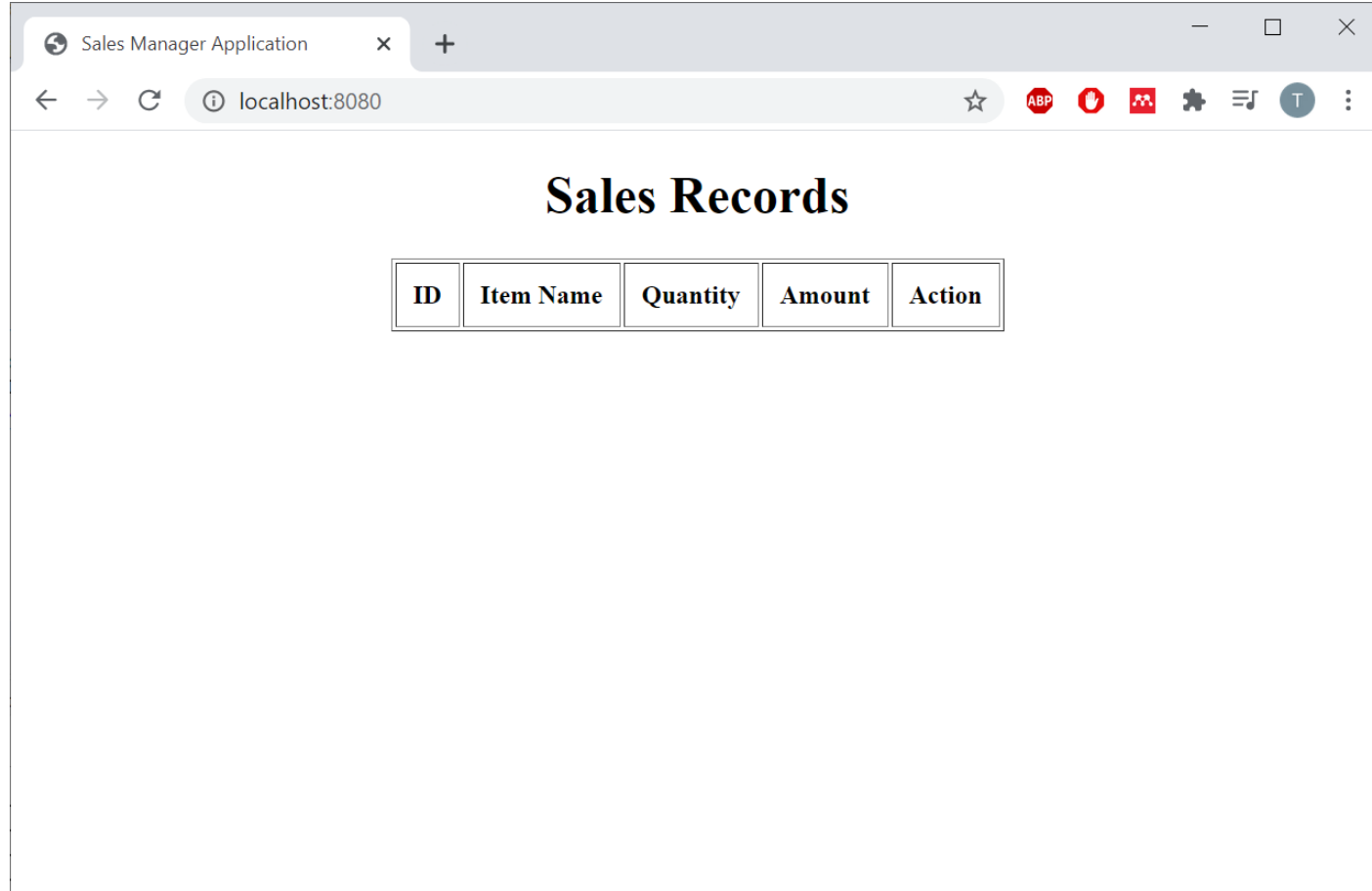


```
8<body>
9
10<div align="center">
11  <!-- Page header -->
12  <h1>Sales Records</h1>
13
14  <!-- Table -->
15  <table border = "1" cellpadding = "10">
16
17    <!-- Table header -->
18    <thead>
19      <tr>
20        <th>ID</th>
21        <th>Item Name</th>
22        <th>Quantity</th>
23        <th>Amount</th>
24        <th>Action</th>
25      </tr>
26    </thead>
27    <!-- Table body -->
28    <tbody>
29      <tr th:each = "sale : ${ListSale}">
30        <td th:text = "${sale.id}">ID</td>
31        <td th:text = "${sale.item}">Item Name</td>
32        <td th:text = "${sale.quantity}">Quantity</td>
33        <td th:text = "${sale.amount}">Amount</td>
34      </tr>
35    </tbody>
36  </table>
37
38</div>
```

```
<!-- Table -->
<table border = "1" cellpadding = "10">      <!-- border -
obramowanie, cellpadding - rozmiar komórki -->

<!-- Table header -->
<thead>
  <tr>
    <th>ID</th>
    <th>Item Name</th>
    <th>Quantity</th>
    <th>Amount</th>
    <th>Action</th>
  </tr>
</thead>
<!-- Table body -->
<tbody>
  <tr th:each = "sale : ${ListSale}">
    <td th:text = "${sale.id}">ID</td>
    <td th:text = "${sale.item}">Item Name</td>
    <td th:text = "${sale.quantity}">Quantity</td>
    <td th:text = "${sale.amount}">Amount</td>
  </tr>
</tbody>
</table>
```

9. Plik index.html – tabela z danymi z bazy



The screenshot shows a web browser window with the title 'Sales Manager Application' and the address bar displaying 'localhost:8080'. The main content area features a heading 'Sales Records' and a table with five columns: ID, Item Name, Quantity, Amount, and Action.

ID	Item Name	Quantity	Amount	Action
----	-----------	----------	--------	--------

9. Plik index.html – tabela z danymi z bazy (test)

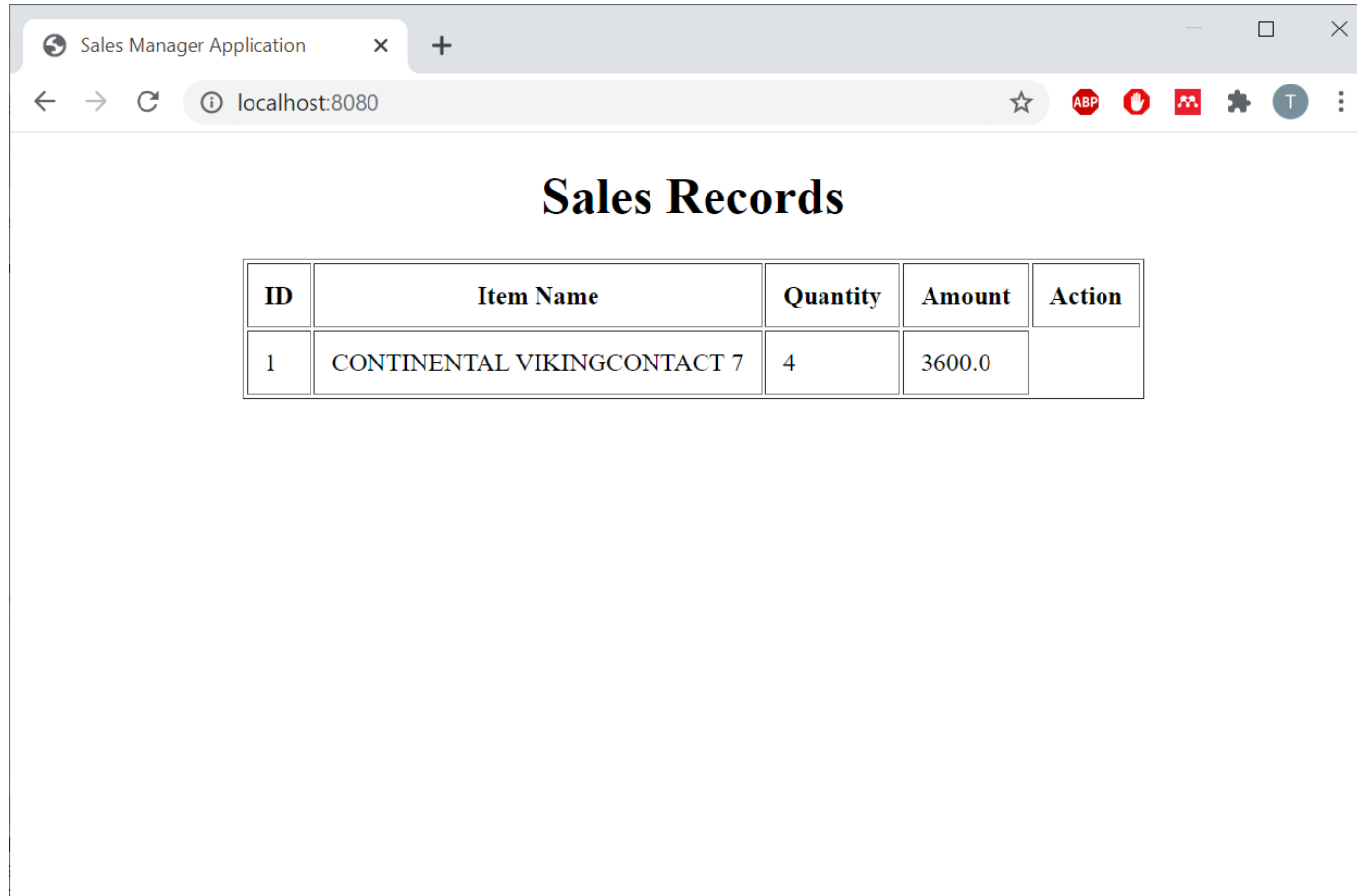
The image displays three overlapping screenshots of the Oracle SQL Developer interface, illustrating the steps to insert data into a table.

Left Screenshot: Shows the 'Connections' pane on the left with 'XE' selected. The 'Tables (Filtered)' list shows 'SALES'. The 'Columns' pane for 'SALES' lists: 1 ID, 2 ITEM, 3 QUANTITY, 4 AMOUNT.

Middle Screenshot: Shows the 'Insert Row (Ctrl+I)' dialog box. The 'Columns' tab is active, and the 'SALES' table is selected. The 'Columns' list shows: ID, ITEM, QUANTITY, AMOUNT. The 'Values' list shows: (null), (null), (null), (null).

Right Screenshot: Shows the 'Commit Changes (F11)' dialog box. The 'Columns' tab is active, and the 'SALES' table is selected. The 'Columns' list shows: ID, ITEM, QUANTITY, AMOUNT. The 'Values' list shows: +1, (null), CONTINE..., 4, 3600.

9. Plik index.html – tabela z danymi z bazy (test)



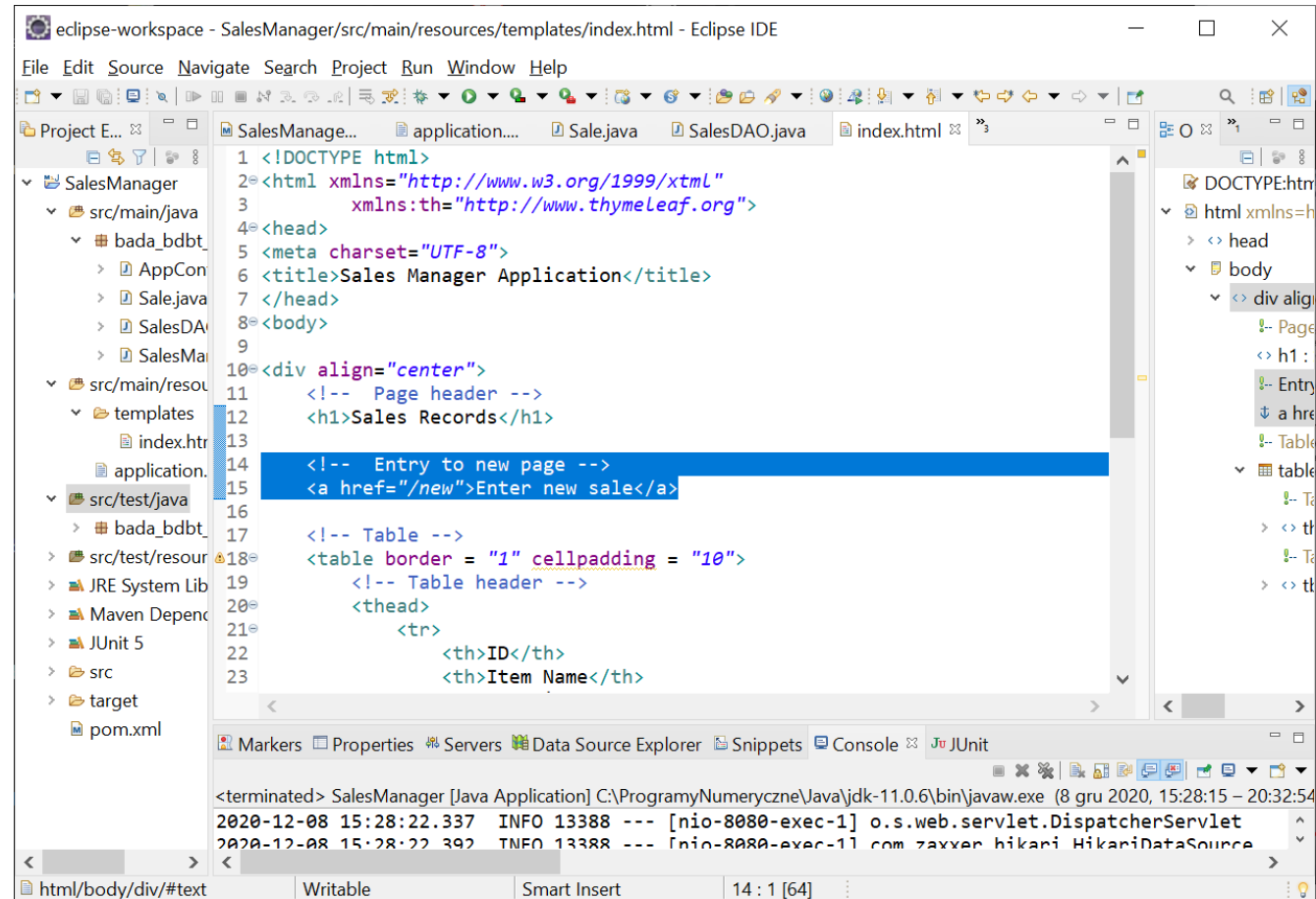
The screenshot shows a web browser window with the title 'Sales Manager Application' and the address bar displaying 'localhost:8080'. The page content features a table titled 'Sales Records' with the following data:

ID	Item Name	Quantity	Amount	Action
1	CONTINENTAL VIKINGCONTACT 7	4	3600.0	

10

**PLIK INDEX.HTML – DODAWANIE NOWYCH
REKODÓW (CREATE)**

10. Plik index.html – dodawanie nowych rekordów (create)



```
1 <!DOCTYPE html>
2 <html xmlns="http://www.w3.org/1999/xhtml"
3     xmlns:th="http://www.thymeleaf.org">
4 <head>
5 <meta charset="UTF-8">
6 <title>Sales Manager Application</title>
7 </head>
8 <body>
9
10 <div align="center">
11 <!-- Page header -->
12 <h1>Sales Records</h1>
13
14 <!-- Entry to new page -->
15 <a href="/new">Enter new sale</a>
16
17 <!-- Table -->
18 <table border = "1" cellpadding = "10">
19 <!-- Table header -->
20 <thead>
21 <tr>
22 <th>ID</th>
23 <th>Item Name</th>
```

Console output:

```
<terminated> SalesManager [Java Application] C:\ProgramyNumeryczne\Java\jdk-11.0.6\bin\javaw.exe (8 gru 2020, 15:28:15 – 20:32:54)
2020-12-08 15:28:22.337 INFO 13388 --- [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet
2020-12-08 15:28:22.392 INFO 13388 --- [nio-8080-exec-1] com.zaxxer.hikari.HikariDataSource
```

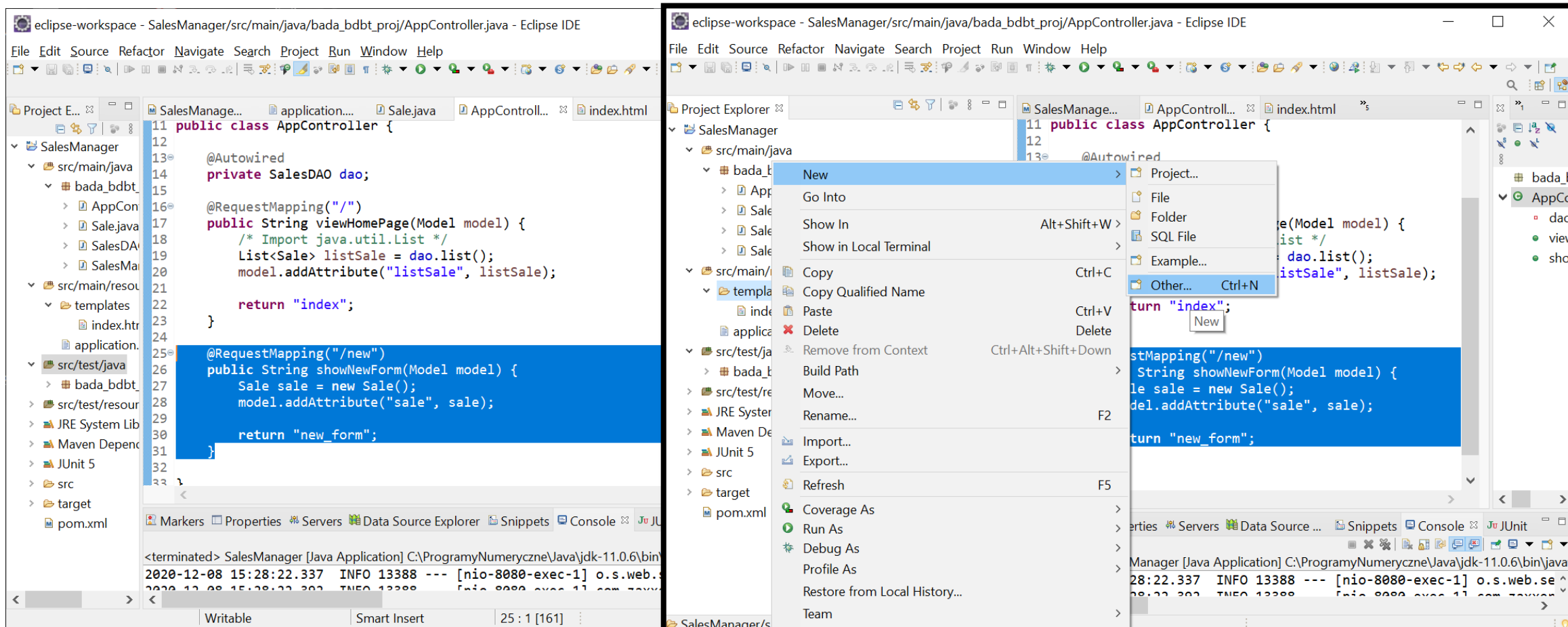
Sale sale = new Sale();

@RequestMapping("/new")

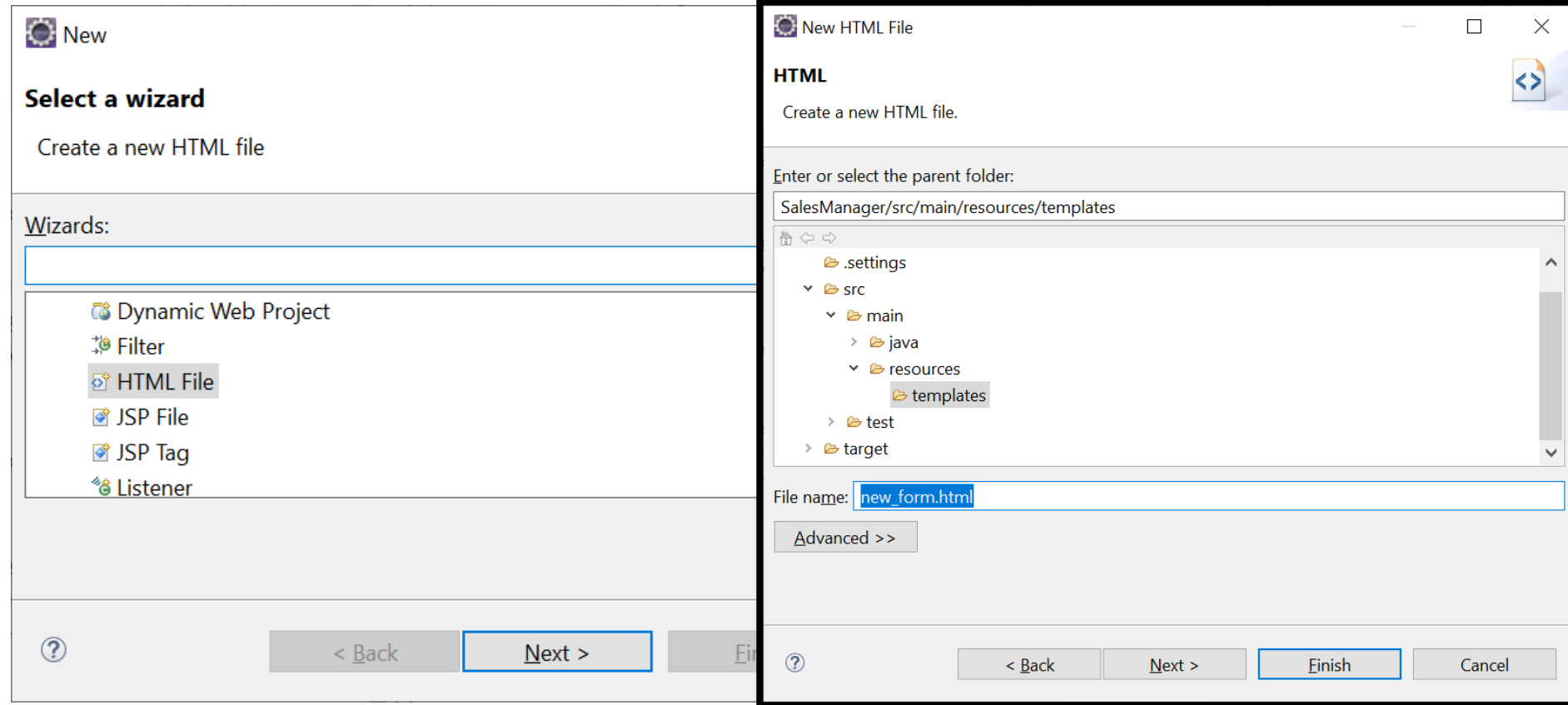
- nowy obiekt

- przejście do nowego okna, zapisanego pod nazwą „new_form.html”

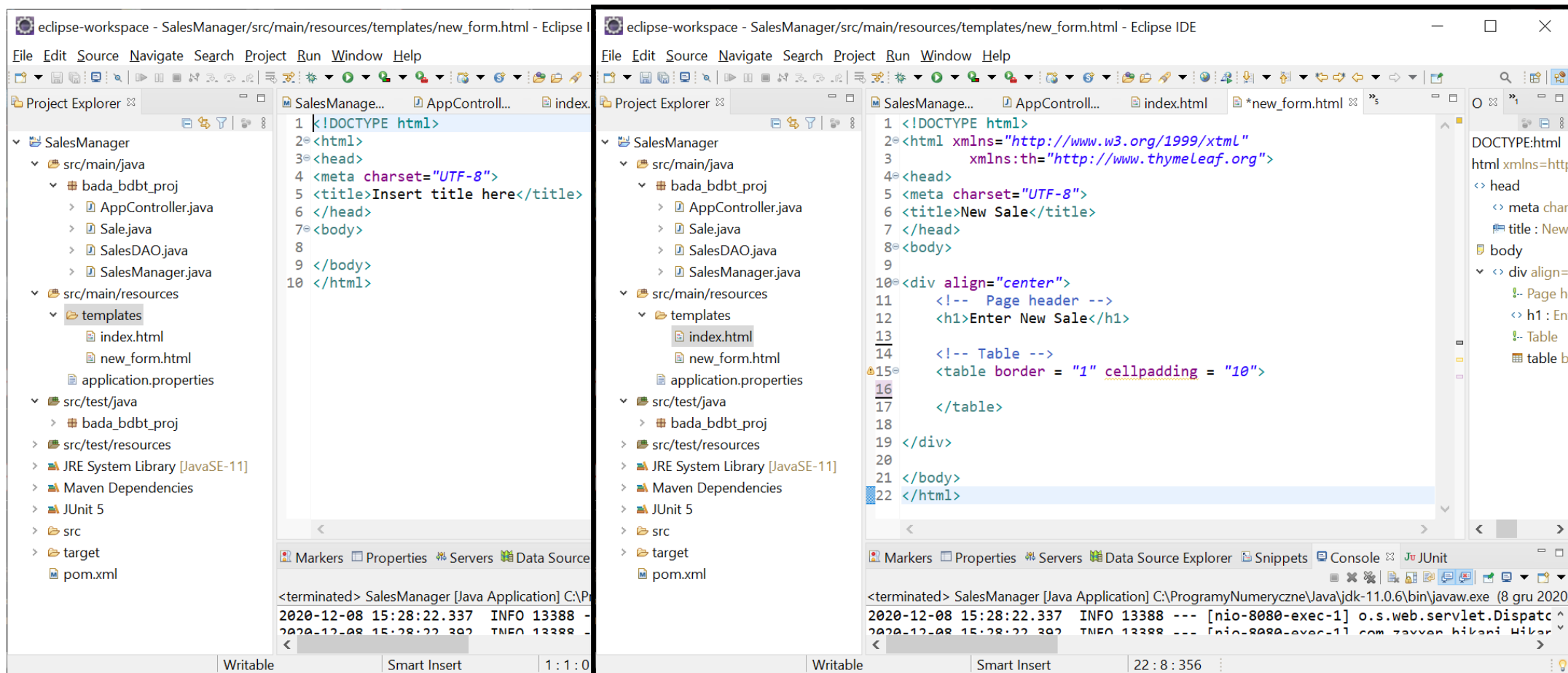
10. Plik index.html – dodawanie nowych rekordów (create)



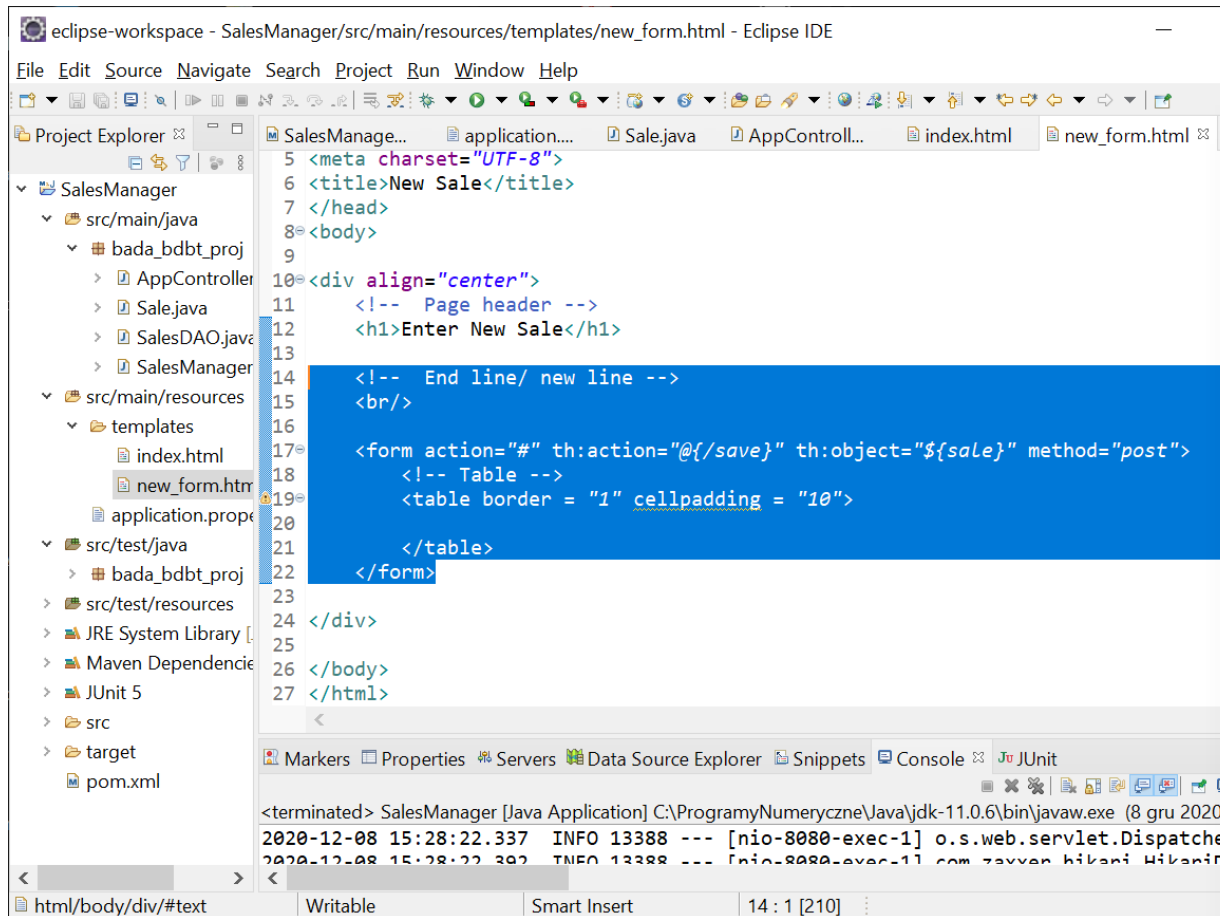
10. Plik index.html – dodawanie nowych rekordów (create)



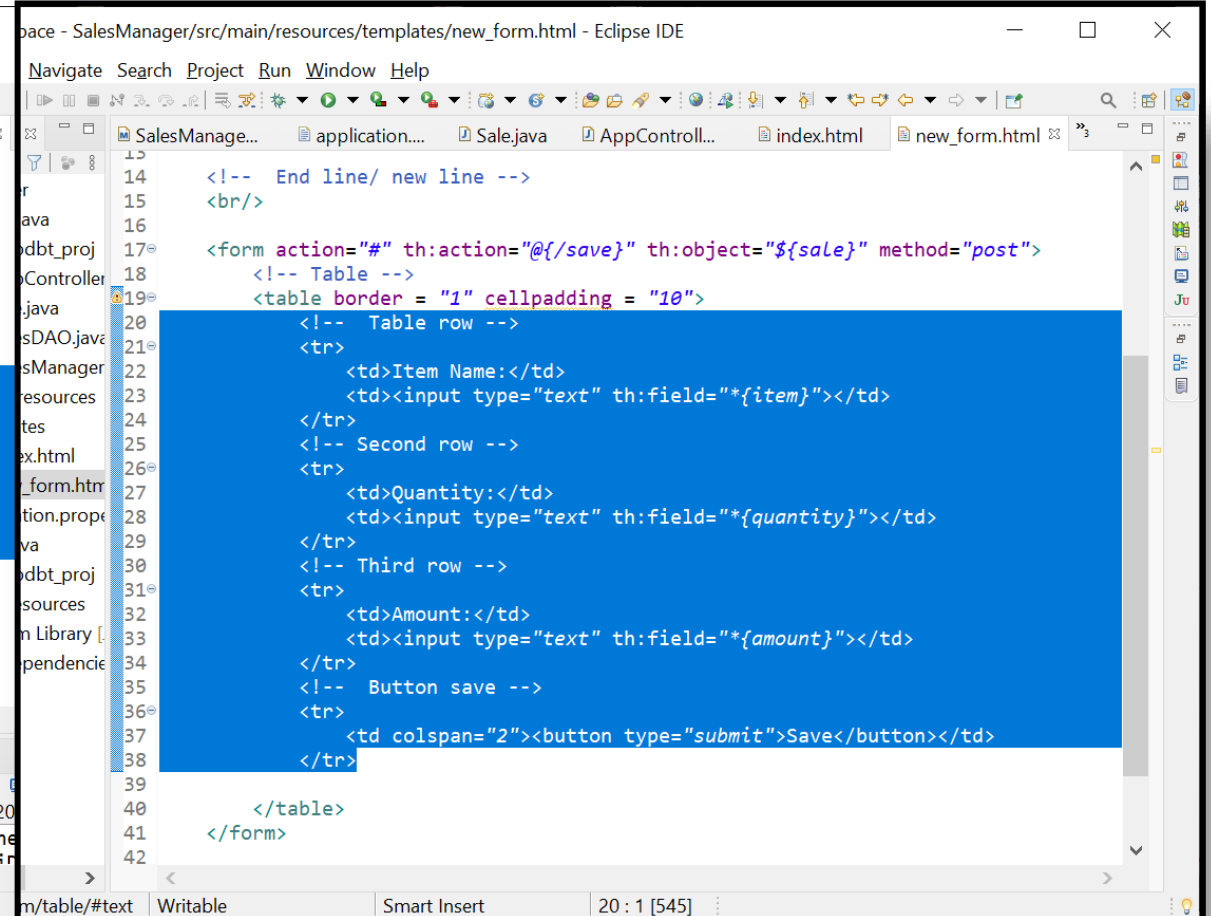
10. Plik index.html – dodawanie nowych rekordów (create)



10. Plik index.html – dodawanie nowych rekordów (create)

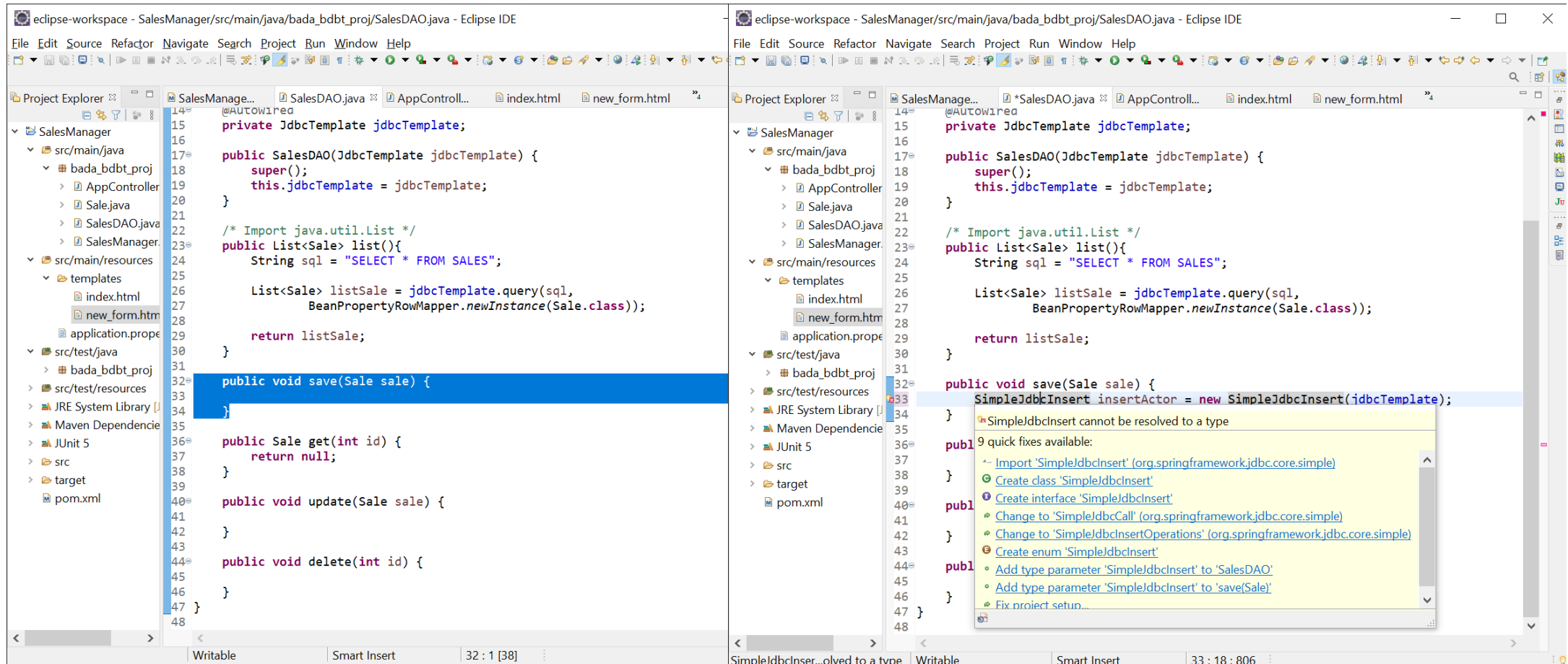


```
5 <meta charset="UTF-8">
6 <title>New Sale</title>
7 </head>
8 <body>
9
10 <div align="center">
11   <!-- Page header -->
12   <h1>Enter New Sale</h1>
13
14   <!-- End line/ new line -->
15   <br/>
16
17   <form action="#" th:action="@{/save}" th:object="${sale}" method="post">
18     <!-- Table -->
19     <table border = "1" cellpadding = "10">
20
21     </table>
22   </form>
23
24 </div>
25
26 </body>
27 </html>
```

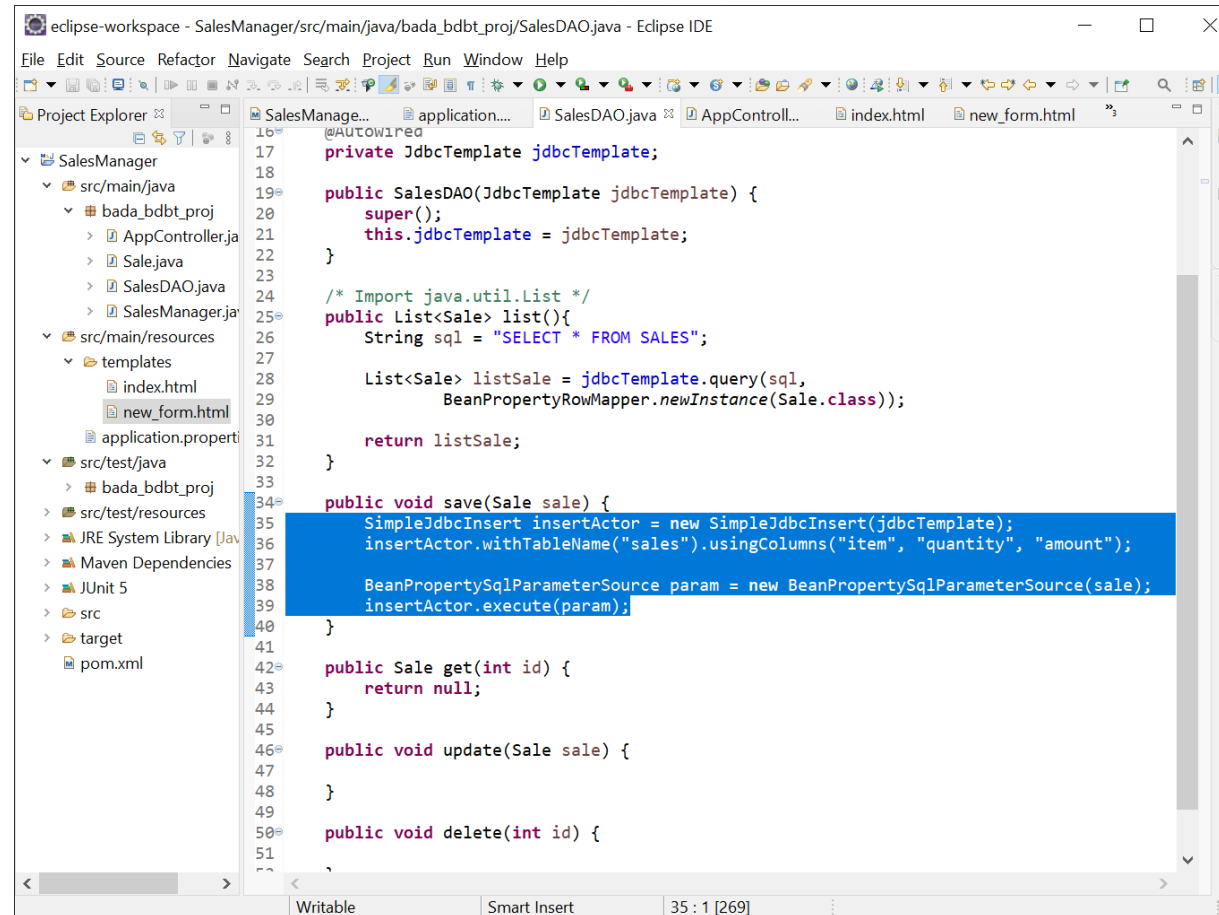


```
15
16
17 <form action="#" th:action="@{/save}" th:object="${sale}" method="post">
18   <!-- Table -->
19   <table border = "1" cellpadding = "10">
20     <!-- Table row -->
21     <tr>
22       <td>Item Name:</td>
23       <td><input type="text" th:field="**{item}"></td>
24     </tr>
25     <!-- Second row -->
26     <tr>
27       <td>Quantity:</td>
28       <td><input type="text" th:field="**{quantity}"></td>
29     </tr>
30     <!-- Third row -->
31     <tr>
32       <td>Amount:</td>
33       <td><input type="text" th:field="**{amount}"></td>
34     </tr>
35     <!-- Button save -->
36     <tr>
37       <td colspan="2"><button type="submit">Save</button></td>
38     </tr>
39   </table>
40 </form>
41
42
```

10. Plik index.html – dodawanie nowych rekordów (create)

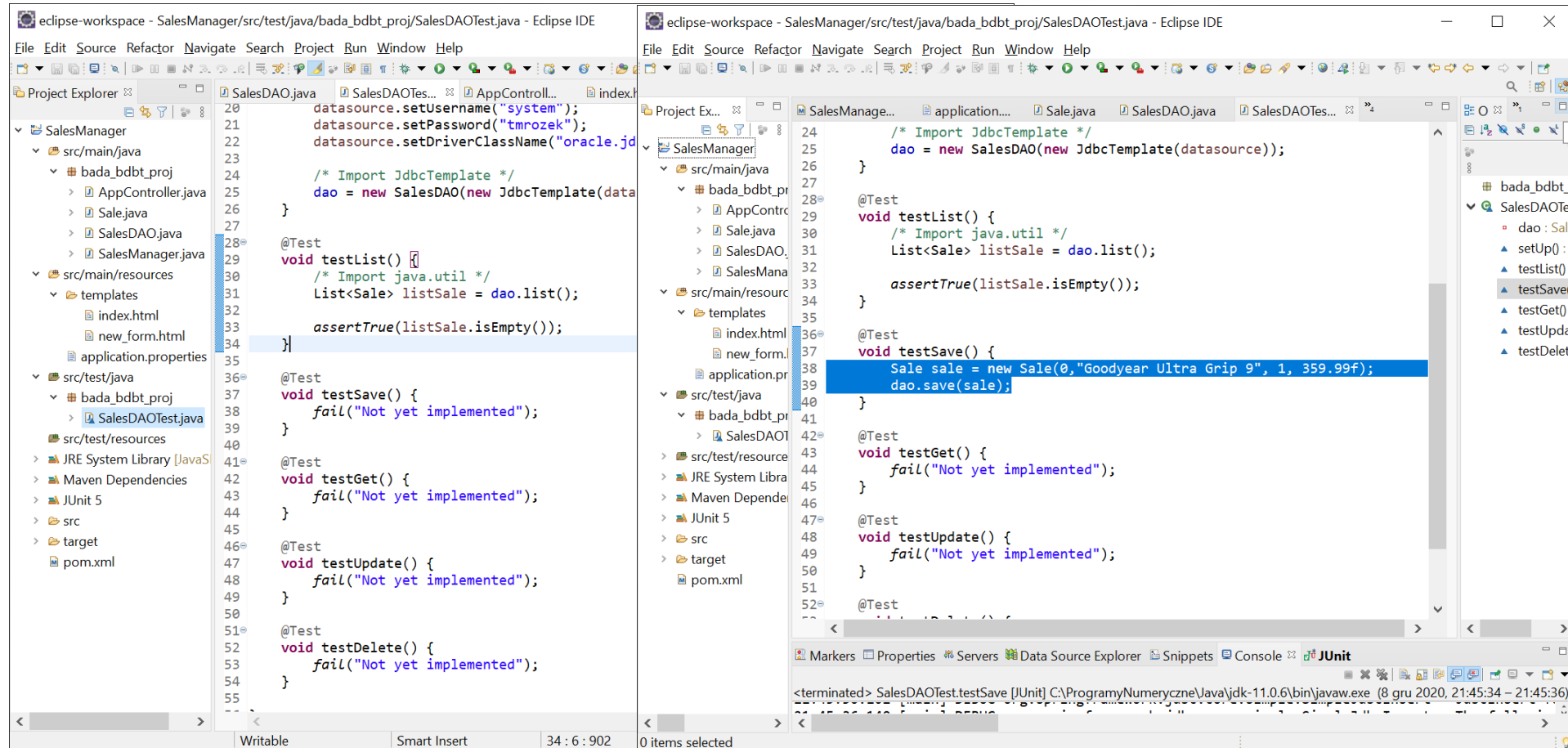


10. Plik index.html – dodawanie nowych rekordów (create)



```
16  
17 private JdbcTemplate jdbcTemplate;  
18  
19 public SalesDAO(JdbcTemplate jdbcTemplate) {  
20     super();  
21     this.jdbcTemplate = jdbcTemplate;  
22 }  
23  
24 /* Import java.util.List */  
25 public List<Sale> list(){  
26     String sql = "SELECT * FROM SALES";  
27  
28     List<Sale> listSale = jdbcTemplate.query(sql,  
29         BeanPropertyRowMapper.newInstance(Sale.class));  
30  
31     return listSale;  
32 }  
33  
34 public void save(Sale sale) {  
35     SimpleJdbcInsert insertActor = new SimpleJdbcInsert(jdbcTemplate);  
36     insertActor.withTableName("sales").usingColumns("item", "quantity", "amount");  
37  
38     BeanPropertySqlParameterSource param = new BeanPropertySqlParameterSource(sale);  
39     insertActor.execute(param);  
40 }  
41  
42 public Sale get(int id) {  
43     return null;  
44 }  
45  
46 public void update(Sale sale) {  
47  
48 }  
49  
50 public void delete(int id) {  
51  
52 }
```

10. Plik index.html – dodawanie nowych rekordów (create) (JUnit test)



10. Plik index.html – dodawanie nowych rekordów (create) (JUnit test)

The screenshot displays the Eclipse IDE interface. The main editor shows the `SalesDAOTest.java` file with the following code:

```
24  /* Import JdbcTemplate */
25  dao = new SalesDAO(new JdbcTemplate(datasource));
26  }
27
28  @Test
29  void testList() {
30      /* Import java.util */
31      List<Sale> listSale = dao.list();
32
33      assertTrue(listSale.isEmpty());
34  }
35
36  @Test
37  void testSave() {
38      Sale sale = new Sale(0, "Goodyear Ultra Grip 9", 1, 359.99f);
39      dao.save(sale);
40  }
41
42  @Test
43  void testGet() {
44      fail("Not yet implemented");
45  }
46
47  @Test
48  void testUpdate() {
49      fail("Not yet implemented");
50  }
51
52  @Test
53  void testDelete() {
54      fail("Not yet implemented");
55  }
```

The `testSave()` method is highlighted. A context menu is open over the code, showing options like "Run As", "Debug As", and "Profile As". The "Run As" option is selected.

The console output shows the test execution results:

```
<terminated> SalesDAOTest.testSave [JUnit] C:\Programy\Numeryczne\Java\jdk-11.0.6\bin\javaw.exe (8 gru 2020, 21:45:34 - 21:45:36)
21:45:36.140 [main] DEBUG org.springframework.jdbc.core.simple.SimpleJdbcInsert - The following parameters are used for insert INSERT INTO sales (item, qual
21:45:36.140 [main] DEBUG org.springframework.jdbc.core.simple.SimpleJdbcInsert - Executing prepared SQL update
```

10. Plik index.html – dodawanie nowych rekordów (create) (JUnit test)

The image displays two side-by-side screenshots of the Oracle SQL Developer application, illustrating the state of the 'SALES' table before and after a data refresh operation.

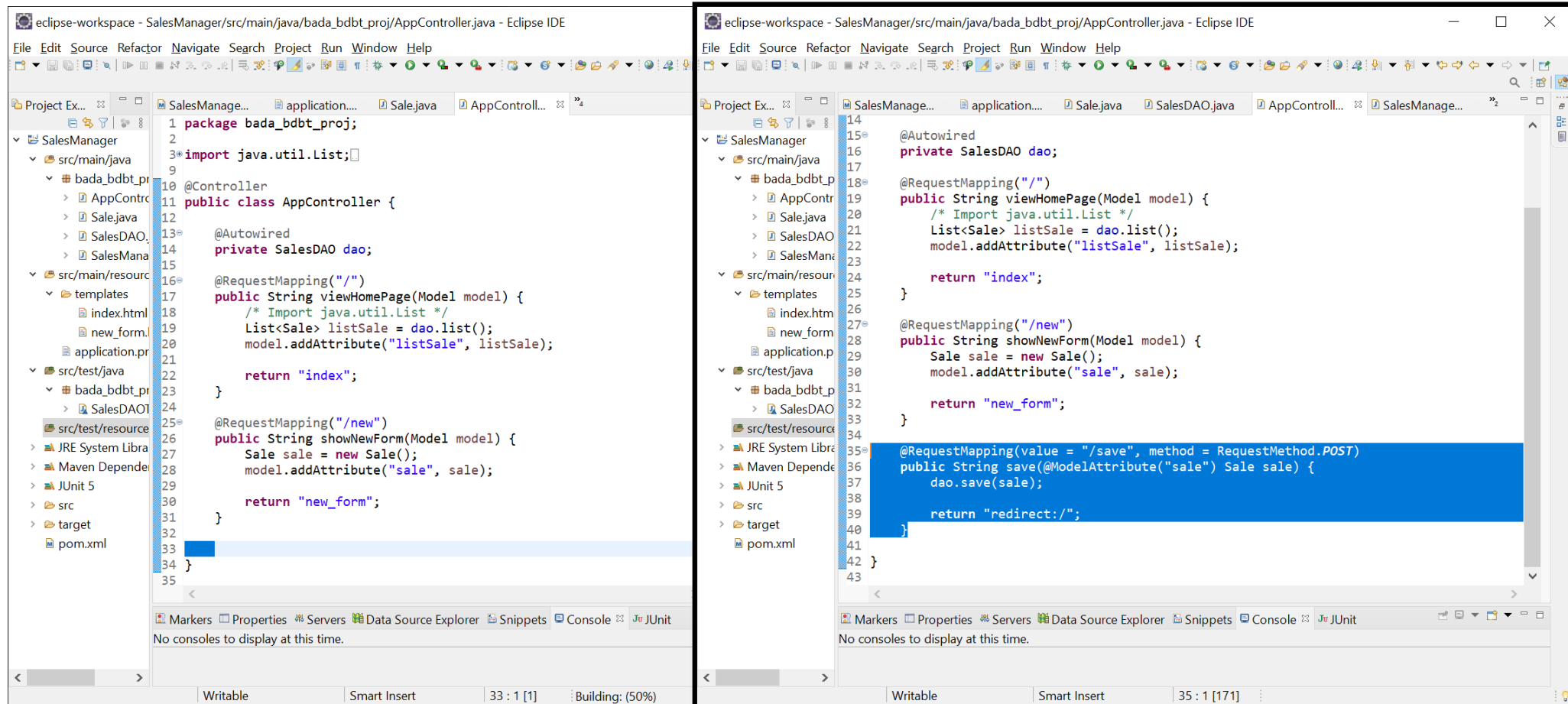
Left Screenshot (Before Refresh):

- The 'Tables (Filtered)' pane on the left shows the 'SALES' table selected.
- The 'Data' tab is active, displaying the 'SALES' table with columns: ID, ITEM, QUANTITY, and AMOUNT.
- The table contains one visible record: ID 1, ITEM 'CONTINE...', QUANTITY 4, and AMOUNT 3600.
- A tooltip 'Refresh (Ctrl+R)' is visible over the 'Refresh' button in the toolbar.
- The 'Messages - Log' pane at the bottom shows the execution of a DELETE statement and a successful commit.

Right Screenshot (After Refresh):

- The 'Data' tab is active, displaying the 'SALES' table with columns: ID, ITEM, QUANTITY, and AMOUNT.
- The table now contains two records: ID 1, ITEM 'CONTINE...', QUANTITY 4, AMOUNT 3600; and ID 2, ITEM '27 Goodyea...', QUANTITY 1, AMOUNT 359,98...
- The 'Messages - Log' pane at the bottom shows the execution of a DELETE statement and a successful commit.

10. Plik index.html – dodawanie nowych rekordów (create)



10. Plik index.html – dodawanie nowych rekordów (create) (test)

Sales Manager Application

localhost:8080

Sales Records

[Enter new sale](#)

ID	Item Name	Quantity	Amount
1	CONTINENTAL VIKINGCONTACT 7	4	360
27	Goodyear Ultra Grip 9	1	359

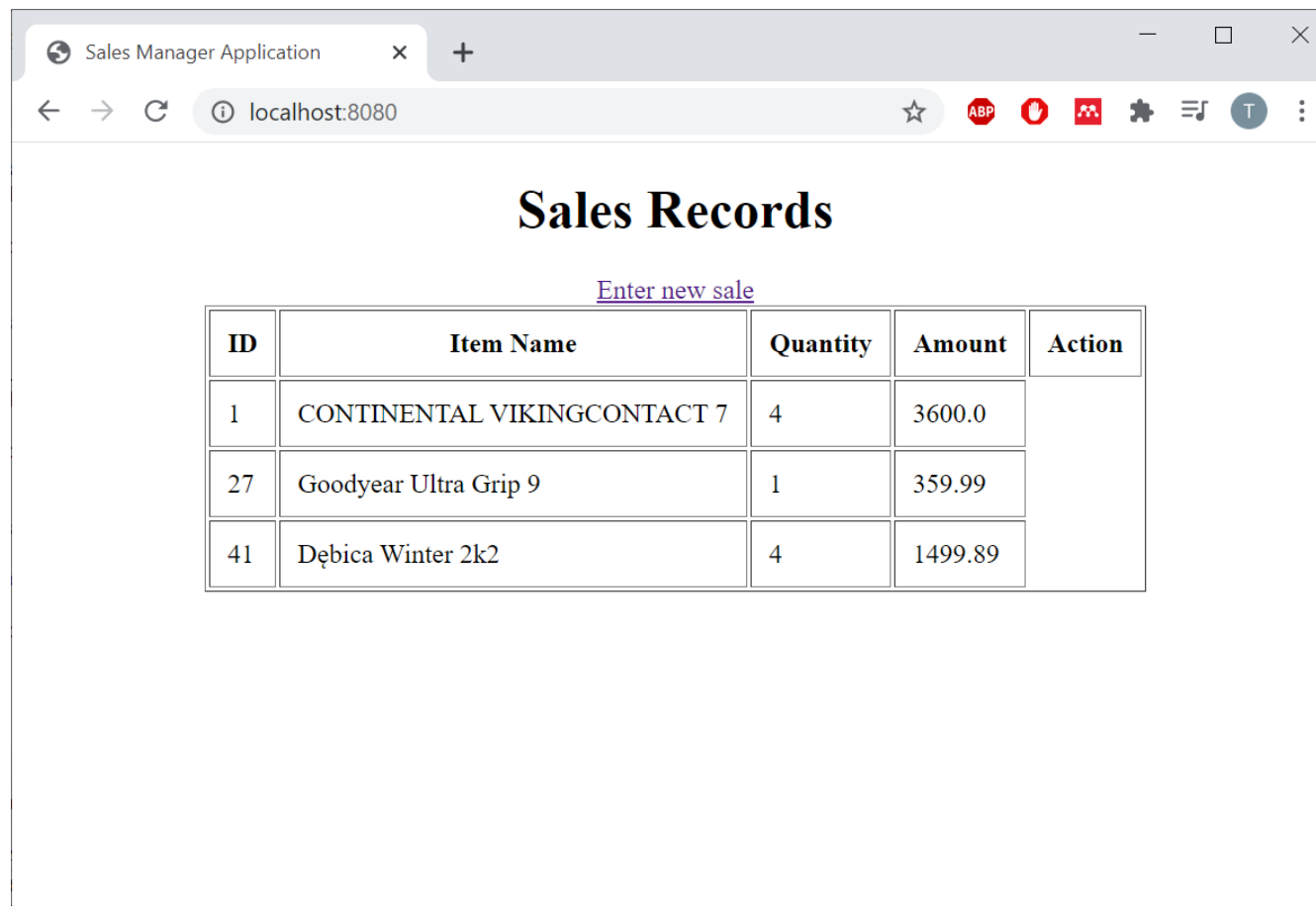
New Sale

localhost:8080/new

Enter New Sale

Item Name:	Dębica Winter 2k2
Quantity:	4
Amount:	1499.89
Save	

10. Plik index.html – dodawanie nowych rekordów (create) (test)



Sales Manager Application

localhost:8080

Sales Records

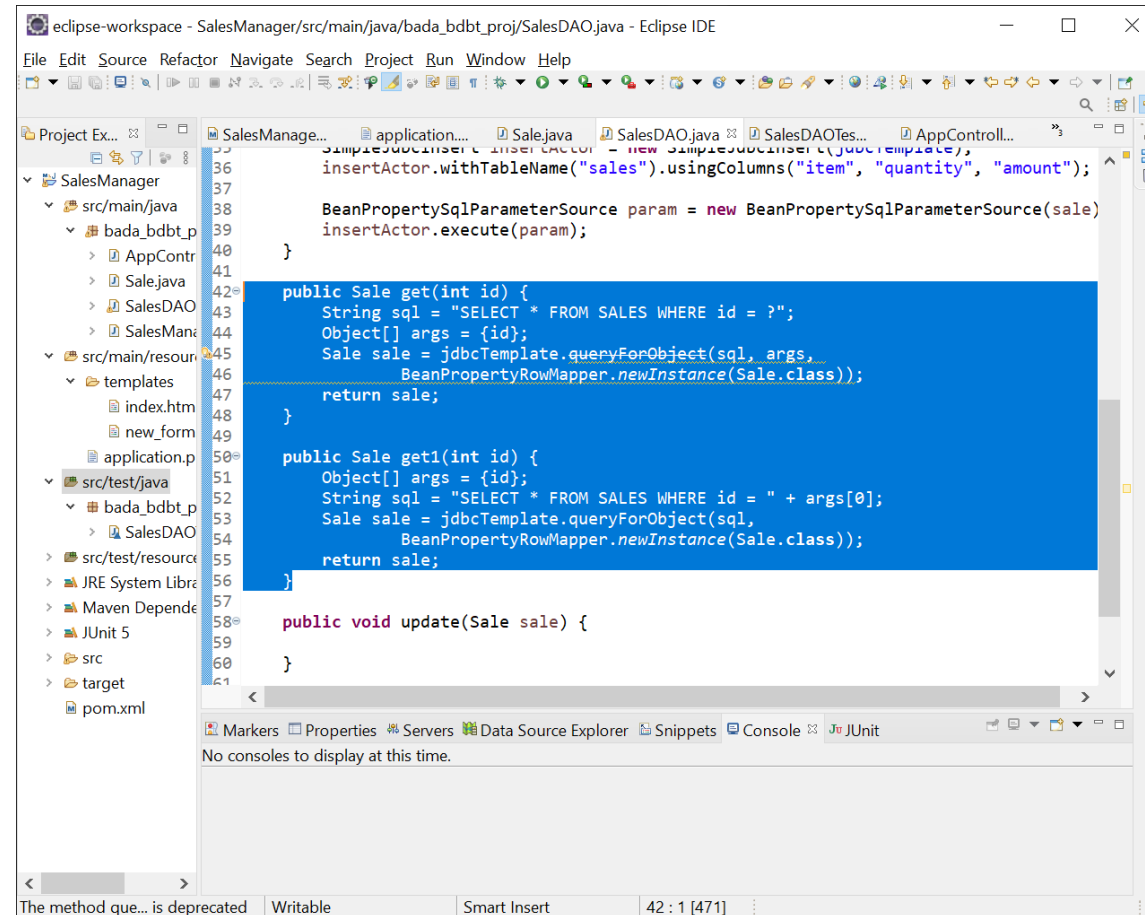
[Enter new sale](#)

ID	Item Name	Quantity	Amount	Action
1	CONTINENTAL VIKINGCONTACT 7	4	3600.0	
27	Goodyear Ultra Grip 9	1	359.99	
41	Dębica Winter 2k2	4	1499.89	

11

IMPLEMENTACJA WŁAŚCIWOŚCI EDYTUJ/AKTUALIZUJ

11. Implementacja właściwości edytuj/aktualizuj



```
36         insertActor.withTableName("sales").usingColumns("item", "quantity", "amount");
37
38         BeanPropertySqlParameterSource param = new BeanPropertySqlParameterSource(sale);
39         insertActor.execute(param);
40     }
41
42     public Sale get(int id) {
43         String sql = "SELECT * FROM SALES WHERE id = ?";
44         Object[] args = {id};
45         Sale sale = jdbcTemplate.queryForObject(sql, args,
46             BeanPropertyRowMapper.newInstance(Sale.class));
47         return sale;
48     }
49
50     public Sale get1(int id) {
51         Object[] args = {id};
52         String sql = "SELECT * FROM SALES WHERE id = " + args[0];
53         Sale sale = jdbcTemplate.queryForObject(sql,
54             BeanPropertyRowMapper.newInstance(Sale.class));
55         return sale;
56     }
57
58     public void update(Sale sale) {
59
60     }
61 }
```

11. Implementacja właściwości edytuj/aktualizuj (JUnit test)

The image displays two side-by-side screenshots of the Eclipse IDE, showing the implementation and testing of the 'update' method in the SalesDAO class.

Left Screenshot: The editor shows the SalesDAO.java file. The testList() method is implemented, and the testUpdate() method is highlighted in blue. The testUpdate() method is currently failing with the message "fail('Not yet implemented')".

```
29 void testList() {
30     /* Import java.util */
31     List<Sale> listSale = dao.list();
32     // assertFalse(listSale.isEmpty());
33     assertTrue(listSale.isEmpty());
34 }
35
36 @Test
37 void testSave() {
38     Sale sale = new Sale(0, "Goodyear Ultra Grip 9", 1, 359.99f);
39     dao.save(sale);
40 }
41
42 @Test
43 void testGet() {
44     fail("Not yet implemented");
45 }
46
47 @Test
48 void testUpdate() {
49     fail("Not yet implemented");
50 }
51
52 @Test
53 void testDelete() {
54     fail("Not yet implemented");
55 }
```

Right Screenshot: The editor shows the SalesDAO.java file. The testList() method is implemented, and the testUpdate() method is highlighted in blue. The testUpdate() method is now implemented with the following code:

```
29 void testList() {
30     /* Import java.util */
31     List<Sale> listSale = dao.list();
32     // assertFalse(listSale.isEmpty());
33     assertTrue(listSale.isEmpty());
34 }
35
36 @Test
37 void testSave() {
38     Sale sale = new Sale(0, "Goodyear Ultra Grip 9", 1, 359.99f);
39     dao.save(sale);
40 }
41
42 @Test
43 void testGet() {
44     int id = 64;
45     Sale sale = dao.get(id);
46     assertNotNull(sale);
47 }
48
49 @Test
50 void testUpdate() {
51     fail("Not yet implemented");
52 }
53
54 @Test
55 void testDelete() {
56     fail("Not yet implemented");
57 }
```

Both screenshots show the same project structure in the Project Explorer, with the SalesManager project containing the src/main/java, src/main/resources, and src/test/java directories. The console at the bottom of both windows shows the output of the JUnit tests, indicating that the tests are running successfully.

11. Implementacja właściwości edytuj/aktualizuj (JUnit test)

The screenshot displays the Eclipse IDE interface. The Project Explorer on the left shows the project structure: SalesManager, src/main/java, src/main/resources, and src/test/java. The main editor shows the SalesDAOTest.java file with the following code:

```
35  
36 @Test  
37 void testSave() {  
38     Sale sale = new Sale(64);  
39     dao.save(sale);  
40 }  
41  
42 @Test  
43 void testGet() {  
44     int id = 64;  
45     Sale sale = dao.get(id);  
46  
47     assertNotNull(sale);  
48 }  
49  
50 @Test  
51 void testUpdate() {  
52     fail("Not yet implemented");  
53 }  
54  
55 @Test  
56 void testDelete() {  
57     fail("Not yet implemented");  
58 }  
59  
60 }  
61
```

The JUnit test results window is open, showing the test suite SalesDAOTest [Runner: JUnit 5] (1,123 s). The results indicate that the tests passed successfully, with 1/1 runs, 0 errors, and 0 failures. A context menu is visible over the JUnit test results, listing various actions such as Open Type Hierarchy, Open Call Hierarchy, Show In, Cut, Copy, Copy Qualified Name, Paste, Delete, Source, Refactor, References, Declarations, Toggle Method Breakpoint, Coverage As, Run As, Debug As, Profile As, Restore from Local History..., Compare With, Replace With, and Properties. The Run As option is highlighted.

11. Implementacja właściwości edytuj/aktualizuj

The image displays two side-by-side screenshots of the Eclipse IDE, showing the implementation of an 'edit' button in an HTML table. The left screenshot shows the initial state of the `index.html` file, where the table body contains four columns: ID, Item Name, Quantity, and Amount. The right screenshot shows the updated state, where an 'Edit' button has been added to the end of each row. The button's href is constructed using the item's ID.

Left Screenshot (Initial State):

```

17 <!-- Table -->
18 <table border = "1" cellpadding = "10">
19 <!-- Table header -->
20 <thead>
21 <tr>
22 <th>ID</th>
23 <th>Item Name</th>
24 <th>Quantity</th>
25 <th>Amount</th>
26 <th>Action</th>
27 </tr>
28 </thead>
29 <!-- Table body -->
30 <tbody>
31 <tr th:each = "sale : ${listSale}">
32 <td th:text = "${sale.id}">ID</td>
33 <td th:text = "${sale.item}">Item Name</td>
34 <td th:text = "${sale.quantity}">Quantity</td>
35 <td th:text = "${sale.amount}">Amount</td>
36 </tr>
37 </tbody>
38 </table>
39 </div>

```

Right Screenshot (Updated State):

```

20 <table border = "1" cellpadding = "10">
21 <!-- Table header -->
22 <thead>
23 <tr>
24 <th>ID</th>
25 <th>Item Name</th>
26 <th>Quantity</th>
27 <th>Amount</th>
28 <th>Action</th>
29 </tr>
30 </thead>
31 <!-- Table body -->
32 <tbody>
33 <tr th:each = "sale : ${listSale}">
34 <td th:text = "${sale.id}">ID</td>
35 <td th:text = "${sale.item}">Item Name</td>
36 <td th:text = "${sale.quantity}">Quantity</td>
37 <td th:text = "${sale.amount}">Amount</td>
38 <td>
39 <a th:href="@{'/edit/' + ${sale.id}}">Edit</a>
40 </td>
41 </tr>
42 </tbody>
43 </table>
44 </div>

```

Both screenshots show the Eclipse IDE interface with the 'SalesManager' project. The left screenshot shows the 'index.html' file in the 'templates' directory. The right screenshot shows the same file with the 'Edit' button added. The status bar at the bottom of both windows indicates the current line and column position.

11. Implementacja właściwości edytuj/aktualizuj

The screenshot displays the Eclipse IDE workspace for a project named 'SalesManager'. The main editor shows the 'index.html' file in the 'templates' directory. The code defines an HTML table with a header and a body. A callout box highlights the addition of a 'break line' (br) in the table header, specifically in the 'Item Name' column header.

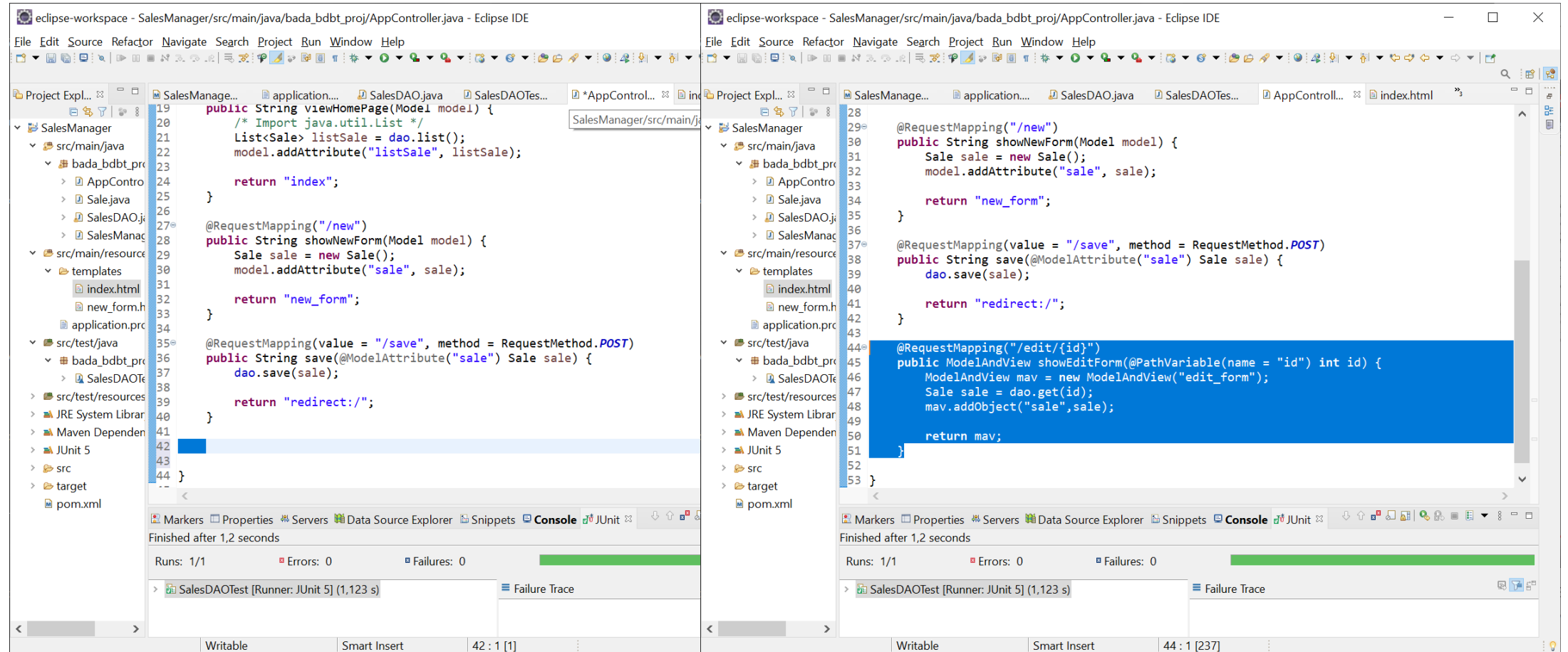
```

10 <div align="center">
11   <!-- Page header -->
12   <h1>Sales Records</h1>
13
14   <!-- Entry to new page -->
15   <a href="/new">Enter new sale</a>
16
17   <!-- Break line -->
18   <br/><br/>
19
20   <!-- Table -->
21   <table border = "1" cellpadding = "10">
22     <!-- Table header -->
23     <thead>
24       <tr>
25         <th>ID</th>
26         <th>Item Name</th>
27         <th>Quantity</th>
28         <th>Amount</th>
29         <th>Action</th>
30       </tr>
31     </thead>
32     <!-- Table body -->
33     <tbody>
34       <tr>
35         <th:each = "sale : ${listSale}">
36           <td th:text = "${sale.id}">ID</td>
37           <td th:text = "${sale.item}">Item Name</td>
38           <td th:text = "${sale.quantity}">Quantity</td>
39           <td th:text = "${sale.amount}">Amount</td>
40           <td>
41             <a th:href="@{'/edit/' + ${sale.id}}">Edit</a>
42           </td>
43         </tr>
44       </tbody>
45     </table>
46   </div>

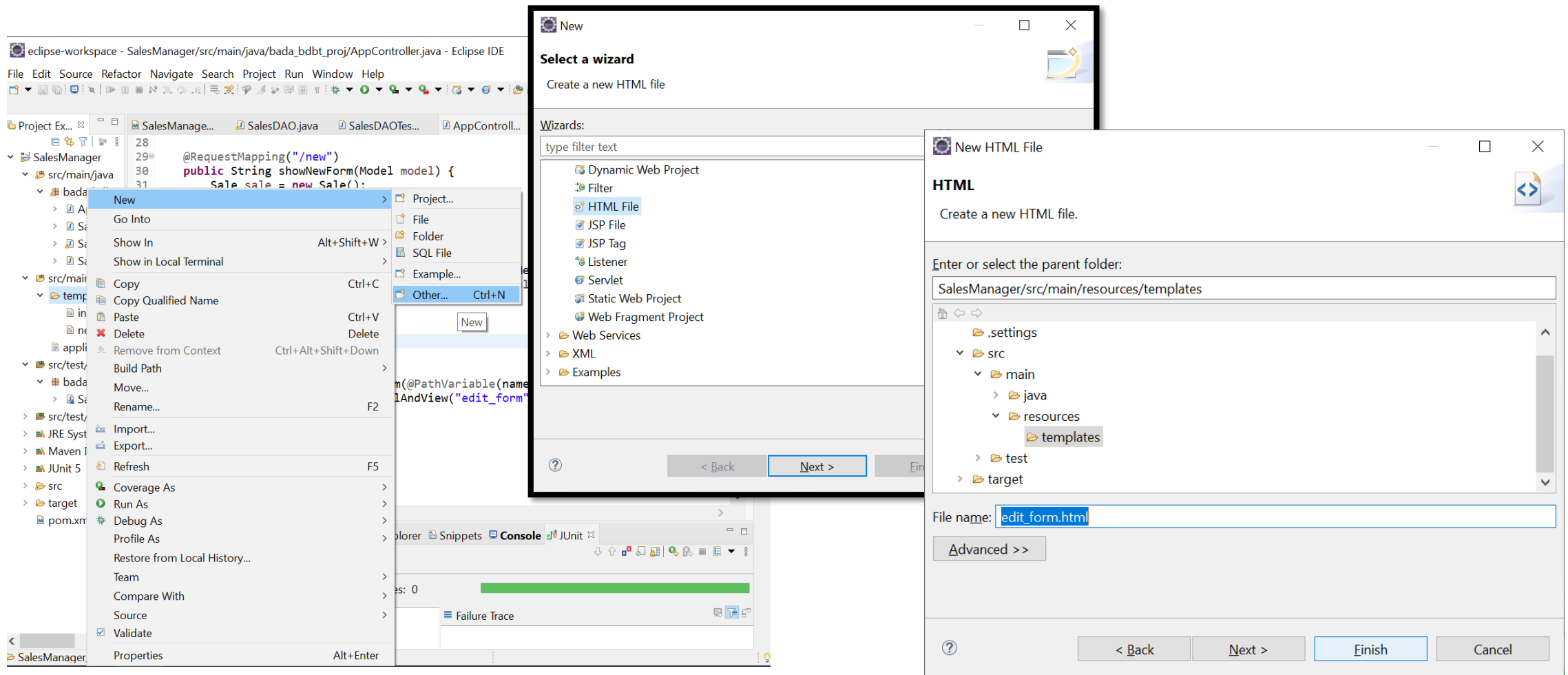
```

The console output shows the successful execution of the JUnit test 'SalesDAOTest' [Runner: JUnit 5] (1,123 s), indicating that the implementation is working correctly.

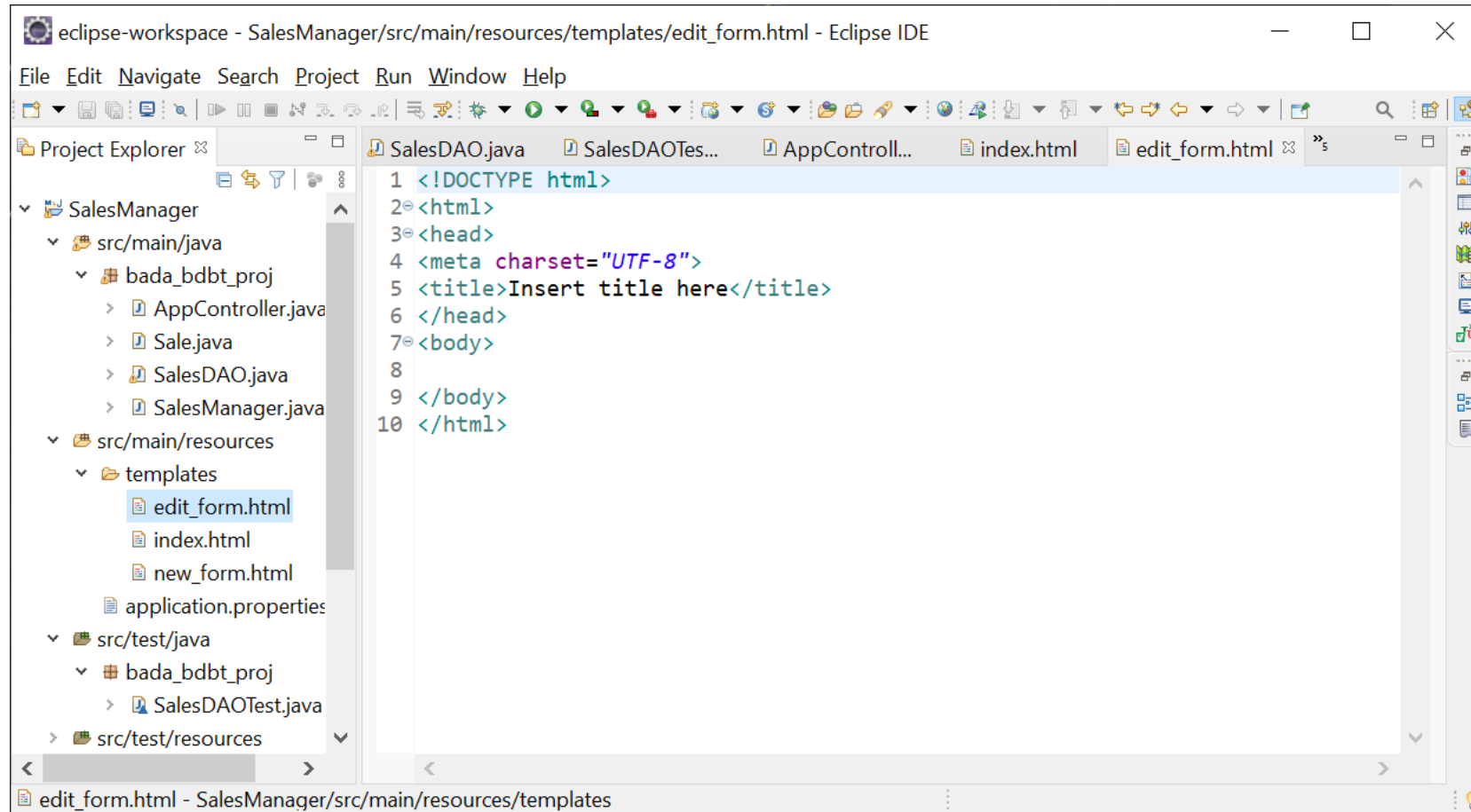
11. Implementacja właściwości edytuj/aktualizuj



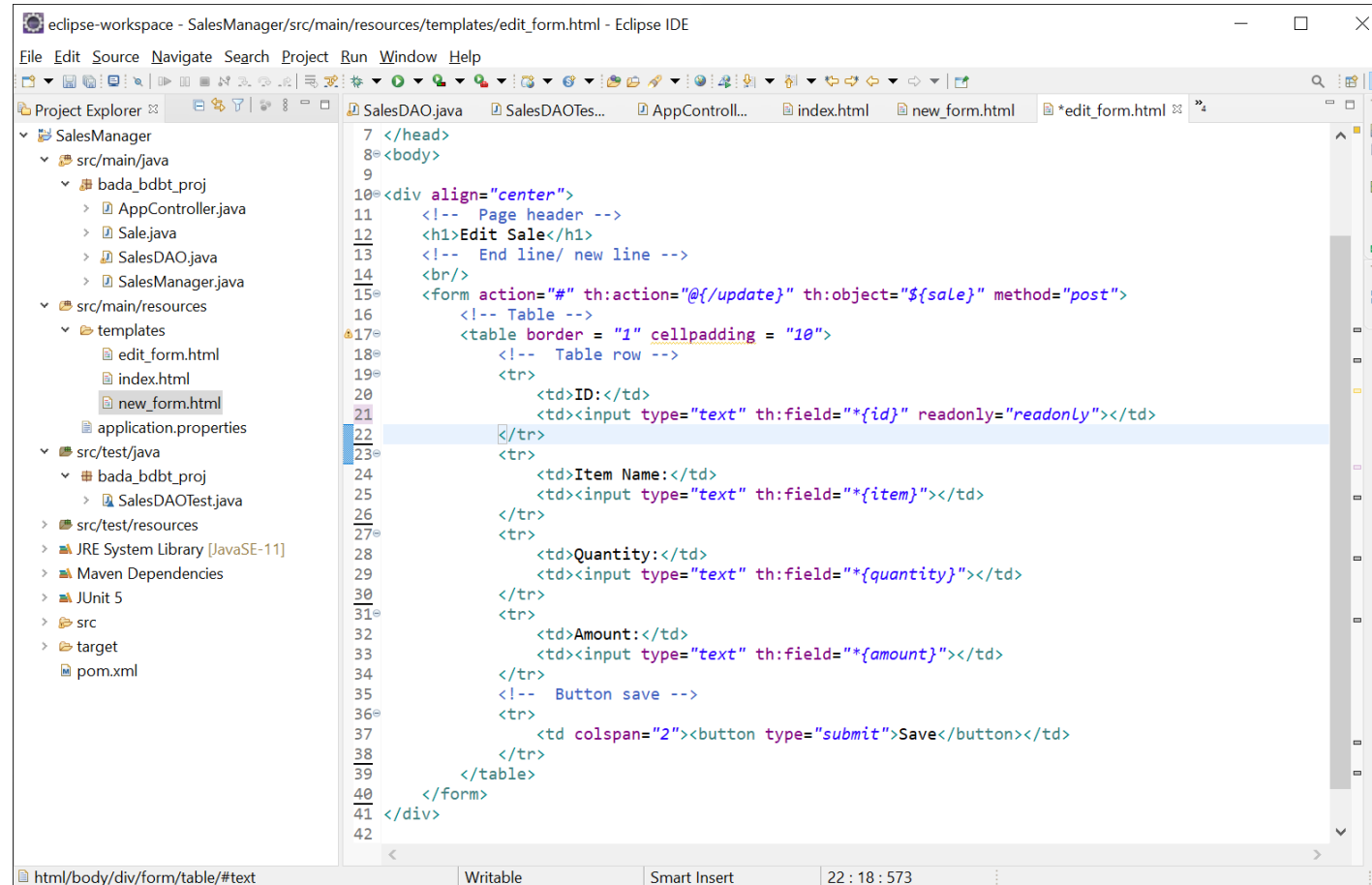
11. Implementacja właściwości edytuj/aktualizuj



11. Implementacja właściwości edytuj/aktualizuj



11. Implementacja właściwości edytuj/aktualizuj



```

7  </head>
8  <body>
9
10 <div align="center">
11   <!-- Page header -->
12   <h1>Edit Sale</h1>
13   <!-- End line/ new line -->
14   <br/>
15   <form action="#" th:action="@{/update}" th:object="${sale}" method="post">
16     <!-- Table -->
17     <table border = "1" cellpadding = "10">
18       <!-- Table row -->
19       <tr>
20         <td>ID:</td>
21         <td><input type="text" th:field="*{id}" readonly="readonly"></td>
22       </tr>
23       <tr>
24         <td>Item Name:</td>
25         <td><input type="text" th:field="*{item}"></td>
26       </tr>
27       <tr>
28         <td>Quantity:</td>
29         <td><input type="text" th:field="*{quantity}"></td>
30       </tr>
31       <tr>
32         <td>Amount:</td>
33         <td><input type="text" th:field="*{amount}"></td>
34       </tr>
35       <!-- Button save -->
36       <tr>
37         <td colspan="2"><button type="submit">Save</button></td>
38       </tr>
39     </table>
40   </form>
41 </div>
42

```

11. Implementacja właściwości edytuj/aktualizuj

The image displays two side-by-side screenshots of the Eclipse IDE, showing the implementation of SQL update and delete functions in the `SalesDAO.java` file.

Left Screenshot: Shows the `SalesDAO.java` file with the following code:

```

41
42 public Sale get(int id) {
43     String sql = "SELECT * FROM SALES WHERE id = ?";
44     Object[] args = {id};
45     Sale sale = jdbcTemplate.queryForObject(sql, args,
46         BeanPropertyRowMapper.newInstance(Sale.class));
47     return sale;
48 }
49
50 public Sale get1(int id) {
51     Object[] args = {id};
52     String sql = "SELECT * FROM SALES WHERE id = " + id;
53     Sale sale = jdbcTemplate.queryForObject(sql, args,
54         BeanPropertyRowMapper.newInstance(Sale.class));
55     return sale;
56 }
57
58 public void update(Sale sale) {
59 }
60
61
62 public void delete(int id) {
63 }
64
65 }
66

```

Right Screenshot: Shows the `SalesDAO.java` file with the following code:

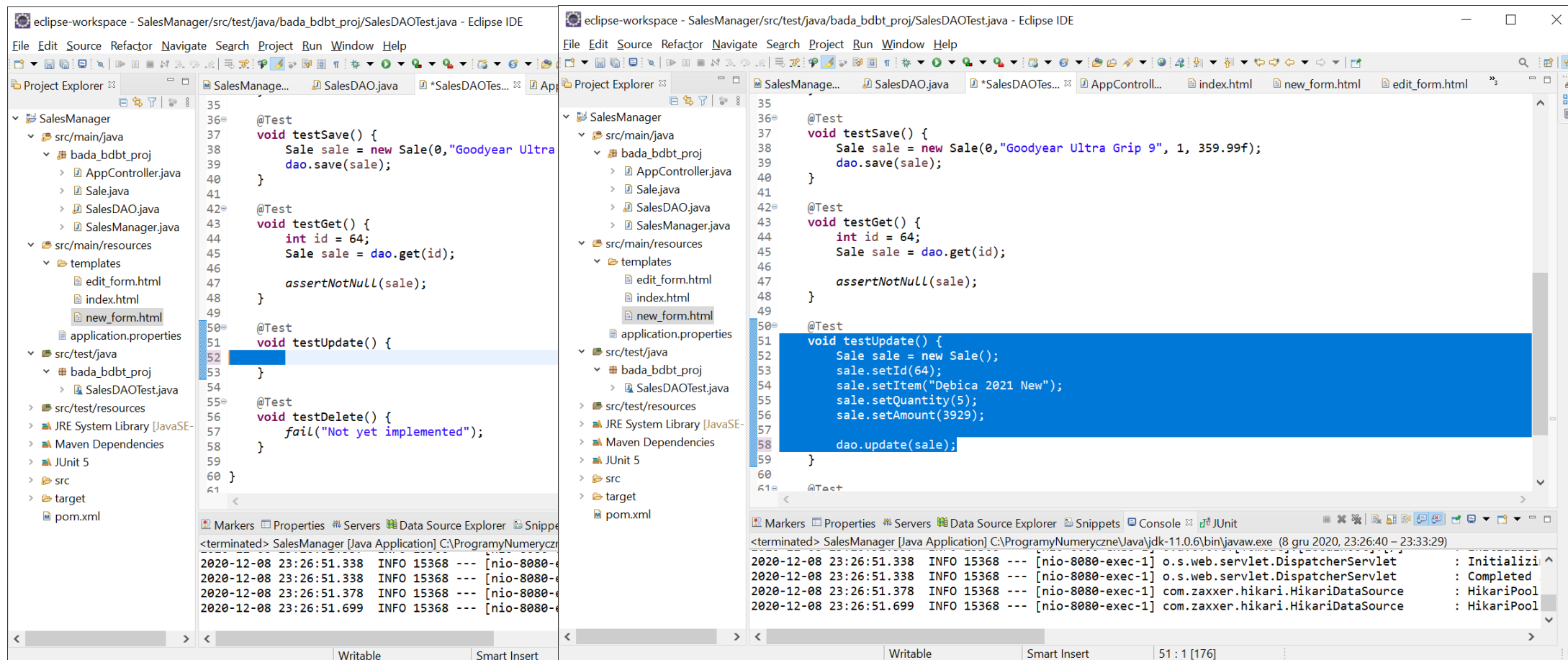
```

45
46 Object[] args = {id};
47 Sale sale = jdbcTemplate.queryForObject(sql, args,
48     BeanPropertyRowMapper.newInstance(Sale.class));
49 return sale;
50 }
51
52 public Sale get1(int id) {
53     Object[] args = {id};
54     String sql = "SELECT * FROM SALES WHERE id = " + args[0];
55     Sale sale = jdbcTemplate.queryForObject(sql, args,
56         BeanPropertyRowMapper.newInstance(Sale.class));
57     return sale;
58 }
59
60 public void update(Sale sale) {
61     String sql = "UPDATE SALES SET item=:item, quantity=:quantity, amount=:amount WHERE id=:id";
62     BeanPropertySqlParameterSource param = new BeanPropertySqlParameterSource(sale);
63     NamedParameterJdbcTemplate template = new NamedParameterJdbcTemplate(jdbcTemplate);
64     template.update(sql, param);
65 }
66
67 public void delete(int id) {
68 }
69
70 }

```

Both screenshots show the Project Explorer on the left, displaying the project structure. The right screenshot also shows the Console window at the bottom, displaying log messages from the application.

11. Implementacja właściwości edytuj/aktualizuj (JUnit test)



11. Implementacja właściwości edytuj/aktualizuj

The image displays two side-by-side screenshots of the Eclipse IDE, showing the implementation of the 'update' method in the `AppController.java` file. The left screenshot shows the initial state of the code, and the right screenshot shows the code after the 'update' method has been added.

Left Screenshot: The `AppController.java` file is open, showing the `showNewForm` and `save` methods. The `showEditForm` method is partially visible at the bottom.

```

30 public String showNewForm(Model model) {
31     Sale sale = new Sale();
32     model.addAttribute("sale", sale);
33
34     return "new_form";
35 }
36
37 @RequestMapping(value = "/save", method = RequestMethod.POST)
38 public String save(@ModelAttribute("sale") Sale sale) {
39     dao.save(sale);
40
41     return "redirect:/";
42 }
43
44 @RequestMapping("/edit/{id}")
45 public ModelAndView showEditForm(@PathVariable(name = "id") int id) {
46     ModelAndView mav = new ModelAndView("edit_form");
47     Sale sale = dao.get(id);
48     mav.addObject("sale", sale);
49
50     return mav;
51 }
52
53 }
54
55 }
56

```

Right Screenshot: The `AppController.java` file is open, showing the `showEditForm` and `update` methods. The `update` method is highlighted in blue.

```

35 }
36
37 @RequestMapping(value = "/save", method = RequestMethod.POST)
38 public String save(@ModelAttribute("sale") Sale sale) {
39     dao.save(sale);
40
41     return "redirect:/";
42 }
43
44 @RequestMapping("/edit/{id}")
45 public ModelAndView showEditForm(@PathVariable(name = "id") int id) {
46     ModelAndView mav = new ModelAndView("edit_form");
47     Sale sale = dao.get(id);
48     mav.addObject("sale", sale);
49
50     return mav;
51 }
52
53 @RequestMapping(value = "/update", method = RequestMethod.POST)
54 public String update(@ModelAttribute("sale") Sale sale) {
55     dao.update(sale);
56
57     return "redirect:/";
58 }
59
60 }
61

```

Both screenshots show the Project Explorer on the left, displaying the project structure. The bottom status bar indicates the test run results: "Runs: 1/1", "Errors: 0", "Failures: 0".

11. Implementacja właściwości edytuj/aktualizuj (testowanie)

Sales Manager Application

localhost:8080

Sales Records

[Enter new sale](#)

ID	Item Name	Quantity	Amount	Action
63	Dębica Winter 2k2	5	5000.0	Edit
64	Dębica 2021 New	5	3929.0	Edit

Edit Sale

localhost:8080/edit/64

ID:

64

Item Name:

Dębica 2021 New

Quantity:

5

Amount:

3929.0

Save

11. Implementacja właściwości edytuj/aktualizuj (testowanie)

Edit Sale

localhost:8080/edit/64

Edit Sale

ID:	64
Item Name:	Dębica 2020 New
Quantity:	6
Amount:	3429.0
Save	

Sales Manager Application

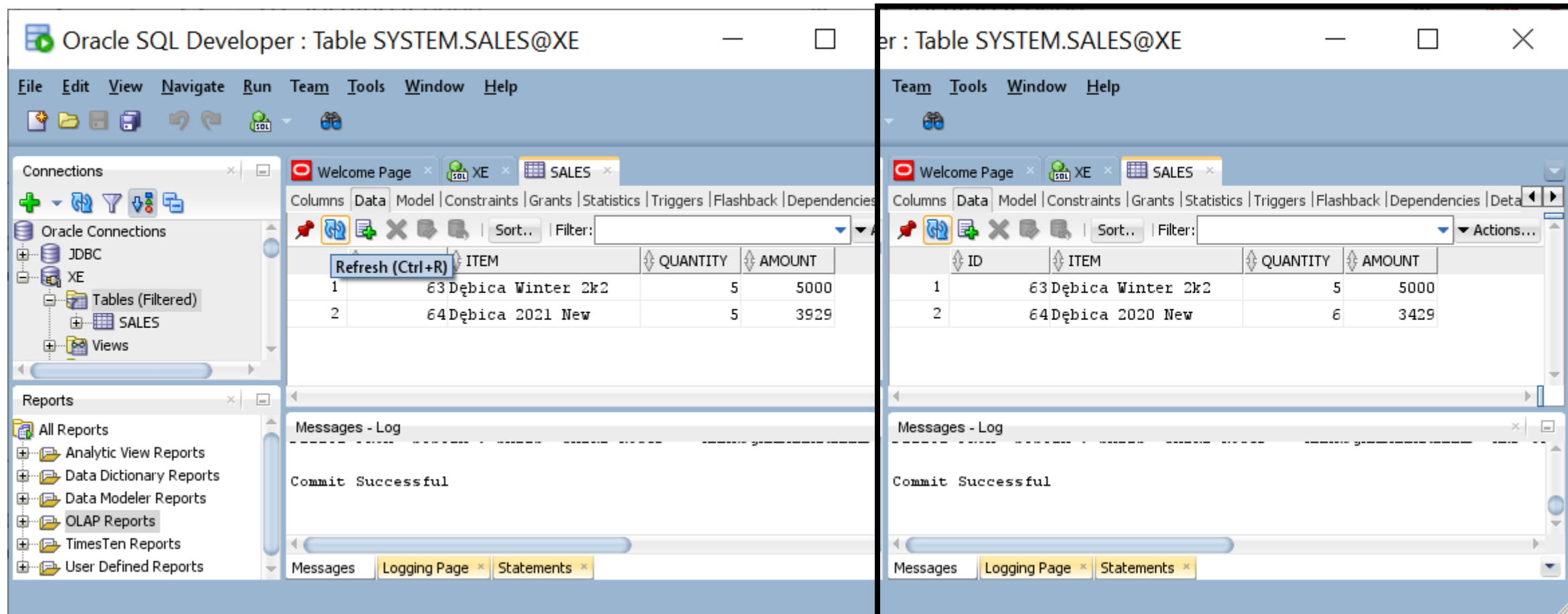
localhost:8080

Sales Records

[Enter new sale](#)

ID	Item Name	Quantity	Amount	Action
63	Dębica Winter 2k2	5	5000.0	Edit
64	Dębica 2020 New	6	3429.0	Edit

11. Implementacja właściwości edytuj/aktualizuj (testowanie)



The image displays two side-by-side screenshots of the Oracle SQL Developer interface, illustrating a table update operation. Both screenshots show the 'Table SYSTEM.SALES@XE' window with the 'Data' tab selected. The table has columns: ID, ITEM, QUANTITY, and AMOUNT.

Left Screenshot: The 'Refresh (Ctrl+R)' button is highlighted. The table data is as follows:

ID	ITEM	QUANTITY	AMOUNT
1	63Dębica Winter 2k2	5	5000
2	64Dębica 2021 New	5	3929

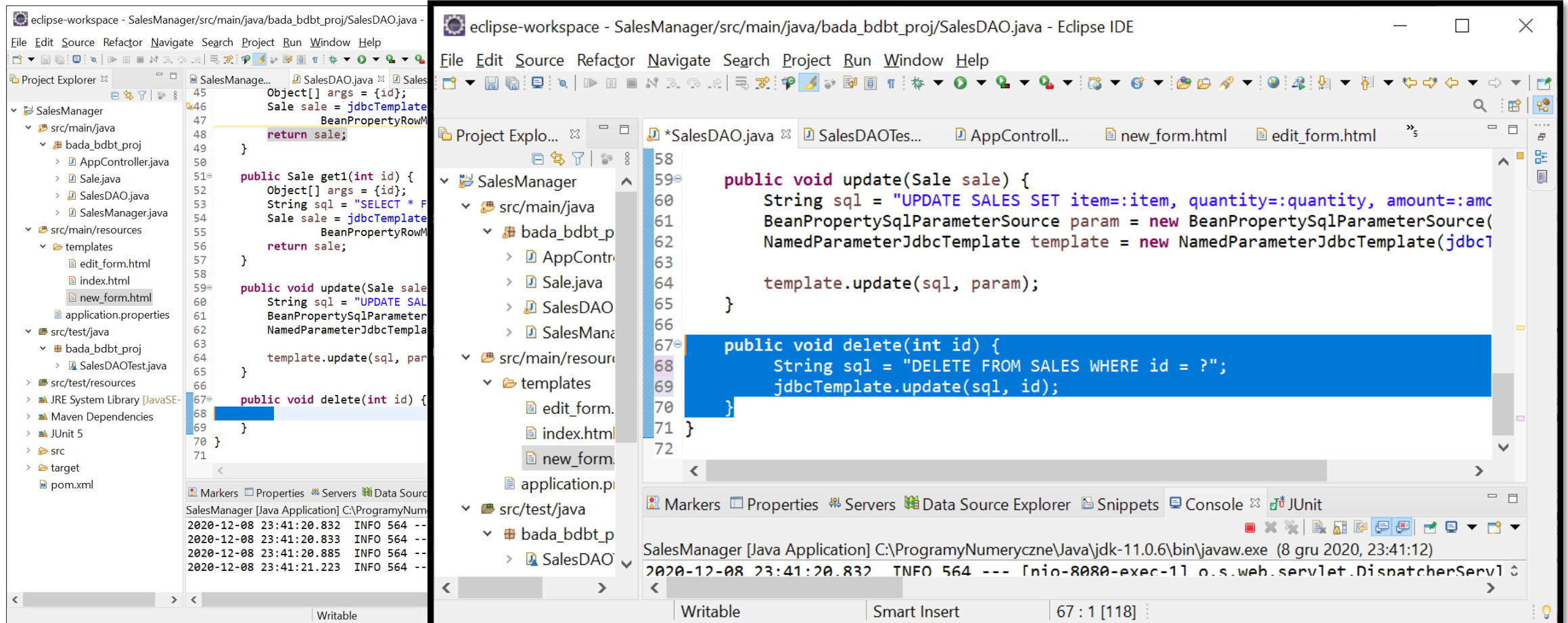
Right Screenshot: The table data has been updated. The 'Commit Successful' message is visible in the 'Messages - Log' pane. The table data is as follows:

ID	ITEM	QUANTITY	AMOUNT
1	63Dębica Winter 2k2	5	5000
2	64Dębica 2020 New	6	3429

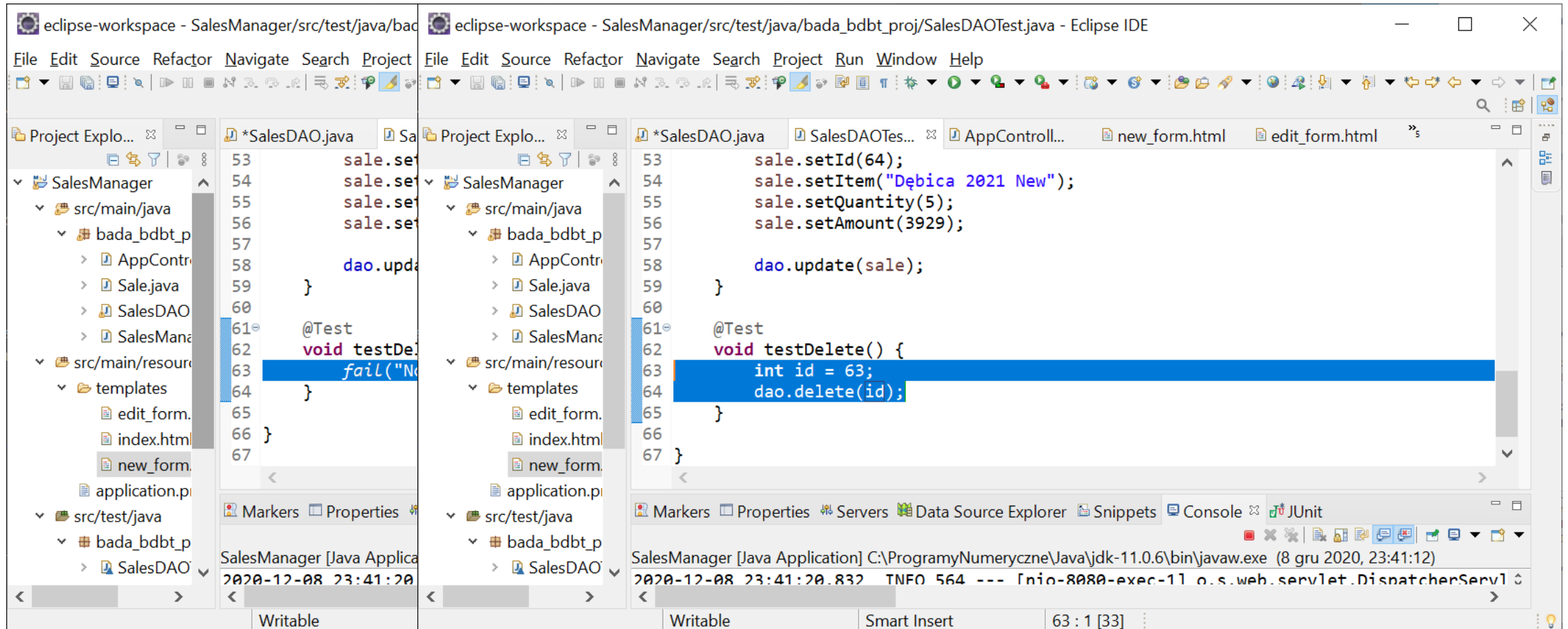
12

IMPLEMENTACJA WŁAŚCIWOŚCI USUŃ

12. Implementacja właściwości usuń



12. Implementacja właściwości usuń (JUnit test)



The screenshot displays the Eclipse IDE interface with two side-by-side views of Java code. The left view shows the `SalesDAO.java` file, and the right view shows the `SalesDAOTest.java` file. Both files contain a JUnit test method `testDelete()`. The right view also shows the implementation of the `dao.delete(id)` method call within the test.

```
53 sale.set
54 sale.set
55 sale.set
56 sale.set
57
58 dao.upda
59 }
60
61 @Test
62 void testDe
63 fail("No
64 }
65
66 }
67
```

```
53 sale.setId(64);
54 sale.setItem("Dębica 2021 New");
55 sale.setQuantity(5);
56 sale.setAmount(3929);
57
58 dao.update(sale);
59 }
60
61 @Test
62 void testDelete() {
63 int id = 63;
64 dao.delete(id);
65 }
66
67 }
```

Markers Properties Servers Data Source Explorer Snippets Console JUnit

SalesManager [Java Application] C:\ProgramyNumeryczne\Java\jdk-11.0.6\bin\javaw.exe (8 gru 2020, 23:41:12)

2020-12-08 23:41:20.832 INFO 564 --- [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet

12. Implementacja właściwości usuń

The screenshot shows the Eclipse IDE interface. The Project Explorer on the left displays the project structure: SalesManager, src/main/java (AppController.java, Sale.java, SalesDAO.java, SalesManager.java), src/main/resources (templates), and src/test/java (SalesDAOTest.java). The main editor displays the 'index.html' file, which contains an HTML table with columns: ID, Item Name, Quantity, Amount, and Action. The table is populated with data from a list of sales. A blue highlight is visible on the 'Edit' link in the table. The bottom console shows the output of the application, including debug messages from Spring Framework and JDBC driver.

[illegible]

12. Implementacja właściwości usuń

The screenshot displays the Eclipse IDE with two side-by-side views of the `AppController.java` file. The left view shows the code up to line 63, and the right view shows the code up to line 68, where the `delete` method has been implemented.

Left View (AppController.java):

```
36
37 @RequestMapping(value = "/save", method = RequestMethod.POST)
38 public String save(@ModelAttribute("sale") Sale sale) {
39     dao.save(sale);
40
41     return "redirect:/";
42 }
43
44 @RequestMapping("/edit/{id}")
45 public ModelAndView showEditForm(@PathVariable("id") int id) {
46     ModelAndView mav = new ModelAndView("edit_form");
47     Sale sale = dao.get(id);
48     mav.addObject("sale", sale);
49
50     return mav;
51 }
52
53 @RequestMapping(value = "/update", method = RequestMethod.POST)
54 public String update(@ModelAttribute("sale") Sale sale) {
55     dao.update(sale);
56
57     return "redirect:/";
58 }
59
60
61
62 }
63
```

Right View (AppController.java):

```
41     return "redirect:/";
42 }
43
44 @RequestMapping("/edit/{id}")
45 public ModelAndView showEditForm(@PathVariable(name = "id") int id) {
46     ModelAndView mav = new ModelAndView("edit_form");
47     Sale sale = dao.get(id);
48     mav.addObject("sale", sale);
49
50     return mav;
51 }
52
53 @RequestMapping(value = "/update", method = RequestMethod.POST)
54 public String update(@ModelAttribute("sale") Sale sale) {
55     dao.update(sale);
56
57     return "redirect:/";
58 }
59
60 @RequestMapping("/delete/{id}")
61 public String delete(@PathVariable(name = "id") int id) {
62     dao.delete(id);
63
64     return "redirect:/";
65 }
66
67 }
68
```

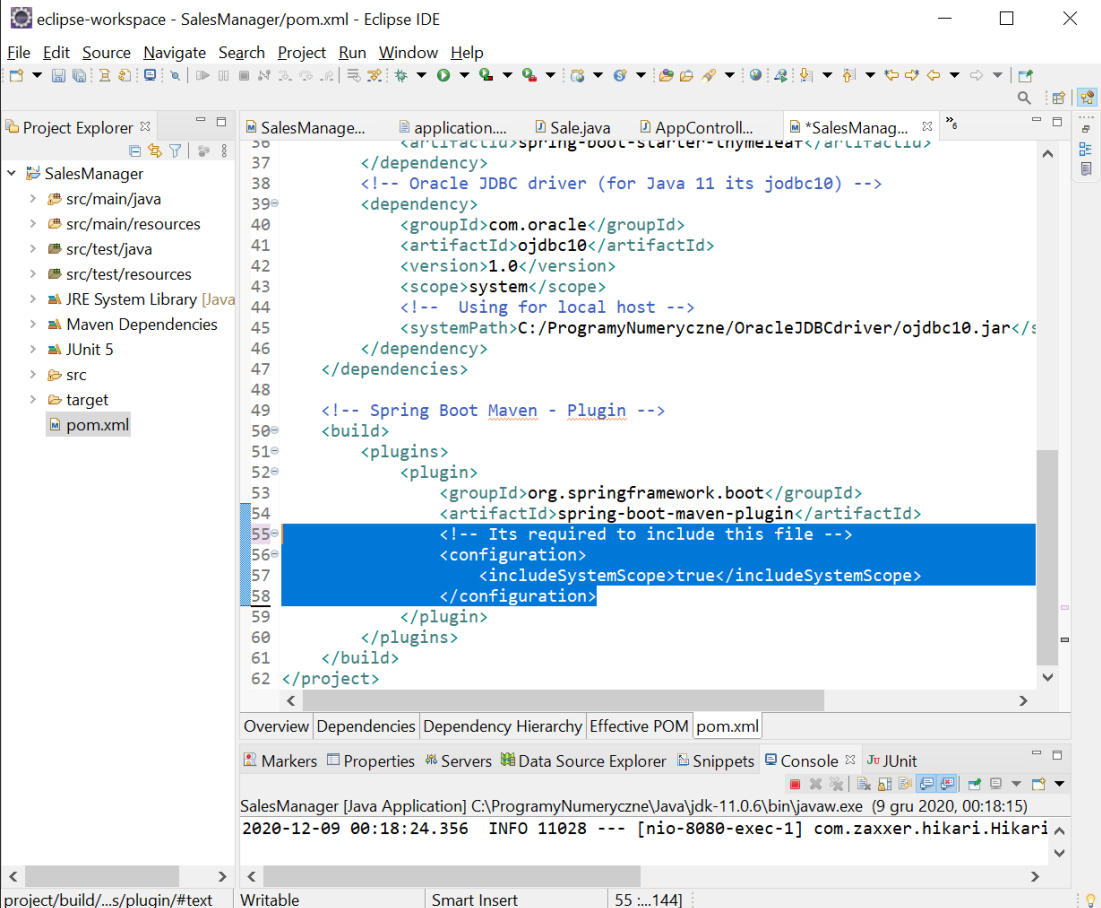
The console at the bottom shows the following log messages:

```
<terminated> SalesDAOTest.testDelete [JUnit] C:\Programy\Java\jdk-11.0.6\bin\javaw.exe (8 gru 2020, 23:49:41 - 23:49:45)
23:49:44.754 [main] DEBUG org.springframework.jdbc.datasource.DriverManagerDataSource - Loaded JDBC driver: ora
23:49:44.765 [main] DEBUG org.springframework.jdbc.core.JdbcTemplate - Executing prepared SQL update
23:49:44.766 [main] DEBUG org.springframework.jdbc.core.JdbcTemplate - Executing prepared SQL statement [DELETE
23:49:44.769 [main] DEBUG org.springframework.jdbc.datasource.DataSourceUtils - Fetching JDBC Connection from D
```

13

TEST DZIAŁANIA APLIKACJI

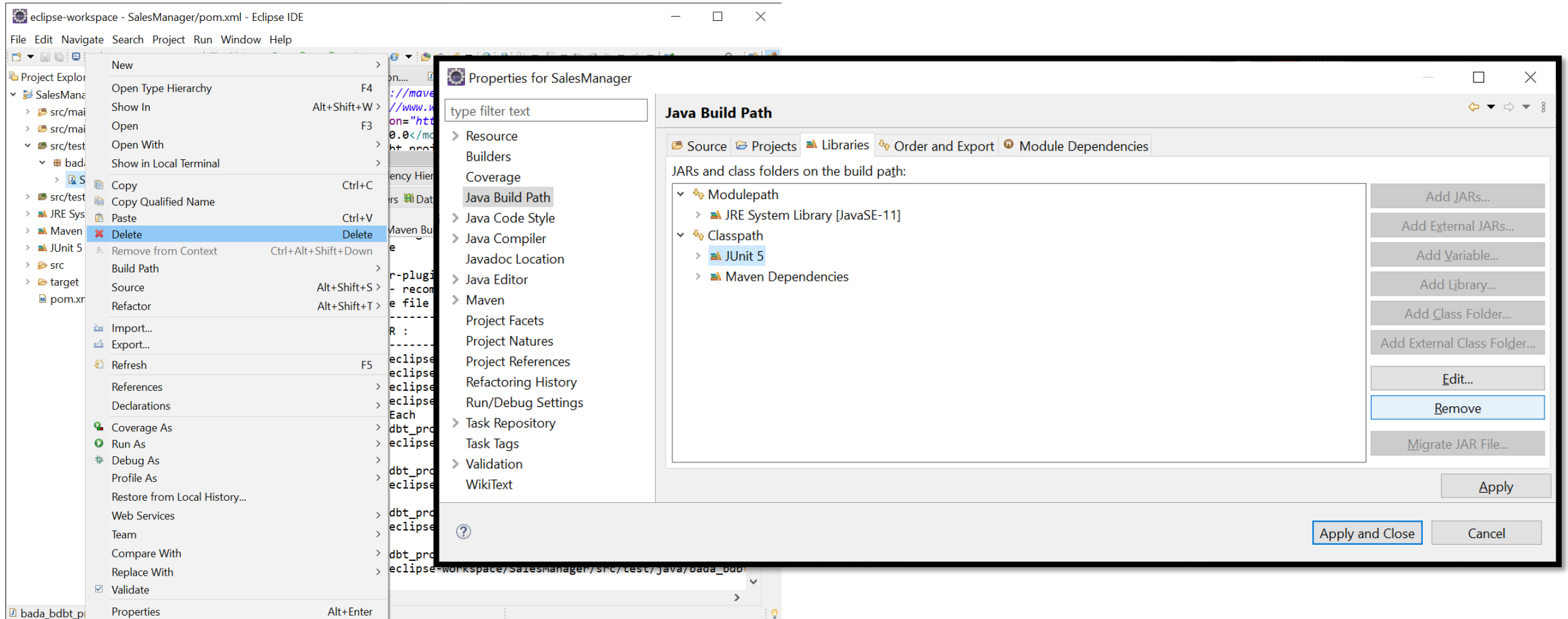
Pakowanie i uruchamianie aplikacji



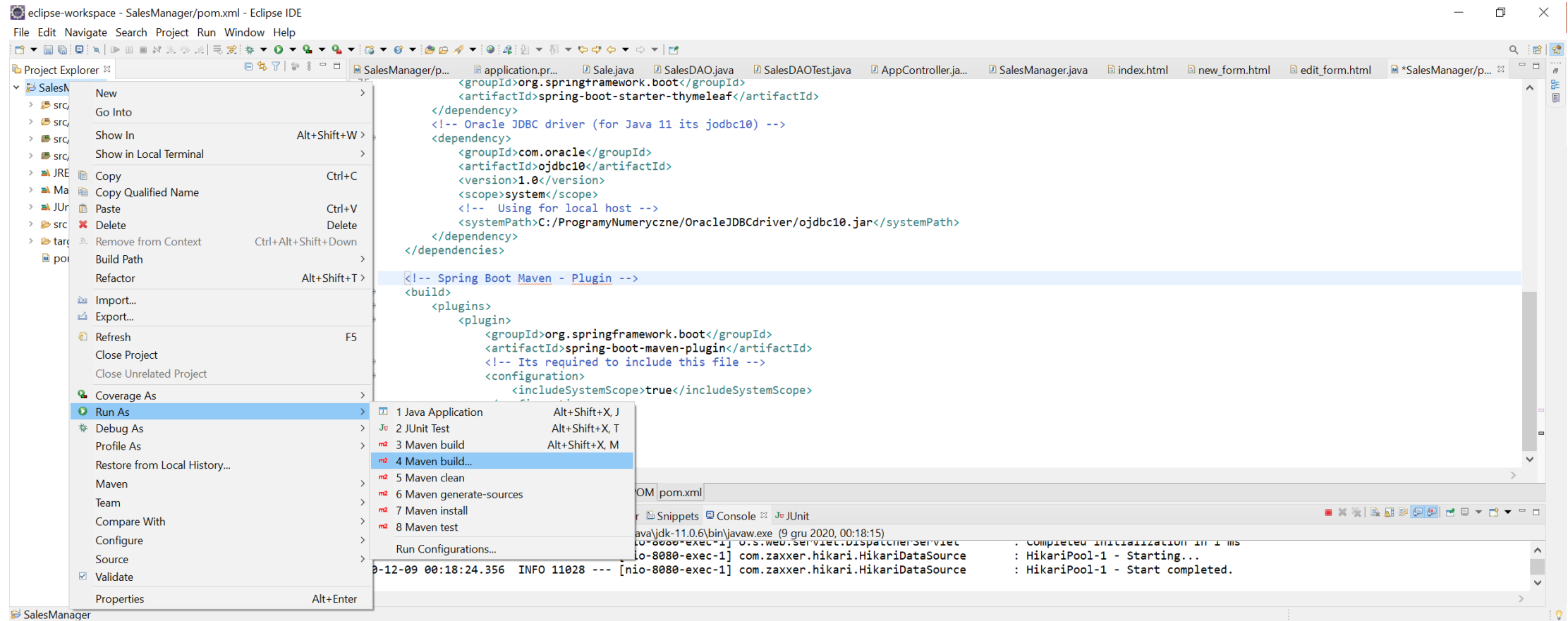
```
36 <!-- Spring Boot Starter -->
37 <!-- Oracle JDBC driver (for Java 11 its jodbc10) -->
38 <!-- Oracle JDBC driver (for Java 11 its jodbc10) -->
39 <dependency>
40 <groupId>com.oracle</groupId>
41 <artifactId>ojdbc10</artifactId>
42 <version>1.0</version>
43 <scope>system</scope>
44 <!-- Using for local host -->
45 <systemPath>C:/ProgramyNumeryczne/OracleJDBCdriver/ojdbc10.jar</systemPath>
46 </dependency>
47 </dependencies>
48
49 <!-- Spring Boot Maven - Plugin -->
50 <build>
51 <plugins>
52 <plugin>
53 <groupId>org.springframework.boot</groupId>
54 <artifactId>spring-boot-maven-plugin</artifactId>
55 <!-- Its required to include this file -->
56 <configuration>
57 <includeSystemScope>true</includeSystemScope>
58 </configuration>
59 </plugin>
60 </plugins>
61 </build>
62 </project>
```

Należy usunąć testy, aby aplikacja się uruchomiła

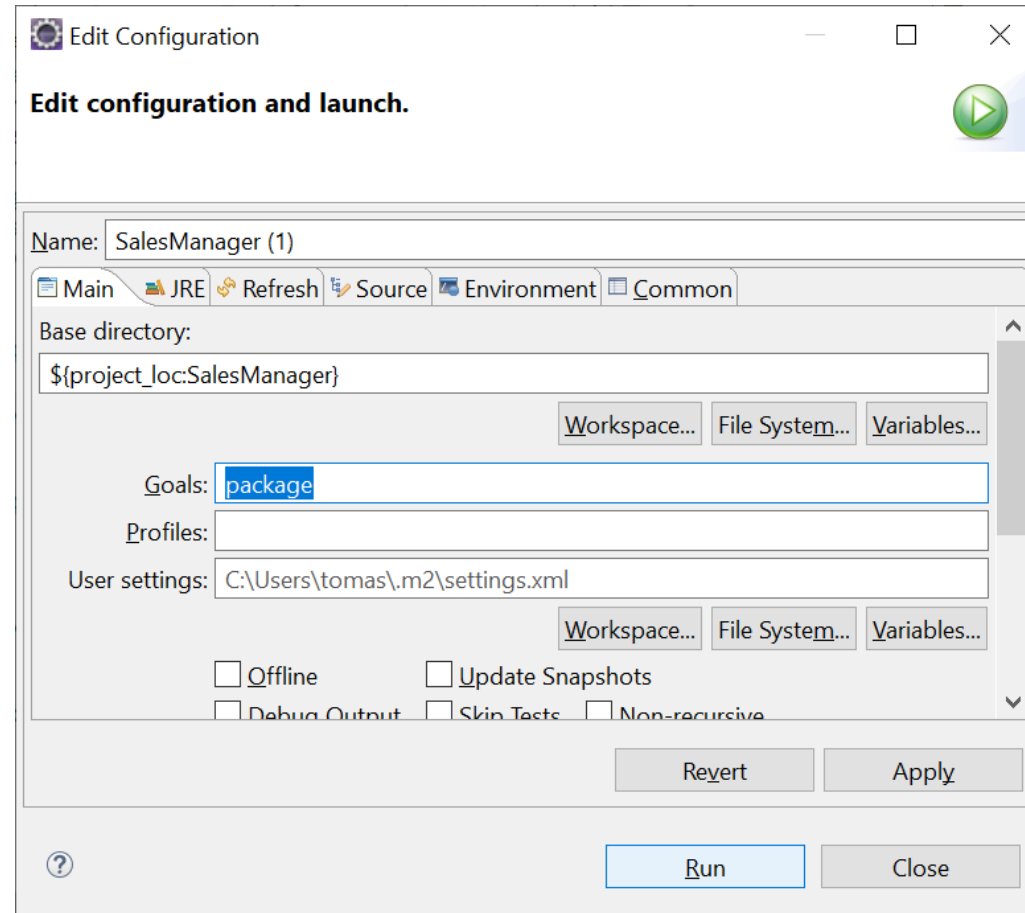
Pakowanie i uruchamianie aplikacji



Pakowanie i uruchamianie aplikacji

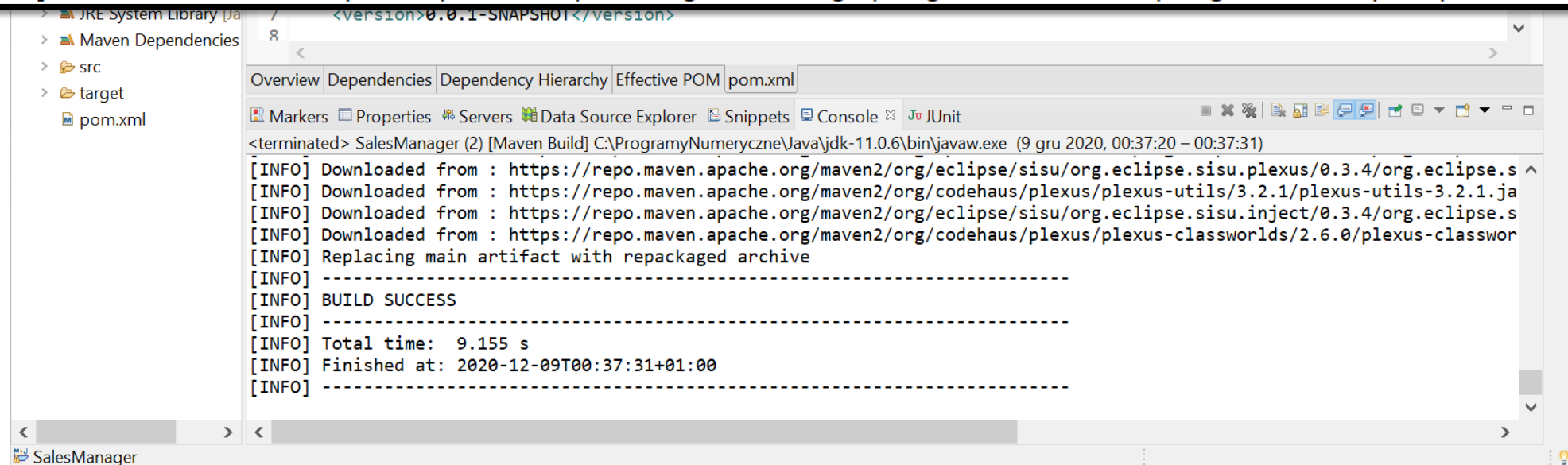


Pakowanie i uruchamianie aplikacji



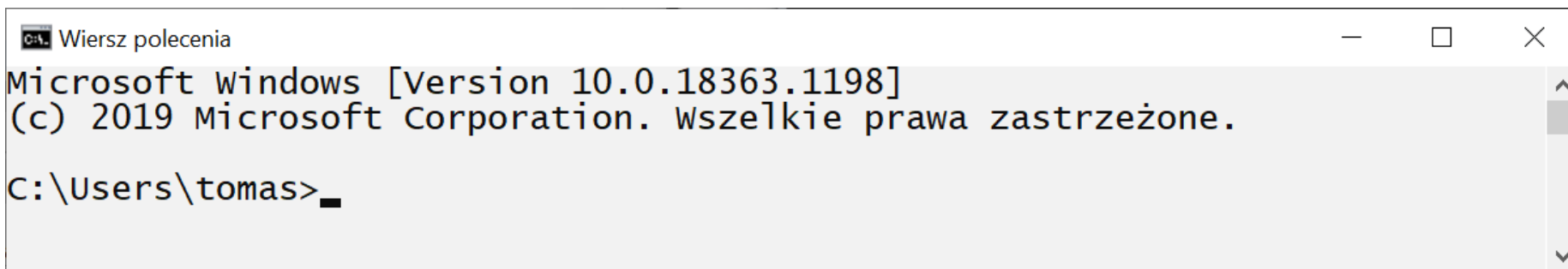
Pakowanie i uruchamianie aplikacji

```
[INFO] Downloaded from : https://repo.maven.apache.org/maven2/org/apache/maven/maven-toolchain/2.2.1/maven-toolchain-2.2.1
[INFO] Downloaded from : https://repo.maven.apache.org/maven2/com/thoughtworks/qdox/qdox/2.0-M8/qdox-2.0-M8.jar (316 kB at
[INFO]
[INFO] --- maven-jar-plugin:3.2.0:jar (default-jar) @ SalesManager ---
[INFO] Building jar: C:\Users\tomas\eclipse-workspace\SalesManager\target\SalesManager-0.0.1-SNAPSHOT.jar
[INFO]
[INFO] --- spring-boot-maven-plugin:2.4.0:repackage (repackage) @ SalesManager ---
[INFO] Downloading from : https://repo.maven.apache.org/maven2/org/springframework/boot/spring-boot-buildpack-platform/2.4
[INFO] Downloaded from : https://repo.maven.apache.org/maven2/org/springframework/boot/spring-boot-buildpack-platform/2.4.
```



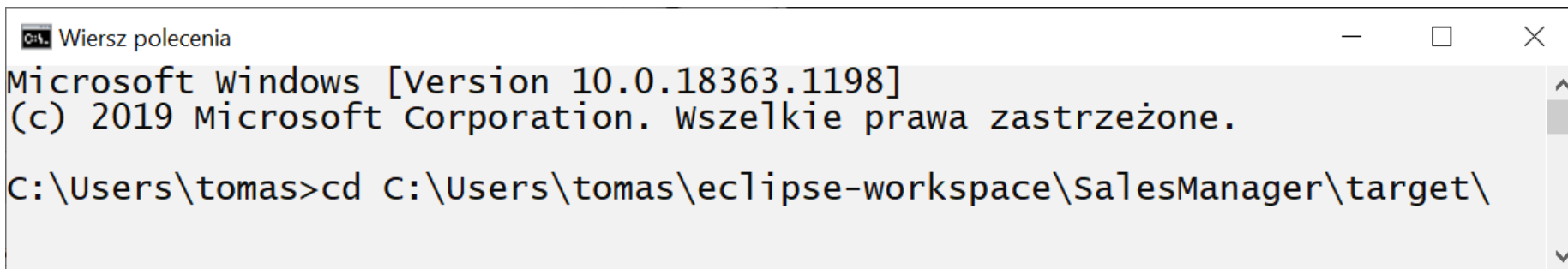
C:\Users\tomas\eclipse-workspace\SalesManager\target\SalesManager-0.0.1-SNAPSHOT.jar

Pakowanie i uruchamianie aplikacji



```
Wiersz polecenia
Microsoft Windows [Version 10.0.18363.1198]
(c) 2019 Microsoft Corporation. Wszelkie prawa zastrzeżone.

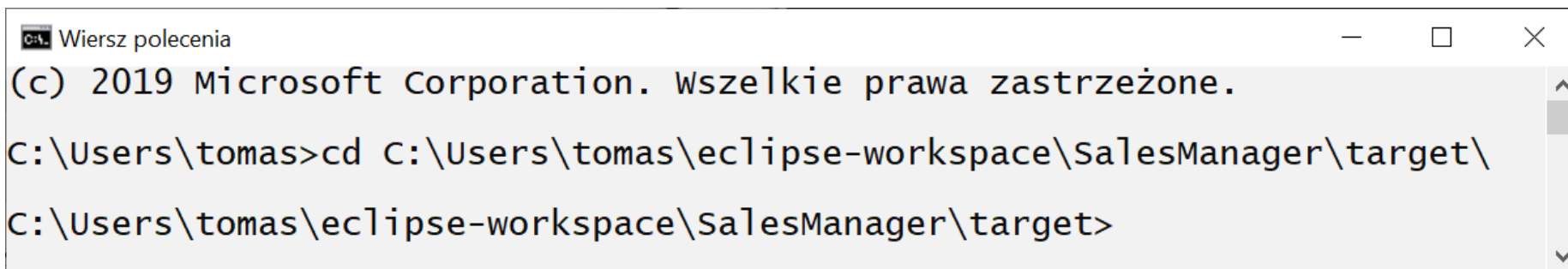
C:\Users\tomas>
```



```
Wiersz polecenia
Microsoft Windows [Version 10.0.18363.1198]
(c) 2019 Microsoft Corporation. Wszelkie prawa zastrzeżone.

C:\Users\tomas>cd C:\Users\tomas\eclipse-workspace\SalesManager\target\

C:\Users\tomas>
```

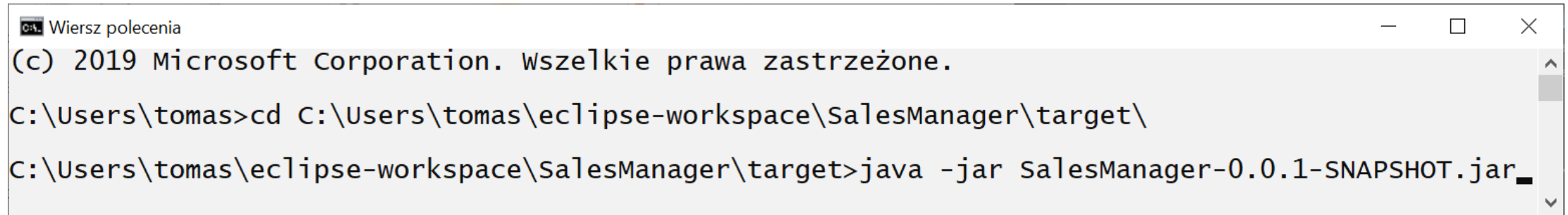


```
Wiersz polecenia
(c) 2019 Microsoft Corporation. Wszelkie prawa zastrzeżone.

C:\Users\tomas>cd C:\Users\tomas\eclipse-workspace\SalesManager\target\

C:\Users\tomas>
```

Pakowanie i uruchamianie aplikacji



```
Wiersz polecenia
(c) 2019 Microsoft Corporation. Wszelkie prawa zastrzeżone.
C:\Users\tomas>cd C:\Users\tomas\eclipse-workspace\SalesManager\target\
C:\Users\tomas\eclipse-workspace\SalesManager\target>java -jar SalesManager-0.0.1-SNAPSHOT.jar
```

Uruchamianie aplikacji w przeglądarce jako localhost:8080 (port 8080)

**Politechnika
Warszawska**

Koniec

